

CRYO Complete Code And Interview Guide

This guide was generated from the current CRYO repository state. It is designed for interview preparation and includes a source appendix so every hand-written source/config line has a visible reference.

Quick Facts

- Project root: `D:\Product Landing Page\cryo`
- GitHub repo: `https://github.com/Penguindrum920/cryo`
- Vercel project name: `cryo`
- Production URL: `https://cryo-iota.vercel.app`
- Framework: Next.js 14 App Router
- Primary technologies: React 18, TypeScript, Tailwind, GSAP, ScrollTrigger, Three.js, React Three Fiber, Drei, model-viewer
- Hand-written source/config files covered here: 40
- Hand-written source/config lines covered in appendix: 5367

How To Explain The Project In 30 Seconds

CRYO is an interactive soda product website. The landing page uses React Three Fiber and GSAP to animate flavour-specific soda cans. A shared flavour data model drives every route, texture, colour, copy block, AR model path, and QR handoff. The flavour page plays a real video, fades it out, keeps one persistent 3D can on screen, and uses scroll progress to rotate the can into a scan-box product profile. The AR page uses model-viewer on mobile for WebXR, Scene Viewer, and Quick Look while desktop users get an orbitable premium Three.js showcase and QR code handoff. Reviews use a typing heading, a scroll-driven keyboard animation adapted from the Nimbus keyboard model, and review cards.

Architecture Map

Area	Files	What to say in an interview
App shell	`src/app/layout.tsx`, `src/app/app.css`, `src/components/SiteNav.tsx`	The shell owns metadata, global font/load behavior, navigation, overscroll control, and global animation CSS.
Product data	`src/data/flavors.ts`	This is the main source of truth for flavour identity, copy, colors, labels, logos, and AR model paths.
Landing	`src/slices/Hero/*`	Landing uses client-side GSAP and React Three Fiber to animate multiple flavour cans, then reveals the flavour picker.
Shared can	`src/components/SodaCan.tsx`	One GLTF can model is reused everywhere, with label textures swapped by flavour id.
Flavour page	`src/slices/Flavours/FlavorExperience.tsx`	The page coordinates autoplay flavour video, a persistent handoff can, scroll-trigger scan box, and flavour details.
AR	`src/slices/AR/*`, `src/app/ar/[slug]/page.tsx`	The AR route serves model-viewer on mobile and a desktop Three.js showcase with QR handoff on desktop.
Reviews	`src/slices/Reviews/*`, `src/components/NimbusKeyboard.tsx`	Reviews combines typing text, a scroll-driven Nimbus keyboard animation, and review cards.
Scripts	`scripts/*.mjs`	Scripts generate AR model variants, check frame/video asset availability, and run a custom dev server.
Deployment	`.vercelignore`, `package.json`, `package-lock.json`	Vercel deployment needed root-anchored ignores and compatible Three.js peer dependencies.

Runtime Data Flow

1. `src/data/flavors.ts` defines all five flavours and their ids, slugs, colours, labels, logos, copy, details, and AR model paths.
2. The landing route renders `Hero`, which stores `activeFlavorId` and passes it into the Three.js hero scene.
3. Hovering/focusing a flavour card updates active flavour state. Clicking routes to `/flavours?flavor=<id>`.
4. The `/flavours` server page resolves the route value with `getFlavorByRouteValue`, then asks `getFlavorVideoSequence` whether the MP4 exists.
5. `FlavorExperience` receives the selected flavour and video sequence. It sets CSS variables for primary/secondary colours.
6. `FrameSequenceVideo` plays the MP4 normally and reports playback progress on every animation frame.
7. Near the end of the video, `PersistentHandoffCan` becomes visible and uses the same selected flavour label texture.
8. Scrolling into the scan section updates `scanProgress` through ScrollTrigger.
9. `PersistentHandoffCanScene` uses that progress to rotate and settle the 3D can while `ScanBox` and `FlavorScanDetails` reveal information.
10. `ARLaunchButton` uses the same flavour object to choose the right AR route and GLTF path.
11. `/ar/[slug]` validates the slug, then `ARExperience` either shows model-viewer on mobile or the desktop showcase plus QR code.
12. `/reviews` renders typing headline, keyboard canvas, and review cards, all animated from one ScrollTrigger timeline.

Feature Walkthrough

Landing Page

- Entry file: `src/app/page.tsx`.
- Main UI: `src/slices/Hero/index.tsx`.
- 3D scene: `src/slices/Hero/Scene.tsx`.
- The page is a long `min-h-[275dvh]` section with a sticky viewport. Scroll progress drives both text and 3D can movement.
- `activeFlavorId` starts at `DEFAULT\_FLAVOR\_ID`, which is Frostbite Berry.
- CSS variables `--flavor-primary`, `--flavor-secondary`, and `--flavor-text` are recalculated from the active flavour.

- `useGSAP` creates text reveal timelines and reduced-motion fallbacks.
- `Scene` positions all five cans apart, then moves them into a clustered composition while rotating the group.
- `Bubbles` adds instanced floating particles using one geometry and one material for performance.
- `FlavorPicker` uses `Link` and hover/focus state so the visual can theme changes before navigation.

Shared Soda Can

- Entry file: `src/components/SodaCan.tsx`.
- The can model is loaded from `/Soda-can.glTF` with Draco decoder support from `/draco/`.
- The base metal material is reused for body and tab.
- `usePreparedLabel` loads the selected flavour's label from `LABEL\_TEXTURES`.
- Texture setup matters: `flipY = false` keeps GLTF UV orientation correct, `SRGBColorSpace` keeps label colour accurate, and wrapping choices prevent seams.
- `getCanMeshes` first tries known mesh names, then traverses the scene as a fallback. This makes the component less brittle if exported mesh names change slightly.

Flavour Page

- Route file: `src/app/flavours/page.tsx`.
- Main file: `src/slices/Flavours/FlavorExperience.tsx`.
- The route accepts either `flavor` or `flavour` for resilience.
- `FrameSequenceShowcase` now uses MP4 video, not frame-scrubbing. This avoids scroll-frame skipping and lets the video play smoothly.
- `CAN\_DROP\_START\_PROGRESS` and `CAN\_DROP\_END\_PROGRESS` decide when the 3D handoff can appears near the end of the video.
- `VIDEO\_FADE\_START\_PROGRESS` and `VIDEO\_FADE\_END\_PROGRESS` fade the video as the persistent can takes over.
- The can is fixed to the right side through `PersistentHandoffCan`, so the viewer perceives continuity from video to scan section.
- `scanProgress` comes from a ScrollTrigger over the scan section. It drives the scan box, can turn, settle scale, and details card.
- `getFlavorIngredients` derives ingredient chips from the flavour pairing text by splitting on `and`.

AR Experience

- Button file: `src/slices/AR/ARLaunchButton.tsx`.
- Route file: `src/app/ar/[slug]/page.tsx`.
- AR page file: `src/slices/AR/ARExperience.tsx`.
- Mobile path: model-viewer, `ar-modes="webxr scene-viewer quick-look"`, fixed scale, floor placement.
- Desktop path: `DesktopShowcase` with OrbitControls, desk surface, flavour lighting, particles, rings, condensation, and shadows.
- QR handoff: `QRCodeSVG` encodes `\${origin}/ar/\${flavor.slug}` so a desktop user can scan into the mobile AR route.
- `autoLaunch` is controlled by `?launch=1` and starts AR after model-viewer reports ready.

Reviews

- Route file: `src/app/reviews/page.tsx`.
- UI file: `src/slices/Reviews/index.tsx`.
- 3D keyboard file: `src/slices/Reviews/Scene.tsx`.
- Keyboard model file: `src/components/NimbusKeyboard.tsx`.
- The heading text is typed by animating a number from 0 to the string length and slicing the heading on every update.
- The visible cursor is a CSS keyframe animation in `src/app/app.css`.
- The keyboard enters early, runs a rotation/translation timeline, triggers a key wave, exits upward, then review cards reveal.
- `ReviewsKeyboardScene` groups keys by columns, collects keycap refs and switch refs, and applies staggered wave animations to positions.

File Coverage Matrix

File	Lines	Interview purpose
`.gitignore`	35	Keeps local dependencies, Next build output, env files, logs, local browser profiles, and Vercel local linkage out of Git.
`.vercelignore`	14	Controls Vercel upload payload. Root-anchored rules skip local caches and `/Assets/` without skipping `public/assets`.
`README.md`	12	Minimal project readme generated with the Next scaffold.
`package.json`	43	Defines scripts and dependency choices: Next, React, Three, R3F, Drei, GSAP, model-viewer, QR code, Tailwind, TypeScript.
`next.config.mjs`	4	Currently empty Next config, meaning the app relies on defaults.
`postcss.config.js`	6	Wires Tailwind and Autoprefixer into CSS processing.
`tailwind.config.js`	21	Scans `src`, sets Orbitron as the sans font, and defines a small drift animation extension.
`tsconfig.json`	26	Strict TypeScript setup with App Router defaults and `@/*` path alias to `src/*`.
`scripts/check-flavor-frames.mjs`	52	Verifies expected frame folders and MP4 videos for all five flavours.
`scripts/dev-server.mjs`	22	Runs Next dev through a Node HTTP server on configurable host/port.
`scripts/generate-ar-models.mjs`	81	Clones the base soda can GLTF into flavour-specific AR model folders and rewires materials/textures.
`src/app/app.css`	78	Global Tailwind imports, dark body baseline, overscroll removal, hero frost background, liquid overlay blend, and review cursor blink keyframes.
`src/app/layout.tsx`	43	Root HTML shell with metadata, scroll restoration reset, Google font links, `SiteNav`, and page children.
`src/app/page.tsx`	5	Landing route that renders the Hero slice.
`src/app/flavours/page.tsx`	29	Server route that parses query params, resolves a flavour, loads its video sequence, and renders `FlavorExperience`.

```
| `src/app/reviews/page.tsx` | 5 | Reviews route that renders the Reviews slice. |
| `src/app/ar/[slug]/page.tsx` | 33 | Static AR route generation for each flavour slug with notFound protection and
optional auto-launch. |
| `src/components/Bounded.tsx` | 23 | Reusable max-width layout wrapper with polymorphic element support. |
| `src/components/CryoLogo.tsx` | 59 | Inline SVG logo component with animated liquid/wave mask behavior. |
| `src/components/FloatingCan.tsx` | 52 | Reusable Drei `Float` wrapper around `SodaCan`. |
| `src/components/NimbusKeyboard.tsx` | 1402 | Large GLTF component for the Nimbus keyboard. It exposes refs for every
important keyboard group and keycap so GSAP can animate them. |
| `src/components/SiteNav.tsx` | 51 | Fixed top navigation with active route styling. |
| `src/components/SodaCan.tsx` | 117 | Reusable can model loader. It loads the GLTF, applies metal material to can
body/tab, applies selected label texture to sleeve, and falls back by traversing mesh names. |
| `src/data/flavorFrameTypes.ts` | 11 | Type definition for frame sequence metadata. |
| `src/data/flavorFrames.ts` | 132 | Discovers exported image frame folders and corresponding MP4 paths from `public`.
|
| `src/data/flavorVideoTypes.ts` | 7 | Type definition for video sequence metadata. |
| `src/data/flavorVideos.ts` | 23 | Checks whether a flavour MP4 exists and returns public path metadata. |
| `src/data/flavors.ts` | 160 | Main data model for five flavours, including slugs, marketing copy, colors, juice
colors, label paths, logos, AR paths, and resolver helpers. |
| `src/hooks/useMediaQuery.ts` | 18 | SSR-safe media query hook using `useSyncExternalStore`. |
| `src/slices/AR/ARExperience.tsx` | 205 | Full AR page. Chooses mobile model-viewer or desktop Three showcase, shows
logo/color/atmosphere, and renders QR code handoff. |
| `src/slices/AR/ARLaunchButton.tsx` | 100 | Inline flavour-page AR launcher that attempts direct mobile AR and falls
back to `/ar/<slug>?launch=1`. |
| `src/slices/AR/DesktopShowcase.tsx` | 279 | Desktop 3D fallback scene with orbit controls, desk surface, flavour
particles, rings, condensation, lighting, and shadows. |
| `src/slices/AR/ModelViewerAR.tsx` | 172 | Model-viewer wrapper with imperative `launchAR`, status messages, auto
launch, AR events, and a visible Place Can button. |
| `src/slices/Flavours/FlavorExperience.tsx` | 748 | Main flavour storytelling page. Manages video playback, can drop
handoff, scroll scan progress, scan box, flavour details, quick links, and helpers. |
| `src/slices/Hero/Bubbles.tsx` | 80 | Instanced floating bubble field around the landing cans. |
| `src/slices/Hero/Scene.tsx` | 358 | React Three Fiber hero scene with multi-can intro, scroll-trigger convergence,
flavour highlight scaling, camera, lights, bubbles, and environment. |
| `src/slices/Hero/index.tsx` | 262 | Client landing UI that manages active flavour state, GSAP text reveal, intro
readiness, 3D canvas, and flavour picker. |
| `src/slices/Reviews/Scene.tsx` | 341 | Reviews 3D scene. Adds mouse-reactive camera and scroll-driven Nimbus key
wave animation. |
| `src/slices/Reviews/index.tsx` | 224 | Reviews page UI. Types the heading, shows blinking cursor, animates keyboard
in/out, and reveals review cards. |
| `src/types/model-viewer.d.ts` | 34 | Extends JSX so TypeScript accepts the custom `<model-viewer>` element and its
AR attributes. |
```

Runtime Asset Inventory

```
| Asset group | Count |
| --- | ---: |
| flavour videos | 5 |
| flavour labels | 5 |
| flavour logos | 5 |
| AR GLTF models | 5 |
| AR binary buffers | 5 |
| Nimbus model assets | 6 |
| HDR environments | 2 |
| source assets folder | 30 |
```

Important distinction: the source appendix below covers hand-written source/config files. Binary assets, generated GLTF numeric data, image frames, MP4s, and `package-lock.json` are not interview source code, but they are explained in the architecture, asset inventory, and script sections. The generated AR model contents are produced by `scripts/generate-ar-models.mjs`, which is included line-by-line.

Build And Deployment Notes

- Local build command: `npm.cmd run build` on Windows or `npm run build` elsewhere.
- Vercel deployment command used: `vercel.cmd deploy --prod --yes --archive=tgz`.
- Vercel project: `cryo`.
- Live alias: `https://cryo-iota.vercel.app`.
- `.vercelignore` is root-anchored. The earlier broad `Assets` ignore skipped `public/assets`, causing production 404s for flavour label images. It was fixed to `Assets/`.
- `three` is pinned to `^0.163.0` because `@google/model-viewer@3.5.0` declares a peer dependency on `three@^0.163.0`. This made clean Vercel installs pass without `--legacy-peer-deps`.
- The Vercel account currently has a direct CLI deployment. Automatic GitHub deploys need the Vercel account to add a GitHub login connection.

Interview Questions And Answers

Why use a shared flavour data file?

Because colour, copy, labels, logos, AR model paths, and routing all need to stay synchronized. A single flavour source of truth avoids mismatched UI state where one part of the site shows Frostbite Berry while another part loads a different label or AR model.

Why use React Three Fiber instead of manual Three.js everywhere?

React Three Fiber lets the 3D scene follow React component structure. The project can reuse `SodaCan`, compose scenes

with JSX, use hooks like `useFrame`, and still access raw Three.js objects when GSAP needs to animate positions and rotations.

Why use GSAP ScrollTrigger?

The project needs scroll progress to drive timelines: landing text, can convergence, flavour scan progress, and reviews keyboard motion. ScrollTrigger gives stable progress, scrub support, start/end control, and cleanup through `revert`/`kill` patterns.

Why switch the flavour intro from frames to video?

Frame scrubbing can skip frames when scroll deltas are large. The user wanted continuous playback, so a normal autoplay video gives smooth motion while the code still derives progress from currentTime to coordinate the can handoff.

How does mobile AR work?

`@google/model-viewer` registers the `` web component. The component receives a flavour-specific GLTF path and supports WebXR, Android Scene Viewer, and iOS Quick Look through `ar-modes`. The code calls `activateAR()` when supported and falls back to the AR route if it cannot launch inline.

How does desktop fallback work?

Desktop does not force true AR. It renders a premium Three.js showcase with OrbitControls, lighting, a desk surface, particles, rings, condensation, and the same flavour label texture. A QR code points to the active flavour's AR route for mobile handoff.

What was the main deployment bug?

The first `.vercelignore` used `Assets`, which was too broad. Vercel treated it as matching nested `public/assets`, so label files were not uploaded. Production returned 404 for `/assets/labels/\*.png`. Root anchoring it to `/Assets/` fixed the upload while still excluding the large source asset folder.

What would you improve next?

- Move the Google font into `next/font` to remove the Next lint warning.
- Add Playwright visual smoke tests for the landing, flavour, review, and AR routes.
- Add automated checks for required public assets before deployment.
- Split the large `NimbusKeyboard.tsx` into generated mesh data and smaller animation-target components if maintainability becomes a priority.
- Connect Vercel to GitHub once the account login connection is configured.

Source-Line Appendix

This appendix is intentionally exhaustive for interview preparation. Every line from the hand-written source/config files listed in the coverage matrix appears below with its file path and line number.

.gitignore

Purpose: Keeps local dependencies, Next build output, env files, logs, local browser profiles, and Vercel local linkage out of Git.

```
0001: # dependencies
0002: /node_modules
0003:
0004: # next.js
0005: /.next/
0006: /out/
0007:
0008: # production
0009: /build
0010:
0011: # misc
0012: .DS_Store
0013: *.pem
0014:
0015: # debug
0016: npm-debug.log*
0017: yarn-debug.log*
0018: yarn-error.log*
0019: *.log
0020:
0021: # local browser/test artifacts
0022: .chrome*/
0023: .codex-screens/
0024:
0025: # local env files
0026: .env*.local
0027:
0028: # vercel
0029: .vercel
```

```
0030:
0031: # typescript
0032: *.tsbuildinfo
0033: next-env.d.ts
0034:
0035: .vercel
```

```
### .vercelignore
```

Purpose: Controls Vercel upload payload. Root-anchored rules skip local caches and `/Assets/` without skipping `public/assets`.

```
0001: /.git
0002: /.next
0003: /node_modules
0004: /out
0005: /build
0006:
0007: # source/design assets are kept in GitHub, runtime assets live under public/
0008: /Assets/
0009:
0010: # local verification artifacts
0011: /.chrome*
0012: /.codex-screens
0013: /*.log
0014: /tsconfig.tsbuildinfo
```

```
### README.md
```

Purpose: Minimal project readme generated with the Next scaffold.

```
0001: # Cryo Landing Page
0002:
0003: Creative hero landing page for the Cryo soda can brand.
0004:
0005: ## Run locally
0006:
0007: bash
0008: npm install
0009: npm run dev
0010:
0011:
0012: Open `http://localhost:3000`.
```

```
### package.json
```

Purpose: Defines scripts and dependency choices: Next, React, Three, R3F, Drei, GSAP, model-viewer, QR code, Tailwind, TypeScript.

```
0001: {
0002:   "name": "cryo",
0003:   "version": "0.1.0",
0004:   "private": true,
0005:   "license": "UNLICENSED",
0006:   "scripts": {
0007:     "ar:models": "node scripts/generate-ar-models.mjs",
0008:     "frames:check": "node scripts/check-flavor-frames.mjs",
0009:     "dev": "next dev",
0010:     "dev:server": "node scripts/dev-server.mjs",
0011:     "build": "next build",
0012:     "start": "next start",
0013:     "lint": "next lint",
0014:     "format": "prettier --write ."
0015:   },
0016:   "dependencies": {
0017:     "@google/model-viewer": "^3.5.0",
0018:     "@gsap/react": "^2.1.1",
0019:     "@react-three/drei": "^9.111.3",
0020:     "@react-three/fiber": "^8.17.6",
0021:     "@types/three": "^0.163.0",
0022:     "clsx": "^2.1.1",
0023:     "gsap": "^3.12.5",
0024:     "next": "14.2.35",
0025:     "qrcode.react": "^4.2.0",
```

```

0026:     "react": "^18.3.1",
0027:     "react-dom": "^18.3.1",
0028:     "three": "^0.163.0"
0029:   },
0030:   "devDependencies": {
0031:     "@types/node": "^20.14.2",
0032:     "@types/react": "^18.3.3",
0033:     "@types/react-dom": "^18.3.0",
0034:     "autoprefixer": "^10.4.20",
0035:     "eslint": "^8.57.0",
0036:     "eslint-config-next": "^14.2.4",
0037:     "postcss": "^8.4.41",
0038:     "prettier": "^3.3.3",
0039:     "prettier-plugin-tailwindcss": "^0.6.6",
0040:     "tailwindcss": "^3.4.10",
0041:     "typescript": "^5.4.5"
0042:   }
0043: }

```

next.config.mjs

Purpose: Currently empty Next config, meaning the app relies on defaults.

```

0001: /** @type {import('next').NextConfig} */
0002: const nextConfig = {};
0003:
0004: export default nextConfig;

```

postcss.config.js

Purpose: Wires Tailwind and Autoprefixer into CSS processing.

```

0001: module.exports = {
0002:   plugins: {
0003:     tailwindcss: {},
0004:     autoprefixer: {},
0005:   },
0006: };

```

tailwind.config.js

Purpose: Scans `src`, sets Orbitron as the sans font, and defines a small drift animation extension.

```

0001: /** @type {import('tailwindcss').Config} */
0002: module.exports = {
0003:   content: ["/src/**/*.{js,ts,jsx,tsx,mdx}"],
0004:   theme: {
0005:     extend: {
0006:       fontFamily: {
0007:         sans: ["Orbitron", "system-ui", "sans-serif"],
0008:       },
0009:       keyframes: {
0010:         drift: {
0011:           "0%, 100%": { transform: "translate3d(0, 0, 0)" },
0012:           "50%": { transform: "translate3d(0, -10px, 0)" },
0013:         },
0014:       },
0015:       animation: {
0016:         drift: "drift 5s ease-in-out infinite",
0017:       },
0018:     },
0019:   },
0020:   plugins: [],
0021: };

```

tsconfig.json

Purpose: Strict TypeScript setup with App Router defaults and `@/\*` path alias to `src/\*`.

```

0001: {
0002:   "compilerOptions": {

```

```

0003:     "lib": ["dom", "dom.iterable", "esnext"],
0004:     "allowJs": true,
0005:     "skipLibCheck": true,
0006:     "strict": true,
0007:     "noEmit": true,
0008:     "esModuleInterop": true,
0009:     "module": "esnext",
0010:     "moduleResolution": "bundler",
0011:     "resolveJsonModule": true,
0012:     "isolatedModules": true,
0013:     "jsx": "preserve",
0014:     "incremental": true,
0015:     "plugins": [
0016:       {
0017:         "name": "next"
0018:       }
0019:     ],
0020:     "paths": {
0021:       "@/*": [".src/*"]
0022:     }
0023:   },
0024:   "include": ["next-env.d.ts", "**/*.ts", "**/*.tsx", ".next/types/**/*.ts"],
0025:   "exclude": ["node_modules"]
0026: }

```

```
### scripts/check-flavor-frames.mjs
```

Purpose: Verifies expected frame folders and MP4 videos for all five flavours.

```

0001: import { existsSync, readdirSync } from "node:fs";
0002: import path from "node:path";
0003:
0004: const publicRoot = path.join(process.cwd(), "public");
0005: const frameRoot = path.join(publicRoot, "flavour-frames");
0006: const videoRoot = path.join(publicRoot, "flavour-videos");
0007: const expectedFolders = [
0008:   ["Frostbite Berry frames", "frostbite-berry.mp4"],
0009:   ["Neon Meltdown frames", "neon-meltdown.mp4"],
0010:   ["Cosmic Crush frames", "cosmic-crush.mp4"],
0011:   ["Midnight Citrus frames", "midnight-citrus.mp4"],
0012:   ["Velvet Frost frames", "velvet-frost.mp4"],
0013: ];
0014: const extensions = new Set([".jpg", ".jpeg", ".png", ".webp"]);
0015:
0016: let failed = false;
0017:
0018: for (const [folder, video] of expectedFolders) {
0019:   const folderPath = path.join(frameRoot, folder);
0020:   const videoPath = path.join(videoRoot, video);
0021:   const frames = existsSync(folderPath)
0022:     ? readdirSync(folderPath)
0023:       .filter((name) => extensions.has(path.extname(name).toLowerCase()))
0024:       .sort((a, b) =>
0025:         a.localeCompare(b, undefined, {
0026:           numeric: true,
0027:           sensitivity: "base",
0028:         })
0029:       )
0030:     : [];
0031:
0032:   const hasVideo = existsSync(videoPath);
0033:   const status = frames.length && hasVideo ? "ok" : "missing";
0034:   console.log(
0035:     `${status.padEnd(7)} ${folder}: ${frames.length} frames, video=${hasVideo ? video : "missing"}`,
0036:   );
0037:
0038:   if (frames.length) {
0039:     console.log(`    first=${frames[0]} last=${frames.at(-1)}`);
0040:   }
0041:
0042:   if (!frames.length || !hasVideo) {
0043:     failed = true;
0044:   }
0045: }
0046:
0047: if (failed) {
0048:   console.error(
0049:     "\nAdd each exported JPG sequence under public/flavour-frames/<Flavor Name> frames/ and matching MP4 under public/flavour-videos/<flavor-slug>.mp4",
0050:   );

```

```
0051: process.exit(1);
0052: }
```

```
### scripts/dev-server.mjs
```

Purpose: Runs Next dev through a Node HTTP server on configurable host/port.

```
0001: import { createServer } from "node:http";
0002: import next from "next";
0003:
0004: const port = Number.parseInt(process.env.PORT ?? "3005", 10);
0005: const hostname = process.env.HOSTNAME ?? "0.0.0.0";
0006:
0007: const app = next({
0008:   dev: true,
0009:   hostname,
0010:   port,
0011:   dir: process.cwd(),
0012: });
0013:
0014: const handle = app.getRequestHandler();
0015:
0016: await app.prepare();
0017:
0018: createServer((req, res) => {
0019:   handle(req, res);
0020: }).listen(port, hostname, () => {
0021:   console.log(`Cryo dev server ready on http://localhost:${port}`);
0022: });
```

```
### scripts/generate-ar-models.mjs
```

Purpose: Clones the base soda can GLTF into flavour-specific AR model folders and rewires materials/textures.

```
0001: import { copyFile, mkdir, readFile, writeFile } from "node:fs/promises";
0002: import path from "node:path";
0003: import { fileURLToPath } from "node:url";
0004:
0005: const __dirname = path.dirname(fileURLToPath(import.meta.url));
0006: const projectRoot = path.resolve(__dirname, "..");
0007: const publicRoot = path.join(projectRoot, "public");
0008: const sourceModelPath = path.join(publicRoot, "Soda-can.gltf");
0009: const sourceBufferPath = path.join(publicRoot, "Soda-can.bin");
0010: const outputRoot = path.join(publicRoot, "ar");
0011:
0012: const flavors = [
0013:   { slug: "frostbite-berry", label: "frostbite-berry.png" },
0014:   { slug: "neon-meltdown", label: "neon-meltdown.png" },
0015:   { slug: "cosmic-crush", label: "cosmic-crush.png" },
0016:   { slug: "midnight-citrus", label: "midnight-citrus.png" },
0017:   { slug: "velvet-frost", label: "velvet-frost.png" },
0018: ];
0019:
0020: const sourceModel = JSON.parse(await readFile(sourceModelPath, "utf8"));
0021:
0022: for (const flavor of flavors) {
0023:   const outputDir = path.join(outputRoot, flavor.slug);
0024:   const outputModel = structuredClone(sourceModel);
0025:
0026:   outputModel.materials = [
0027:     {
0028:       name: "CRYO brushed aluminium",
0029:       pbrMetallicRoughness: {
0030:         baseColorFactor: [0.78, 0.84, 0.88, 1],
0031:         metallicFactor: 1,
0032:         roughnessFactor: 0.26,
0033:       },
0034:     },
0035:     {
0036:       name: `CRYO ${flavor.slug} label`,
0037:       pbrMetallicRoughness: {
0038:         baseColorTexture: { index: 0 },
0039:         metallicFactor: 0.58,
0040:         roughnessFactor: 0.18,
0041:       },
0042:     },
0043:   ];
```



```

0044:   outputModel.samplers = [
0045:     {
0046:       magFilter: 9729,
0047:       minFilter: 9987,
0048:       wrapS: 10497,
0049:       wrapT: 33071,
0050:     },
0051:   ];
0052:   outputModel.images = [
0053:     {
0054:       uri: `../../assets/labels/${flavor.label}`,
0055:     },
0056:   ];
0057:   outputModel.textures = [
0058:     {
0059:       sampler: 0,
0060:       source: 0,
0061:     },
0062:   ];
0063:
0064:   outputModel.meshes[0].primitives[0].material = 0;
0065:   outputModel.meshes[1].primitives[0].material = 0;
0066:   outputModel.meshes[1].primitives[1].material = 1;
0067:
0068:   outputModel.nodes[1] = {
0069:     ...outputModel.nodes[1],
0070:     scale: [0.18, 0.18, 0.18],
0071:   };
0072:
0073:   outputModel.buffers[0].uri = "Soda-can.bin";
0074:
0075:   await mkdir(outputDir, { recursive: true });
0076:   await writeFile(
0077:     path.join(outputDir, "can.gltf"),
0078:     `${JSON.stringify(outputModel, null, 2)}\n`,
0079:   );
0080:   await copyFile(sourceBufferPath, path.join(outputDir, "Soda-can.bin"));
0081: }

```

```

### src/app/app.css

```

Purpose: Global Tailwind imports, dark body baseline, overscroll removal, hero frost background, liquid overlay blend, and review cursor blink keyframes.

```

0001: @tailwind base;
0002: @tailwind components;
0003: @tailwind utilities;
0004:
0005: :root {
0006:   color-scheme: dark;
0007: }
0008:
0009: * {
0010:   box-sizing: border-box;
0011: }
0012:
0013: html {
0014:   overscroll-behavior: none;
0015:   scroll-behavior: smooth;
0016: }
0017:
0018: body {
0019:   min-height: 100vh;
0020:   margin: 0;
0021:   overscroll-behavior: none;
0022:   overflow-x: hidden;
0023:   font-family: "Orbitron", system-ui, sans-serif;
0024:   background: #0b0f14;
0025: }
0026:
0027: button {
0028:   font: inherit;
0029: }
0030:
0031: .cryo-hero-bg {
0032:   background: #0b0f14;
0033: }
0034:
0035: .cryo-second-panel,
0036: .cryo-second-word,

```

```

0037: .cryo-second-copy {
0038:   opacity: 0;
0039: }
0040:
0041: .cryo-second-panel {
0042:   transform: translate3d(0, 48px, 0);
0043: }
0044:
0045: .cryo-frost {
0046:   background-image: radial-gradient(
0047:     circle at 50% 20%,
0048:     rgba(245, 247, 255, 0.08),
0049:     transparent 22rem
0050:   ),
0051:   linear-gradient(rgba(245, 247, 255, 0.06) 1px, transparent 1px),
0052:   linear-gradient(90deg, rgba(245, 247, 255, 0.06) 1px, transparent 1px);
0053:   background-size:
0054:     100% 100%,
0055:     44px 44px,
0056:     44px 44px;
0057:   mask-image: linear-gradient(to bottom, black, transparent 82%);
0058: }
0059:
0060: .liquid-canvas-overlay {
0061:   mix-blend-mode: screen;
0062: }
0063:
0064: .reviews-typing-cursor {
0065:   animation: reviews-cursor-blink 0.78s steps(1, end) infinite;
0066: }
0067:
0068: @keyframes reviews-cursor-blink {
0069:   0%,
0070:   48% {
0071:     opacity: 1;
0072:   }
0073:
0074:   49%,
0075:   100% {
0076:     opacity: 0;
0077:   }
0078: }

```

```

### src/app/layout.tsx

```

Purpose: Root HTML shell with metadata, scroll restoration reset, Google font links, `SiteNav`, and page children.

```

0001: import type { Metadata } from "next";
0002:
0003: import { SiteNav } from "@components/SiteNav";
0004: import "./app.css";
0005:
0006: export const metadata: Metadata = {
0007:   title: "Cryo - Fruit Soda From The Deep Freeze",
0008:   description:
0009:     "A creative animated landing page for Cryo, a cold-wave fruit soda can brand.",
0010: };
0011:
0012: export default function RootLayout({
0013:   children,
0014: }: Readonly<{
0015:   children: React.ReactNode;
0016: }>) {
0017:   return (
0018:     <html lang="en" suppressHydrationWarning>
0019:       <head>
0020:         <script
0021:           dangerouslySetInnerHTML={{
0022:             __html:
0023:               '(()=>{try{if("scrollRestoration"in
history){history.scrollRestoration="manual"}window.scrollTo(0,0)}catch(error){}})();',
0024:           }}
0025:         />
0026:         <link rel="preconnect" href="https://fonts.googleapis.com" />
0027:         <link
0028:           rel="preconnect"
0029:           href="https://fonts.gstatic.com"
0030:           crossOrigin="anonymous"
0031:         />
0032:       </head>

```

```

0033:         href="https://fonts.googleapis.com/css2?family=Orbitron:wght@400..900&display=swap"
0034:         rel="stylesheet"
0035:       />
0036:     </head>
0037:     <body className="font-sans" suppressHydrationWarning>
0038:       <SiteNav />
0039:       {children}
0040:     </body>
0041:   </html>
0042: );
0043: }

```

src/app/page.tsx

Purpose: Landing route that renders the Hero slice.

```

0001: import Hero from "@/slices/Hero";
0002:
0003: export default function Page() {
0004:   return <Hero />;
0005: }

```

src/app/flavours/page.tsx

Purpose: Server route that parses query params, resolves a flavour, loads its video sequence, and renders `FlavorExperience`.

```

0001: import { FlavorExperience } from "@/slices/Flavours/FlavorExperience";
0002: import { getFlavorVideoSequence } from "@/data/flavorVideos";
0003: import { getFlavorByRouteValue } from "@/data/flavors";
0004:
0005: type FlavoursPageProps = {
0006:   searchParams?: {
0007:     flavor?: string | string[];
0008:     flavour?: string | string[];
0009:   };
0010: };
0011:
0012: export default function FlavoursPage({ searchParams }: FlavoursPageProps) {
0013:   const routeValue =
0014:     getSingleValue(searchParams?.flavor) ??
0015:     getSingleValue(searchParams?.flavour);
0016:   const flavor = getFlavorByRouteValue(routeValue);
0017:   const videoSequence = getFlavorVideoSequence(flavor);
0018:
0019:   return (
0020:     <FlavorExperience
0021:       initialFlavorId={flavor.id}
0022:       videoSequence={videoSequence}
0023:     />
0024:   );
0025: }
0026:
0027: function getSingleValue(value: string | string[] | undefined) {
0028:   return Array.isArray(value) ? value[0] : value;
0029: }

```

src/app/reviews/page.tsx

Purpose: Reviews route that renders the Reviews slice.

```

0001: import Reviews from "@/slices/Reviews";
0002:
0003: export default function ReviewsPage() {
0004:   return <Reviews />;
0005: }

```

src/app/ar/[slug]/page.tsx

Purpose: Static AR route generation for each flavour slug with notFound protection and optional auto-launch.

```

0001: import { notFound } from "next/navigation";
0002:
0003: import { ARExperience } from "@/slices/AR/ARExperience";
0004: import { CRYO_FLAVORS, getFlavorBySlug } from "@/data/flavors";
0005:
0006: type ARPageProps = {
0007:   params: {
0008:     slug: string;
0009:   };
0010:   searchParams?: {
0011:     launch?: string | string[];
0012:   };
0013: };
0014:
0015: export function generateStaticParams() {
0016:   return CRYO_FLAVORS.map((flavor) => ({
0017:     slug: flavor.slug,
0018:   }));
0019: }
0020:
0021: export default function ARPage({ params, searchParams }: ARPageProps) {
0022:   const flavor = getFlavorBySlug(params.slug);
0023:
0024:   if (flavor.slug !== params.slug) {
0025:     notFound();
0026:   }
0027:
0028:   const launchValue = Array.isArray(searchParams?.launch)
0029:     ? searchParams?.launch[0]
0030:     : searchParams?.launch;
0031:
0032:   return <ARExperience autoLaunch={launchValue === "1"} flavor={flavor} />;
0033: }

```

src/components/Bounded.tsx

Purpose: Reusable max-width layout wrapper with polymorphic element support.

```

0001: import clsx from "clsx";
0002:
0003: type BoundedProps = {
0004:   as?: React.ElementType;
0005:   className?: string;
0006:   children: React.ReactNode;
0007: };
0008:
0009: export function Bounded({
0010:   as: Comp = "div",
0011:   className,
0012:   children,
0013:   ...restProps
0014: }: BoundedProps) {
0015:   return (
0016:     <Comp
0017:       className={clsx("mx-auto w-full max-w-7xl px-5 md:px-8", className)}
0018:       {...restProps}
0019:     >
0020:       {children}
0021:     </Comp>
0022:   );
0023: }

```

src/components/CryoLogo.tsx

Purpose: Inline SVG logo component with animated liquid/wave mask behavior.

```

0001: import { SVGProps } from "react";
0002: import clsx from "clsx";
0003:
0004: const FIZZI_WAVE_PATH =
0005:   "M45.3-31C24.9-31.7 15.9-26.6 0-16.4V87h342V-16.5l-4 1.7A74 74 0 0 1
297.3-7c-11.9-.3-24.7-5.7-38-11.3-14-5.9-28.6-12-43-12.6-20.4-.9-29.4 4.2-45.3 14.4l-4 1.7A74 74 0 0 1
126.3-7c-11.9-.3-24.7-5.7-38-11.3-14-5.9-28.6-12-43-12.6Z";
0006:
0007: const CRYO_LOGO_PATH =
0008:   "M390.5 2.972C344.056 3.41 334.13 3.746 332.329 4.941C331.135 5.734 321.927 14.619 311.867 24.687C299.25
37.313 293.348 42.587 292.844 41.685C292.441 40.966 291.186 34.78 290.055 27.939C288.923 21.097 287.753 14.038 287.454

```

```

12.25L286.909 9H267.455C252.551 9 248.003 9.292 248.013 10.25C248.021 10.938 251.666 22.075 256.115 35C267.772 68.869
267.055 64.398 262.565 75.234C254.15 95.545 251.674 102.007 252.13 102.463C252.392 102.725 261.341 100.591 272.016
97.72C294.409 91.698 291.876 93.88 298.14 75.21C300.994 66.704 303.07 62.182 304.882 60.521C306.322 59.202 313.613
53.032 321.085 46.811C328.556 40.59 338.456 32.301 343.085 28.391C347.713 24.48 352.405 20.809 353.512 20.233C355.338
19.281 365.081 18.412 429.5 13.451C454.356 11.537 503.468 7.18 507.5 6.532C509.15 6.266 516.273 5.591 523.329
5.03C530.385 4.469 536.91 3.572 537.829 3.036C539.589 2.009 494.754 1.99 390.5 2.972M52.951 10.362C50.448 11.037
46.749 12.61 44.729 13.858C41.109 16.096 33 24.537 33 26.069C33 26.502 25.971 41.176 17.379 58.678C2.259 89.48 0.3
94.699 2.883 97.283C3.805 98.206 15.954 98.5 53.127 98.5H102.154L106.553 88.5C108.972 83 110.963 77.938 110.976
77.25C110.995 76.277 104.271 76 80.623 76H50.246L58.527 57.114C63.082 46.726 67.317 37.276 67.939 36.114C69.053 34.032
69.538 34 100.043 34H131.016L137.008 21.903C140.304 15.249 143 9.624 143 9.403C143 8.301 57.202 9.215 52.951
10.362M145.565 22.263C142.3 29.543 136.172 43.15 131.947 52.5C119.071 80.989 112.728 96.062 113.353 96.686C113.955
97.289 147.561 90.459 151.8 88.872C153.065 88.399 154.542 86.771 155.082 85.256C155.622 83.74 160.7 71.25 166.366
57.5L176.668 32.5L192.336 32C200.953 31.725 208.008 31.725 208.013 32C208.018 32.275 206.375 36.867 204.361
42.204C200.135 53.402 201.69 52.518 183.656 53.963C167.141 55.285 166.516 56.107 172.858 68.165C174.98 72.199 179.755
81.35 183.469 88.5C187.184 95.65 190.589 101.893 191.036 102.374C192.327 103.763 237.471 120.898 237.766
120.111C238.085 119.264 225.11 95.208 210.335 69.25L207.916 65H211.791C217.119 65 226.152 63.093 228.314
61.512C230.301 60.059 243.661 26.025 244.972 19.076C245.739 15.012 245.577 14.423 242.98 11.826L240.154
9H151.5L145.565 22.263M429.5 19.024C422.9 19.42 413.225 20.024 408 20.366C402.775 20.707 391.075 21.445 382
22.005C354.998 23.67 357.035 23.062 346.454 32.619C337.75 40.479 337.011 41.47 332.944 50.717C314.635 92.351 314.599
92.456 317.545 95.402C320.918 98.775 326.443 99.115 372.812 98.805C422.788 98.471 421.055 98.668 430.28 92.297C437.11
87.58 440.305 81.737 448.482 59C452.735 47.175 457.067 35.149 458.108 32.275C460.269 26.308 460.552 20.952 458.8
19.2C457.486 17.886 449.274 17.836 429.5 19.024M415 35.041C415 35.849 399.252 75.636 398.486 76.763C398.166 77.234
367.551 77.686 365.848 77.246C364.418 76.875 364.661 75.798 367.662 69.214C369.568 65.032 373.174 56.636 375.676
50.555C378.177 44.475 380.86 38.263 381.638 36.75L383.051 34H399.026C409.426 34 415 34.363 415 35.041";
0009:
0010: export function CryoLogo(props: SVGProps<SVGSVGElement>) {
0011:   return (
0012:     <svg
0013:       xmlns="http://www.w3.org/2000/svg"
0014:       {...props}
0015:       width="540"
0016:       height="121"
0017:       fill="none"
0018:       viewBox="0 0 540 121"
0019:       className={clsx("group", props.className)}
0020:       aria-labelledby="cryo-logo-title"
0021:     >
0022:       <title id="cryo-logo-title">Cryo</title>
0023:       <g clipPath="url(#cryo-logo-clip)">
0024:         <mask
0025:           id="cryo-logo-wave-mask"
0026:           style={{ maskType: "alpha" }}
0027:           width="1112.5"
0028:           height="164.2"
0029:           x="-16.3"
0030:           y="-43.1"
0031:           maskUnits="userSpaceOnUse"
0032:         >
0033:           <g className="transition-transform duration-500 ease-in-out group-hover:translate-y-[80%]">
0034:             <path
0035:               fill="currentColor"
0036:               className="animate-slide-left"
0037:               d={FIZZI_WAVE_PATH}
0038:               transform="scale(3.253012 1.390805)"
0039:             />
0040:           </g>
0041:         </mask>
0042:         <g fill="currentColor" mask="url(#cryo-logo-wave-mask)">
0043:           <path d={CRYO_LOGO_PATH} />
0044:         </g>
0045:         <path
0046:           d={CRYO_LOGO_PATH}
0047:           stroke="currentColor"
0048:           strokeWidth="4"
0049:           vectorEffect="non-scaling-stroke"
0050:         />
0051:       </g>
0052:       <defs>
0053:         <clipPath id="cryo-logo-clip">
0054:           <path fill="currentColor" d="M0 0h540v121H0z" />
0055:         </clipPath>
0056:       </defs>
0057:     </svg>
0058:   );
0059: }

```

```
### src/components/FloatingCan.tsx
```

Purpose: Reusable Drei `Float` wrapper around `SodaCan`.

```

0001: "use client";
0002:
0003: import { Float } from "@react-three/drei";
0004: import { forwardRef, type ReactNode } from "react";
0005: import { Group } from "three";
0006:
0007: import { SodaCan } from "@components/SodaCan";
0008: import type { CryoFlavorId } from "@data/flavors";
0009:
0010: type FloatingCanProps = React.ComponentProps<"group"> & {
0011:   flavor: CryoFlavorId;
0012:   canScale?: number;
0013:   floatSpeed?: number;
0014:   rotationIntensity?: number;
0015:   floatIntensity?: number;
0016:   floatingRange?: [number, number];
0017:   children?: ReactNode;
0018: };
0019:
0020: const FloatingCan = forwardRef<Group, FloatingCanProps>(
0021:   (
0022:     {
0023:       flavor,
0024:       canScale = 2,
0025:       floatSpeed = 1.45,
0026:       rotationIntensity = 0.65,
0027:       floatIntensity = 0.55,
0028:       floatingRange = [-0.08, 0.08],
0029:       children,
0030:       ...props
0031:     },
0032:     ref,
0033:   ) => {
0034:     return (
0035:       <group ref={ref} {...props}>
0036:         <Float
0037:           speed={floatSpeed}
0038:           rotationIntensity={rotationIntensity}
0039:           floatIntensity={floatIntensity}
0040:           floatingRange={floatingRange}
0041:         >
0042:           {children}
0043:           <SodaCan flavor={flavor} canScale={canScale} />
0044:         </Float>
0045:       </group>
0046:     );
0047:   },
0048: );
0049:
0050: FloatingCan.displayName = "FloatingCan";
0051:
0052: export default FloatingCan;

```

src/components/NimbusKeyboard.tsx

Purpose: Large GLTF component for the Nimbus keyboard. It exposes refs for every important keyboard group and keycap so GSAP can animate them.

```

0001: import * as THREE from "three";
0002: import React, { useRef, forwardRef, useImperativeHandle } from "react";
0003: import { useGLTF, useTexture } from "@react-three/drei";
0004: import { GLTF } from "three-stdlib";
0005:
0006: type GLTFResult = GLTF & {
0007:   nodes: {
0008:     Plate: THREE.Mesh;
0009:     Knob: THREE.Mesh;
0010:     PCB: THREE.Mesh;
0011:     ["625u_Wire001"]: THREE.Mesh;
0012:     Cube005: THREE.Mesh;
0013:     Cube005_1: THREE.Mesh;
0014:     Top_Case: THREE.Mesh;
0015:     Weight: THREE.Mesh;
0016:     Screen: THREE.Mesh;
0017:     K_LCONTROL: THREE.Mesh;
0018:     K_GRAVE: THREE.Mesh;
0019:     K_A: THREE.Mesh;
0020:     K_Q: THREE.Mesh;
0021:     K_ESC: THREE.Mesh;
0022:     K_SPACE: THREE.Mesh;

```

```
0023: K_Z: THREE.Mesh;
0024: K_ARROWLEFT: THREE.Mesh;
0025: K_TAB: THREE.Mesh;
0026: K_ENTER: THREE.Mesh;
0027: K_BACKSPACE: THREE.Mesh;
0028: K_CAPS: THREE.Mesh;
0029: K_LSHIFT: THREE.Mesh;
0030: K_RSHIFT: THREE.Mesh;
0031: K_ARROWDOWN: THREE.Mesh;
0032: K_ARROWRIGHT: THREE.Mesh;
0033: K_LALT: THREE.Mesh;
0034: K_LWIN: THREE.Mesh;
0035: K_RALT: THREE.Mesh;
0036: K_FN: THREE.Mesh;
0037: K_1: THREE.Mesh;
0038: K_2: THREE.Mesh;
0039: K_3: THREE.Mesh;
0040: K_4: THREE.Mesh;
0041: K_5: THREE.Mesh;
0042: K_6: THREE.Mesh;
0043: K_7: THREE.Mesh;
0044: K_8: THREE.Mesh;
0045: K_9: THREE.Mesh;
0046: K_0: THREE.Mesh;
0047: K_DASH: THREE.Mesh;
0048: K_EQUAL: THREE.Mesh;
0049: K_DEL: THREE.Mesh;
0050: K_S: THREE.Mesh;
0051: K_D: THREE.Mesh;
0052: K_F: THREE.Mesh;
0053: K_G: THREE.Mesh;
0054: K_H: THREE.Mesh;
0055: K_J: THREE.Mesh;
0056: K_K: THREE.Mesh;
0057: K_L: THREE.Mesh;
0058: K_SEMICOLON: THREE.Mesh;
0059: K_QUOTE: THREE.Mesh;
0060: K_PAGEDOWN: THREE.Mesh;
0061: K_W: THREE.Mesh;
0062: K_E: THREE.Mesh;
0063: K_R: THREE.Mesh;
0064: K_T: THREE.Mesh;
0065: K_Y: THREE.Mesh;
0066: K_U: THREE.Mesh;
0067: K_I: THREE.Mesh;
0068: K_O: THREE.Mesh;
0069: K_P: THREE.Mesh;
0070: K_LSQUAREBRACKET: THREE.Mesh;
0071: K_RSQUAREBRACKET: THREE.Mesh;
0072: K_PAGEUP: THREE.Mesh;
0073: K_F1: THREE.Mesh;
0074: K_F2: THREE.Mesh;
0075: K_F3: THREE.Mesh;
0076: K_F4: THREE.Mesh;
0077: K_F5: THREE.Mesh;
0078: K_F6: THREE.Mesh;
0079: K_F7: THREE.Mesh;
0080: K_F8: THREE.Mesh;
0081: K_F9: THREE.Mesh;
0082: K_F10: THREE.Mesh;
0083: K_F11: THREE.Mesh;
0084: K_F12: THREE.Mesh;
0085: K_X: THREE.Mesh;
0086: K_C: THREE.Mesh;
0087: K_V: THREE.Mesh;
0088: K_B: THREE.Mesh;
0089: K_N: THREE.Mesh;
0090: K_M: THREE.Mesh;
0091: K_COMMA: THREE.Mesh;
0092: K_PERIOD: THREE.Mesh;
0093: K_SLASH: THREE.Mesh;
0094: K_ARROWUP: THREE.Mesh;
0095: K_END: THREE.Mesh;
0096: K_BACKSLASH: THREE.Mesh;
0097: Switch_Heavy002: THREE.InstancedMesh;
0098: Switch_Heavy002_1: THREE.InstancedMesh;
0099: Switch_Heavy002_2: THREE.InstancedMesh;
0100: Switch_Heavy002_3: THREE.InstancedMesh;
0101: ["2U_Wires"]: THREE.InstancedMesh;
0102: Stab_Housing_Instances: THREE.InstancedMesh;
0103: };
0104: materials: {
0105: PC: THREE.MeshStandardMaterial;
```

```
0106:   Knob: THREE.MeshStandardMaterial;
0107:   PCB_Black: THREE.MeshStandardMaterial;
0108:   Gold: THREE.MeshStandardMaterial;
0109:   Bottom_Case: THREE.MeshStandardMaterial;
0110:   Feet: THREE.MeshStandardMaterial;
0111:   Top_Case: THREE.MeshStandardMaterial;
0112:   Weight: THREE.MeshStandardMaterial;
0113:   Screen: THREE.MeshPhysicalMaterial;
0114:   Keycaps: THREE.MeshPhysicalMaterial;
0115:   Switch_Bottom_Housing: THREE.MeshStandardMaterial;
0116:   Stem: THREE.MeshStandardMaterial;
0117:   Switch_Top_Housing: THREE.MeshStandardMaterial;
0118: };
0119:   animations: THREE.AnimationClip[];
0120: };
0121:
0122: export interface NimbusKeyboardRefs {
0123:   // Main keyboard structure
0124:   plate: React.RefObject<THREE.Mesh | null>;
0125:   topCase: React.RefObject<THREE.Mesh | null>;
0126:   weight: React.RefObject<THREE.Mesh | null>;
0127:   screen: React.RefObject<THREE.Mesh | null>;
0128:   knob: React.RefObject<THREE.Mesh | null>;
0129:
0130:   // Switch groups for wave animation
0131:   switches: {
0132:     functionRow: React.RefObject<THREE.Group | null>;
0133:     numberRow: React.RefObject<THREE.Group | null>;
0134:     topRow: React.RefObject<THREE.Group | null>;
0135:     homeRow: React.RefObject<THREE.Group | null>;
0136:     bottomRow: React.RefObject<THREE.Group | null>;
0137:     modifiers: React.RefObject<THREE.Group | null>;
0138:     arrows: React.RefObject<THREE.Group | null>;
0139:   };
0140:
0141:   // Keycap groups for easy animation targeting
0142:   keycaps: {
0143:     functionRow: React.RefObject<THREE.Group | null>;
0144:     numberRow: React.RefObject<THREE.Group | null>;
0145:     topRow: React.RefObject<THREE.Group | null>;
0146:     homeRow: React.RefObject<THREE.Group | null>;
0147:     bottomRow: React.RefObject<THREE.Group | null>;
0148:     modifiers: React.RefObject<THREE.Group | null>;
0149:     arrows: React.RefObject<THREE.Group | null>;
0150:   };
0151:
0152:   // Individual keycaps for detailed animations
0153:   keys: {
0154:     [key: string]: React.RefObject<THREE.Mesh | null>;
0155:   };
0156:
0157:   // Main container
0158:   container: React.RefObject<THREE.Group | null>;
0159: }
0160:
0161: type NimbusKeyboardProps = JSX.IntrinsicElements["group"] & {
0162:   keycapMaterial?: THREE.Material;
0163:   knobColor?: string;
0164: };
0165:
0166: export const NimbusKeyboard = forwardRef<NimbusKeyboardRefs, NimbusKeyboardProps>({
0167:   ({ keycapMaterial, knobColor, ...props }, ref) => {
0168:     const { nodes, materials } = useGLTF(
0169:       "/nimbus/keyboard.gltf",
0170:       "/draco/",
0171:     ) as unknown as GLTFResult;
0172:
0173:     // Main structure refs
0174:     const containerRef = useRef<THREE.Group>(null);
0175:     const plateRef = useRef<THREE.Mesh>(null);
0176:     const topCaseRef = useRef<THREE.Mesh>(null);
0177:     const weightRef = useRef<THREE.Mesh>(null);
0178:     const screenRef = useRef<THREE.Mesh>(null);
0179:     const knobRef = useRef<THREE.Mesh>(null);
0180:
0181:     // Switch group refs
0182:     const switchFunctionRowRef = useRef<THREE.Group>(null);
0183:     const switchNumberRowRef = useRef<THREE.Group>(null);
0184:     const switchTopRowRef = useRef<THREE.Group>(null);
0185:     const switchHomeRowRef = useRef<THREE.Group>(null);
0186:     const switchBottomRowRef = useRef<THREE.Group>(null);
0187:     const switchModifiersRef = useRef<THREE.Group>(null);
0188:     const switchArrowsRef = useRef<THREE.Group>(null);
```



```
0189:
0190: // Keycap group refs
0191: const functionRowRef = useRef<THREE.Group>(null);
0192: const numberRowRef = useRef<THREE.Group>(null);
0193: const topRowRef = useRef<THREE.Group>(null);
0194: const homeRowRef = useRef<THREE.Group>(null);
0195: const bottomRowRef = useRef<THREE.Group>(null);
0196: const modifiersRef = useRef<THREE.Group>(null);
0197: const arrowsRef = useRef<THREE.Group>(null);
0198:
0199: // Individual key refs
0200: const keyRefs = {
0201:   esc: useRef<THREE.Mesh>(null),
0202:   f1: useRef<THREE.Mesh>(null),
0203:   f2: useRef<THREE.Mesh>(null),
0204:   f3: useRef<THREE.Mesh>(null),
0205:   f4: useRef<THREE.Mesh>(null),
0206:   f5: useRef<THREE.Mesh>(null),
0207:   f6: useRef<THREE.Mesh>(null),
0208:   f7: useRef<THREE.Mesh>(null),
0209:   f8: useRef<THREE.Mesh>(null),
0210:   f9: useRef<THREE.Mesh>(null),
0211:   f10: useRef<THREE.Mesh>(null),
0212:   f11: useRef<THREE.Mesh>(null),
0213:   f12: useRef<THREE.Mesh>(null),
0214:   del: useRef<THREE.Mesh>(null),
0215:   grave: useRef<THREE.Mesh>(null),
0216:   one: useRef<THREE.Mesh>(null),
0217:   two: useRef<THREE.Mesh>(null),
0218:   three: useRef<THREE.Mesh>(null),
0219:   four: useRef<THREE.Mesh>(null),
0220:   five: useRef<THREE.Mesh>(null),
0221:   six: useRef<THREE.Mesh>(null),
0222:   seven: useRef<THREE.Mesh>(null),
0223:   eight: useRef<THREE.Mesh>(null),
0224:   nine: useRef<THREE.Mesh>(null),
0225:   zero: useRef<THREE.Mesh>(null),
0226:   dash: useRef<THREE.Mesh>(null),
0227:   equal: useRef<THREE.Mesh>(null),
0228:   backspace: useRef<THREE.Mesh>(null),
0229:   tab: useRef<THREE.Mesh>(null),
0230:   q: useRef<THREE.Mesh>(null),
0231:   w: useRef<THREE.Mesh>(null),
0232:   e: useRef<THREE.Mesh>(null),
0233:   r: useRef<THREE.Mesh>(null),
0234:   t: useRef<THREE.Mesh>(null),
0235:   y: useRef<THREE.Mesh>(null),
0236:   u: useRef<THREE.Mesh>(null),
0237:   i: useRef<THREE.Mesh>(null),
0238:   o: useRef<THREE.Mesh>(null),
0239:   p: useRef<THREE.Mesh>(null),
0240:   lsquarebracket: useRef<THREE.Mesh>(null),
0241:   rsquarebracket: useRef<THREE.Mesh>(null),
0242:   backslash: useRef<THREE.Mesh>(null),
0243:   pageup: useRef<THREE.Mesh>(null),
0244:   caps: useRef<THREE.Mesh>(null),
0245:   a: useRef<THREE.Mesh>(null),
0246:   s: useRef<THREE.Mesh>(null),
0247:   d: useRef<THREE.Mesh>(null),
0248:   f: useRef<THREE.Mesh>(null),
0249:   g: useRef<THREE.Mesh>(null),
0250:   h: useRef<THREE.Mesh>(null),
0251:   j: useRef<THREE.Mesh>(null),
0252:   k: useRef<THREE.Mesh>(null),
0253:   l: useRef<THREE.Mesh>(null),
0254:   semicolon: useRef<THREE.Mesh>(null),
0255:   quote: useRef<THREE.Mesh>(null),
0256:   enter: useRef<THREE.Mesh>(null),
0257:   pagedown: useRef<THREE.Mesh>(null),
0258:   lshift: useRef<THREE.Mesh>(null),
0259:   z: useRef<THREE.Mesh>(null),
0260:   x: useRef<THREE.Mesh>(null),
0261:   c: useRef<THREE.Mesh>(null),
0262:   v: useRef<THREE.Mesh>(null),
0263:   b: useRef<THREE.Mesh>(null),
0264:   n: useRef<THREE.Mesh>(null),
0265:   m: useRef<THREE.Mesh>(null),
0266:   comma: useRef<THREE.Mesh>(null),
0267:   period: useRef<THREE.Mesh>(null),
0268:   slash: useRef<THREE.Mesh>(null),
0269:   rshift: useRef<THREE.Mesh>(null),
0270:   arrowup: useRef<THREE.Mesh>(null),
0271:   end: useRef<THREE.Mesh>(null),
```

```
0272:         lcontrol: useRef<THREE.Mesh>(null),
0273:         lwin: useRef<THREE.Mesh>(null),
0274:         lalt: useRef<THREE.Mesh>(null),
0275:         space: useRef<THREE.Mesh>(null),
0276:         ralt: useRef<THREE.Mesh>(null),
0277:         fn: useRef<THREE.Mesh>(null),
0278:         arrowleft: useRef<THREE.Mesh>(null),
0279:         arrowdown: useRef<THREE.Mesh>(null),
0280:         arrowright: useRef<THREE.Mesh>(null),
0281:     };
0282:
0283:     // Expose refs through imperative handle
0284:     useImperativeHandle(ref, () => ({
0285:         plate: plateRef,
0286:         topCase: topCaseRef,
0287:         weight: weightRef,
0288:         screen: screenRef,
0289:         knob: knobRef,
0290:         switches: {
0291:             functionRow: switchFunctionRowRef,
0292:             numberRow: switchNumberRowRef,
0293:             topRow: switchTopRowRef,
0294:             homeRow: switchHomeRowRef,
0295:             bottomRow: switchBottomRowRef,
0296:             modifiers: switchModifiersRef,
0297:             arrows: switchArrowsRef,
0298:         },
0299:         keycaps: {
0300:             functionRow: functionRowRef,
0301:             numberRow: numberRowRef,
0302:             topRow: topRowRef,
0303:             homeRow: homeRowRef,
0304:             bottomRow: bottomRowRef,
0305:             modifiers: modifiersRef,
0306:             arrows: arrowsRef,
0307:         },
0308:         keys: keyRefs,
0309:         container: containerRef,
0310:     }));
0311:
0312:     const keycapTexture = useTexture("/nimbus/goodwell_uv.png");
0313:     keycapTexture.flipY = false;
0314:     keycapTexture.colorSpace = THREE.SRGBColorSpace;
0315:
0316:     const knurlTexture = useTexture("/nimbus/Knurl.jpg");
0317:     knurlTexture.flipY = false;
0318:
0319:     knurlTexture.repeat.set(6, 6);
0320:     knurlTexture.wrapS = THREE.RepeatWrapping;
0321:     knurlTexture.wrapT = THREE.RepeatWrapping;
0322:
0323:     const screenTexture = useTexture("/nimbus/screen_uv.png");
0324:     screenTexture.flipY = false;
0325:
0326:     screenTexture.repeat.set(-1, -1);
0327:     screenTexture.offset.set(1, 1);
0328:
0329:     const defaultKeycapMat = new THREE.MeshStandardMaterial({
0330:         roughness: 0.7,
0331:         map: keycapTexture,
0332:     });
0333:
0334:     const keycapMat = keycapMaterial || defaultKeycapMat;
0335:
0336:     const knobMat = new THREE.MeshStandardMaterial({
0337:         color: knobColor || "#E24818",
0338:         roughness: 0.4,
0339:         metalness: 1,
0340:         bumpMap: knurlTexture,
0341:         bumpScale: 0.8,
0342:     });
0343:
0344:     const plateMat = new THREE.MeshStandardMaterial({
0345:         color: "#888888",
0346:         roughness: 0.4,
0347:     });
0348:
0349:     const bottomCaseMat = new THREE.MeshStandardMaterial({
0350:         color: "#1E548A",
0351:         roughness: 0.4,
0352:     });
0353:
0354:     const topCaseMat = new THREE.MeshStandardMaterial({
```

```
0355:         color: "#ddddd",
0356:         roughness: 0.7,
0357:     });
0358:
0359:     const feetMat = new THREE.MeshStandardMaterial({
0360:         color: "#333333",
0361:         roughness: 0.6,
0362:     });
0363:
0364:     const screenMat = new THREE.MeshStandardMaterial({
0365:         map: screenTexture,
0366:         roughness: 0.4,
0367:     });
0368:
0369:     const switchMat = new THREE.MeshStandardMaterial({
0370:         color: "#cccccc",
0371:         roughness: 0.4,
0372:     });
0373:
0374:     const switchStemMat = new THREE.MeshStandardMaterial({
0375:         color: "#bb2222",
0376:         roughness: 0.4,
0377:     });
0378:
0379:     const switchContactsMat = new THREE.MeshStandardMaterial({
0380:         color: "#FFCF5F",
0381:         roughness: 0.1,
0382:         metalness: 1,
0383:     });
0384:
0385:     return (
0386:         <group {...props} dispose={null} ref={containerRef}>
0387:             <group position={[0.02, 0, 0]}>
0388:                 <mesh
0389:                     ref={plateRef}
0390:                     castShadow
0391:                     receiveShadow
0392:                     geometry={nodes.Plate.geometry}
0393:                     material={plateMat}
0394:                     position={[ -0.022, -0.006, -0.057]}
0395:                 />
0396:                 <mesh
0397:                     ref={knobRef}
0398:                     castShadow
0399:                     receiveShadow
0400:                     geometry={nodes.Knob.geometry}
0401:                     material={knobMat}
0402:                     position={[0.121, 0.004, -0.106]}
0403:                 />
0404:                 <mesh
0405:                     castShadow
0406:                     receiveShadow
0407:                     geometry={nodes.PCB.geometry}
0408:                     material={plateMat}
0409:                     position={[ -0.022, -0.009, -0.057]}
0410:                 />
0411:
0412:                 { /* Switches - organized by rows with individual meshes for animation */ }
0413:                 { /* Function Row Switches */ }
0414:                 <group ref={switchFunctionRowRef}>
0415:                     [
0416:                         -0.165, -0.145, -0.126, -0.107, -0.088, -0.069, -0.05, -0.031,
0417:                         -0.012, 0.007, 0.026, 0.045, 0.064,
0418:                     ].map((x, i) => (
0419:                         <group key={`switch-f-${i}`} position={[x, -0.002, -0.106]}>
0420:                             <mesh
0421:                                 castShadow
0422:                                 receiveShadow
0423:                                 geometry={nodes.Switch_Heavy002.geometry}
0424:                                 material={switchMat}
0425:                             />
0426:                             <mesh
0427:                                 castShadow
0428:                                 receiveShadow
0429:                                 geometry={nodes.Switch_Heavy002_1.geometry}
0430:                                 material={switchContactsMat}
0431:                             />
0432:                             <mesh
0433:                                 castShadow
0434:                                 receiveShadow
0435:                                 geometry={nodes.Switch_Heavy002_2.geometry}
0436:                                 material={switchStemMat}
0437:                             />
```

```

0438:         <mesh
0439:             castShadow
0440:             receiveShadow
0441:             geometry={nodes.Switch_Heavy002_3.geometry}
0442:             material={switchMat}
0443:         />
0444:     </group>
0445: )}}
0446: </group>
0447:
0448: {/* Number Row Switches */}
0449: <group ref={switchNumberRowRef}>
0450:     {[
0451:         -0.165, -0.146, -0.127, -0.108, -0.089, -0.07, -0.051, -0.032,
0452:         -0.013, 0.006, 0.025, 0.044, 0.063, 0.092, 0.121,
0453:     ]}.map((x, i) => (
0454:         <group key={`switch-n-${i}`} position={[x, -0.002, -0.087]}>
0455:             <mesh
0456:                 castShadow
0457:                 receiveShadow
0458:                 geometry={nodes.Switch_Heavy002.geometry}
0459:                 material={switchMat}
0460:             />
0461:             <mesh
0462:                 castShadow
0463:                 receiveShadow
0464:                 geometry={nodes.Switch_Heavy002_1.geometry}
0465:                 material={switchContactsMat}
0466:             />
0467:             <mesh
0468:                 castShadow
0469:                 receiveShadow
0470:                 geometry={nodes.Switch_Heavy002_2.geometry}
0471:                 material={switchStemMat}
0472:             />
0473:             <mesh
0474:                 castShadow
0475:                 receiveShadow
0476:                 geometry={nodes.Switch_Heavy002_3.geometry}
0477:                 material={switchMat}
0478:             />
0479:         </group>
0480:     )}}
0481: </group>
0482:
0483: {/* Top Row Switches */}
0484: <group ref={switchTopRowRef}>
0485:     {[
0486:         -0.16, -0.136, -0.117, -0.098, -0.079, -0.06, -0.041, -0.022,
0487:         -0.003, 0.016, 0.035, 0.054, 0.073, 0.097, 0.121,
0488:     ]}.map((x, i) => (
0489:         <group key={`switch-t-${i}`} position={[x, -0.002, -0.068]}>
0490:             <mesh
0491:                 castShadow
0492:                 receiveShadow
0493:                 geometry={nodes.Switch_Heavy002.geometry}
0494:                 material={switchMat}
0495:             />
0496:             <mesh
0497:                 castShadow
0498:                 receiveShadow
0499:                 geometry={nodes.Switch_Heavy002_1.geometry}
0500:                 material={switchContactsMat}
0501:             />
0502:             <mesh
0503:                 castShadow
0504:                 receiveShadow
0505:                 geometry={nodes.Switch_Heavy002_2.geometry}
0506:                 material={switchStemMat}
0507:             />
0508:             <mesh
0509:                 castShadow
0510:                 receiveShadow
0511:                 geometry={nodes.Switch_Heavy002_3.geometry}
0512:                 material={switchMat}
0513:             />
0514:         </group>
0515:     )}}
0516: </group>
0517:
0518: {/* Home Row Switches */}
0519: <group ref={switchHomeRowRef}>
0520:     {[

```

```

0521:         -0.158, -0.132, -0.113, -0.094, -0.075, -0.056, -0.037, -0.018,
0522:         0.001, 0.02, 0.039, 0.058, 0.09, 0.121,
0523:     ].map((x, i) => (
0524:         <group key={`switch-h-${i}`} position=[[x, -0.002, -0.049]]>
0525:         <mesh
0526:             castShadow
0527:             receiveShadow
0528:             geometry={nodes.Switch_Heavy002.geometry}
0529:             material={switchMat}
0530:         />
0531:         <mesh
0532:             castShadow
0533:             receiveShadow
0534:             geometry={nodes.Switch_Heavy002_1.geometry}
0535:             material={switchContactsMat}
0536:         />
0537:         <mesh
0538:             castShadow
0539:             receiveShadow
0540:             geometry={nodes.Switch_Heavy002_2.geometry}
0541:             material={switchStemMat}
0542:         />
0543:         <mesh
0544:             castShadow
0545:             receiveShadow
0546:             geometry={nodes.Switch_Heavy002_3.geometry}
0547:             material={switchMat}
0548:         />
0549:     </group>
0550: )))
0551: </group>
0552:
0553: {/* Bottom Row Switches */}
0554: <group ref={switchBottomRowRef}>
0555:     {[
0556:         -0.153, -0.122, -0.103, -0.084, -0.065, -0.046, -0.027, -0.008,
0557:         0.011, 0.03, 0.049, 0.076, 0.121,
0558:     ]}.map((x, i) => (
0559:         <group key={`switch-b-${i}`} position=[[x, 0.0, -0.03]]>
0560:         <mesh
0561:             castShadow
0562:             receiveShadow
0563:             geometry={nodes.Switch_Heavy002.geometry}
0564:             material={switchMat}
0565:         />
0566:         <mesh
0567:             castShadow
0568:             receiveShadow
0569:             geometry={nodes.Switch_Heavy002_1.geometry}
0570:             material={switchContactsMat}
0571:         />
0572:         <mesh
0573:             castShadow
0574:             receiveShadow
0575:             geometry={nodes.Switch_Heavy002_2.geometry}
0576:             material={switchStemMat}
0577:         />
0578:         <mesh
0579:             castShadow
0580:             receiveShadow
0581:             geometry={nodes.Switch_Heavy002_3.geometry}
0582:             material={switchMat}
0583:         />
0584:     </group>
0585:     )))
0586: </group>
0587:
0588: {/* Modifier Switches */}
0589: <group ref={switchModifiersRef}>
0590:     {[
0591:         [-0.162, -0.011],
0592:         [-0.139, -0.011],
0593:         [-0.115, -0.011],
0594:         [-0.043, -0.01], // Space key
0595:         [0.028, -0.011],
0596:         [0.052, -0.011],
0597:     ]}.map(([x, z], i) => (
0598:         <group key={`switch-m-${i}`} position=[[x, -0.002, z]]>
0599:         <mesh
0600:             castShadow
0601:             receiveShadow
0602:             geometry={nodes.Switch_Heavy002.geometry}
0603:             material={switchMat}

```

```

0604:         />
0605:     <mesh
0606:         castShadow
0607:         receiveShadow
0608:         geometry={nodes.Switch_Heavy002_1.geometry}
0609:         material={switchContactsMat}
0610:     />
0611:     <mesh
0612:         castShadow
0613:         receiveShadow
0614:         geometry={nodes.Switch_Heavy002_2.geometry}
0615:         material={switchStemMat}
0616:     />
0617:     <mesh
0618:         castShadow
0619:         receiveShadow
0620:         geometry={nodes.Switch_Heavy002_3.geometry}
0621:         material={switchMat}
0622:     />
0623: </group>
0624: )))
0625: </group>
0626:
0627: {/* Arrow Switches */}
0628: <group ref={switchArrowsRef}>
0629:     {[
0630:         [0.102, -0.03],
0631:         [0.083, -0.011],
0632:         [0.102, -0.011],
0633:         [0.121, -0.011],
0634:     ]}.map(([x, z], i) => (
0635:         <group key={`switch-a-${i}`} position={[x, -0.002, z]}>
0636:             <mesh
0637:                 castShadow
0638:                 receiveShadow
0639:                 geometry={nodes.Switch_Heavy002.geometry}
0640:                 material={switchMat}
0641:             />
0642:             <mesh
0643:                 castShadow
0644:                 receiveShadow
0645:                 geometry={nodes.Switch_Heavy002_1.geometry}
0646:                 material={switchContactsMat}
0647:             />
0648:             <mesh
0649:                 castShadow
0650:                 receiveShadow
0651:                 geometry={nodes.Switch_Heavy002_2.geometry}
0652:                 material={switchStemMat}
0653:             />
0654:             <mesh
0655:                 castShadow
0656:                 receiveShadow
0657:                 geometry={nodes.Switch_Heavy002_3.geometry}
0658:                 material={switchMat}
0659:             />
0660:         </group>
0661:     ]})
0662: </group>
0663:
0664: <mesh
0665:     castShadow
0666:     receiveShadow
0667:     geometry={nodes["625u_Wire001"].geometry}
0668:     material={materials.Gold}
0669:     position=[[-0.043, -0.001, -0.014]]
0670:     rotation=[Math.PI, 0, Math.PI]}
0671: />
0672: <group position=[[-0.022, -0.014, -0.057]]>
0673:     <mesh
0674:         castShadow
0675:         receiveShadow
0676:         geometry={nodes.Cube005.geometry}
0677:         material={bottomCaseMat}
0678:     />
0679:     <mesh
0680:         castShadow
0681:         receiveShadow
0682:         geometry={nodes.Cube005_1.geometry}
0683:         material={feetMat}
0684:     />
0685: </group>
0686: <mesh

```

```

0687:         ref={topCaseRef}
0688:         castShadow
0689:         receiveShadow
0690:         geometry={nodes.Top_Case.geometry}
0691:         material={topCaseMat}
0692:         position={[-0.022, -0.014, -0.057]}
0693:     />
0694:     <mesh
0695:         ref={weightRef}
0696:         castShadow
0697:         receiveShadow
0698:         geometry={nodes.Weight.geometry}
0699:         material={keycapMat}
0700:         position={[-0.022, -0.014, -0.057]}
0701:     />
0702:     <mesh
0703:         ref={screenRef}
0704:         castShadow
0705:         receiveShadow
0706:         geometry={nodes.Screen.geometry}
0707:         material={screenMat}
0708:         position={[0.092, 0.001, -0.106]}
0709:         scale={-1}
0710:     />
0711:
0712:     { /* Function Row */ }
0713:     <group ref={functionRowRef}>
0714:         <mesh
0715:             ref={keyRefs.esc}
0716:             castShadow
0717:             receiveShadow
0718:             geometry={nodes.K_ESC.geometry}
0719:             material={keycapMat}
0720:             position={[-0.051, 0.01, -0.106]}
0721:         />
0722:         <mesh
0723:             ref={keyRefs.f1}
0724:             castShadow
0725:             receiveShadow
0726:             geometry={nodes.K_F1.geometry}
0727:             material={keycapMat}
0728:             position={[-0.051, 0.01, -0.106]}
0729:         />
0730:         <mesh
0731:             ref={keyRefs.f2}
0732:             castShadow
0733:             receiveShadow
0734:             geometry={nodes.K_F2.geometry}
0735:             material={keycapMat}
0736:             position={[-0.051, 0.01, -0.106]}
0737:         />
0738:         <mesh
0739:             ref={keyRefs.f3}
0740:             castShadow
0741:             receiveShadow
0742:             geometry={nodes.K_F3.geometry}
0743:             material={keycapMat}
0744:             position={[-0.051, 0.01, -0.106]}
0745:         />
0746:         <mesh
0747:             ref={keyRefs.f4}
0748:             castShadow
0749:             receiveShadow
0750:             geometry={nodes.K_F4.geometry}
0751:             material={keycapMat}
0752:             position={[-0.051, 0.01, -0.106]}
0753:         />
0754:         <mesh
0755:             ref={keyRefs.f5}
0756:             castShadow
0757:             receiveShadow
0758:             geometry={nodes.K_F5.geometry}
0759:             material={keycapMat}
0760:             position={[-0.051, 0.01, -0.106]}
0761:         />
0762:         <mesh
0763:             ref={keyRefs.f6}
0764:             castShadow
0765:             receiveShadow
0766:             geometry={nodes.K_F6.geometry}
0767:             material={keycapMat}
0768:             position={[-0.051, 0.01, -0.106]}
0769:         />

```

```

0770:      <mesh
0771:        ref={keyRefs.f7}
0772:        castShadow
0773:        receiveShadow
0774:        geometry={nodes.K_F7.geometry}
0775:        material={keycapMat}
0776:        position={[-0.051, 0.01, -0.106]}
0777:      />
0778:      <mesh
0779:        ref={keyRefs.f8}
0780:        castShadow
0781:        receiveShadow
0782:        geometry={nodes.K_F8.geometry}
0783:        material={keycapMat}
0784:        position={[-0.051, 0.01, -0.106]}
0785:      />
0786:      <mesh
0787:        ref={keyRefs.f9}
0788:        castShadow
0789:        receiveShadow
0790:        geometry={nodes.K_F9.geometry}
0791:        material={keycapMat}
0792:        position={[-0.051, 0.01, -0.106]}
0793:      />
0794:      <mesh
0795:        ref={keyRefs.f10}
0796:        castShadow
0797:        receiveShadow
0798:        geometry={nodes.K_F10.geometry}
0799:        material={keycapMat}
0800:        position={[-0.051, 0.01, -0.106]}
0801:      />
0802:      <mesh
0803:        ref={keyRefs.f11}
0804:        castShadow
0805:        receiveShadow
0806:        geometry={nodes.K_F11.geometry}
0807:        material={keycapMat}
0808:        position={[-0.051, 0.01, -0.106]}
0809:      />
0810:      <mesh
0811:        ref={keyRefs.f12}
0812:        castShadow
0813:        receiveShadow
0814:        geometry={nodes.K_F12.geometry}
0815:        material={keycapMat}
0816:        position={[-0.051, 0.01, -0.106]}
0817:      />
0818:      <mesh
0819:        ref={keyRefs.del}
0820:        castShadow
0821:        receiveShadow
0822:        geometry={nodes.K_DEL.geometry}
0823:        material={keycapMat}
0824:        position={[-0.165, 0.01, -0.087]}
0825:      />
0826:    </group>
0827:
0828:    {/* Number Row */}
0829:    <group ref={numberRowRef}>
0830:      <mesh
0831:        ref={keyRefs.grave}
0832:        castShadow
0833:        receiveShadow
0834:        geometry={nodes.K_GRAVE.geometry}
0835:        material={keycapMat}
0836:        position={[-0.165, 0.01, -0.087]}
0837:      />
0838:      <mesh
0839:        ref={keyRefs.one}
0840:        castShadow
0841:        receiveShadow
0842:        geometry={nodes.K_1.geometry}
0843:        material={keycapMat}
0844:        position={[-0.165, 0.01, -0.087]}
0845:      />
0846:      <mesh
0847:        ref={keyRefs.two}
0848:        castShadow
0849:        receiveShadow
0850:        geometry={nodes.K_2.geometry}
0851:        material={keycapMat}
0852:        position={[-0.165, 0.01, -0.087]}

```



```
0853:      />
0854:      <mesh
0855:          ref={keyRefs.three}
0856:          castShadow
0857:          receiveShadow
0858:          geometry={nodes.K_3.geometry}
0859:          material={keycapMat}
0860:          position={[-0.165, 0.01, -0.087]}
0861:      />
0862:      <mesh
0863:          ref={keyRefs.four}
0864:          castShadow
0865:          receiveShadow
0866:          geometry={nodes.K_4.geometry}
0867:          material={keycapMat}
0868:          position={[-0.165, 0.01, -0.087]}
0869:      />
0870:      <mesh
0871:          ref={keyRefs.five}
0872:          castShadow
0873:          receiveShadow
0874:          geometry={nodes.K_5.geometry}
0875:          material={keycapMat}
0876:          position={[-0.165, 0.01, -0.087]}
0877:      />
0878:      <mesh
0879:          ref={keyRefs.six}
0880:          castShadow
0881:          receiveShadow
0882:          geometry={nodes.K_6.geometry}
0883:          material={keycapMat}
0884:          position={[-0.165, 0.01, -0.087]}
0885:      />
0886:      <mesh
0887:          ref={keyRefs.seven}
0888:          castShadow
0889:          receiveShadow
0890:          geometry={nodes.K_7.geometry}
0891:          material={keycapMat}
0892:          position={[-0.165, 0.01, -0.087]}
0893:      />
0894:      <mesh
0895:          ref={keyRefs.eight}
0896:          castShadow
0897:          receiveShadow
0898:          geometry={nodes.K_8.geometry}
0899:          material={keycapMat}
0900:          position={[-0.165, 0.01, -0.087]}
0901:      />
0902:      <mesh
0903:          ref={keyRefs.nine}
0904:          castShadow
0905:          receiveShadow
0906:          geometry={nodes.K_9.geometry}
0907:          material={keycapMat}
0908:          position={[-0.165, 0.01, -0.087]}
0909:      />
0910:      <mesh
0911:          ref={keyRefs.zero}
0912:          castShadow
0913:          receiveShadow
0914:          geometry={nodes.K_0.geometry}
0915:          material={keycapMat}
0916:          position={[-0.165, 0.01, -0.087]}
0917:      />
0918:      <mesh
0919:          ref={keyRefs.dash}
0920:          castShadow
0921:          receiveShadow
0922:          geometry={nodes.K_DASH.geometry}
0923:          material={keycapMat}
0924:          position={[-0.165, 0.01, -0.087]}
0925:      />
0926:      <mesh
0927:          ref={keyRefs.equal}
0928:          castShadow
0929:          receiveShadow
0930:          geometry={nodes.K_EQUAL.geometry}
0931:          material={keycapMat}
0932:          position={[-0.165, 0.01, -0.087]}
0933:      />
0934:      <mesh
0935:          ref={keyRefs.backspace}
```

```

0936:         castShadow
0937:         receiveShadow
0938:         geometry={nodes.K_BACKSPACE.geometry}
0939:         material={keycapMat}
0940:         position={[0.092, 0, -0.087]}
0941:     />
0942: </group>
0943:
0944: {/* Top Row (QWERTY) */}
0945: <group ref={topRowRef}>
0946:     <mesh
0947:         ref={keyRefs.tab}
0948:         castShadow
0949:         receiveShadow
0950:         geometry={nodes.K_TAB.geometry}
0951:         material={keycapMat}
0952:         position={[-0.16, 0.008, -0.068]}
0953:     />
0954:     <mesh
0955:         ref={keyRefs.q}
0956:         castShadow
0957:         receiveShadow
0958:         geometry={nodes.K_Q.geometry}
0959:         material={keycapMat}
0960:         position={[-0.136, 0.008, -0.068]}
0961:     />
0962:     <mesh
0963:         ref={keyRefs.w}
0964:         castShadow
0965:         receiveShadow
0966:         geometry={nodes.K_W.geometry}
0967:         material={keycapMat}
0968:         position={[-0.136, 0.008, -0.068]}
0969:     />
0970:     <mesh
0971:         ref={keyRefs.e}
0972:         castShadow
0973:         receiveShadow
0974:         geometry={nodes.K_E.geometry}
0975:         material={keycapMat}
0976:         position={[-0.136, 0.008, -0.068]}
0977:     />
0978:     <mesh
0979:         ref={keyRefs.r}
0980:         castShadow
0981:         receiveShadow
0982:         geometry={nodes.K_R.geometry}
0983:         material={keycapMat}
0984:         position={[-0.136, 0.008, -0.068]}
0985:     />
0986:     <mesh
0987:         ref={keyRefs.t}
0988:         castShadow
0989:         receiveShadow
0990:         geometry={nodes.K_T.geometry}
0991:         material={keycapMat}
0992:         position={[-0.136, 0.008, -0.068]}
0993:     />
0994:     <mesh
0995:         ref={keyRefs.y}
0996:         castShadow
0997:         receiveShadow
0998:         geometry={nodes.K_Y.geometry}
0999:         material={keycapMat}
1000:         position={[-0.136, 0.008, -0.068]}
1001:     />
1002:     <mesh
1003:         ref={keyRefs.u}
1004:         castShadow
1005:         receiveShadow
1006:         geometry={nodes.K_U.geometry}
1007:         material={keycapMat}
1008:         position={[-0.136, 0.008, -0.068]}
1009:     />
1010:     <mesh
1011:         ref={keyRefs.i}
1012:         castShadow
1013:         receiveShadow
1014:         geometry={nodes.K_I.geometry}
1015:         material={keycapMat}
1016:         position={[-0.136, 0.008, -0.068]}
1017:     />
1018:     <mesh

```

```

1019:         ref={keyRefs.o}
1020:         castShadow
1021:         receiveShadow
1022:         geometry={nodes.K_O.geometry}
1023:         material={keycapMat}
1024:         position={[-0.136, 0.008, -0.068]}
1025:     />
1026:     <mesh
1027:       ref={keyRefs.p}
1028:       castShadow
1029:       receiveShadow
1030:       geometry={nodes.K_P.geometry}
1031:       material={keycapMat}
1032:       position={[-0.136, 0.008, -0.068]}
1033:     />
1034:     <mesh
1035:       ref={keyRefs.lsquarebracket}
1036:       castShadow
1037:       receiveShadow
1038:       geometry={nodes.K_LSQUAREBRACKET.geometry}
1039:       material={keycapMat}
1040:       position={[-0.136, 0.008, -0.068]}
1041:     />
1042:     <mesh
1043:       ref={keyRefs.rsquarebracket}
1044:       castShadow
1045:       receiveShadow
1046:       geometry={nodes.K_RSQUAREBRACKET.geometry}
1047:       material={keycapMat}
1048:       position={[-0.136, 0.008, -0.068]}
1049:     />
1050:     <mesh
1051:       ref={keyRefs.backslash}
1052:       castShadow
1053:       receiveShadow
1054:       geometry={nodes.K_BACKSLASH.geometry}
1055:       material={keycapMat}
1056:       position={[-0.16, 0.008, -0.068]}
1057:     />
1058:     <mesh
1059:       ref={keyRefs.pageup}
1060:       castShadow
1061:       receiveShadow
1062:       geometry={nodes.K_PAGEUP.geometry}
1063:       material={keycapMat}
1064:       position={[-0.136, 0.008, -0.068]}
1065:     />
1066:   </group>
1067:
1068:   { /* Home Row (ASDF) */ }
1069:   <group ref={homeRowRef}>
1070:     <mesh
1071:       ref={keyRefs.caps}
1072:       castShadow
1073:       receiveShadow
1074:       geometry={nodes.K_CAPS.geometry}
1075:       material={keycapMat}
1076:       position={[-0.158, 0, -0.049]}
1077:     />
1078:     <mesh
1079:       ref={keyRefs.a}
1080:       castShadow
1081:       receiveShadow
1082:       geometry={nodes.K_A.geometry}
1083:       material={keycapMat}
1084:       position={[-0.132, 0.007, -0.049]}
1085:     />
1086:     <mesh
1087:       ref={keyRefs.s}
1088:       castShadow
1089:       receiveShadow
1090:       geometry={nodes.K_S.geometry}
1091:       material={keycapMat}
1092:       position={[-0.132, 0.007, -0.049]}
1093:     />
1094:     <mesh
1095:       ref={keyRefs.d}
1096:       castShadow
1097:       receiveShadow
1098:       geometry={nodes.K_D.geometry}
1099:       material={keycapMat}
1100:       position={[-0.132, 0.007, -0.049]}
1101:     />

```

```

1102:      <mesh
1103:          ref={keyRefs.f}
1104:          castShadow
1105:          receiveShadow
1106:          geometry={nodes.K_F.geometry}
1107:          material={keycapMat}
1108:          position={[-0.132, 0.007, -0.049]}
1109:      />
1110:      <mesh
1111:          ref={keyRefs.g}
1112:          castShadow
1113:          receiveShadow
1114:          geometry={nodes.K_G.geometry}
1115:          material={keycapMat}
1116:          position={[-0.132, 0.007, -0.049]}
1117:      />
1118:      <mesh
1119:          ref={keyRefs.h}
1120:          castShadow
1121:          receiveShadow
1122:          geometry={nodes.K_H.geometry}
1123:          material={keycapMat}
1124:          position={[-0.132, 0.007, -0.049]}
1125:      />
1126:      <mesh
1127:          ref={keyRefs.j}
1128:          castShadow
1129:          receiveShadow
1130:          geometry={nodes.K_J.geometry}
1131:          material={keycapMat}
1132:          position={[-0.132, 0.007, -0.049]}
1133:      />
1134:      <mesh
1135:          ref={keyRefs.k}
1136:          castShadow
1137:          receiveShadow
1138:          geometry={nodes.K_K.geometry}
1139:          material={keycapMat}
1140:          position={[-0.132, 0.007, -0.049]}
1141:      />
1142:      <mesh
1143:          ref={keyRefs.l}
1144:          castShadow
1145:          receiveShadow
1146:          geometry={nodes.K_L.geometry}
1147:          material={keycapMat}
1148:          position={[-0.132, 0.007, -0.049]}
1149:      />
1150:      <mesh
1151:          ref={keyRefs.semicolon}
1152:          castShadow
1153:          receiveShadow
1154:          geometry={nodes.K_SEMICOLON.geometry}
1155:          material={keycapMat}
1156:          position={[-0.132, 0.007, -0.049]}
1157:      />
1158:      <mesh
1159:          ref={keyRefs.quote}
1160:          castShadow
1161:          receiveShadow
1162:          geometry={nodes.K_QUOTE.geometry}
1163:          material={keycapMat}
1164:          position={[-0.132, 0.007, -0.049]}
1165:      />
1166:      <mesh
1167:          ref={keyRefs.enter}
1168:          castShadow
1169:          receiveShadow
1170:          geometry={nodes.K_ENTER.geometry}
1171:          material={keycapMat}
1172:          position={[0.09, 0, -0.049]}
1173:      />
1174:      <mesh
1175:          ref={keyRefs.pagedown}
1176:          castShadow
1177:          receiveShadow
1178:          geometry={nodes.K_PAGEDOWN.geometry}
1179:          material={keycapMat}
1180:          position={[-0.132, 0.007, -0.049]}
1181:      />
1182:  </group>
1183:
1184:  { /* Bottom Row (ZXCv) */ }

```

```
1185:         <group ref={bottomRowRef}>
1186:             <mesh
1187:                 ref={keyRefs.lshift}
1188:                 castShadow
1189:                 receiveShadow
1190:                 geometry={nodes.K_LSHIFT.geometry}
1191:                 material={keycapMat}
1192:                 position={[ -0.153, 0, -0.03]}
1193:             />
1194:             <mesh
1195:                 ref={keyRefs.z}
1196:                 castShadow
1197:                 receiveShadow
1198:                 geometry={nodes.K_Z.geometry}
1199:                 material={keycapMat}
1200:                 position={[ -0.122, 0.008, -0.03]}
1201:             />
1202:             <mesh
1203:                 ref={keyRefs.x}
1204:                 castShadow
1205:                 receiveShadow
1206:                 geometry={nodes.K_X.geometry}
1207:                 material={keycapMat}
1208:                 position={[ -0.122, 0.008, -0.03]}
1209:             />
1210:             <mesh
1211:                 ref={keyRefs.c}
1212:                 castShadow
1213:                 receiveShadow
1214:                 geometry={nodes.K_C.geometry}
1215:                 material={keycapMat}
1216:                 position={[ -0.122, 0.008, -0.03]}
1217:             />
1218:             <mesh
1219:                 ref={keyRefs.v}
1220:                 castShadow
1221:                 receiveShadow
1222:                 geometry={nodes.K_V.geometry}
1223:                 material={keycapMat}
1224:                 position={[ -0.122, 0.008, -0.03]}
1225:             />
1226:             <mesh
1227:                 ref={keyRefs.b}
1228:                 castShadow
1229:                 receiveShadow
1230:                 geometry={nodes.K_B.geometry}
1231:                 material={keycapMat}
1232:                 position={[ -0.122, 0.008, -0.03]}
1233:             />
1234:             <mesh
1235:                 ref={keyRefs.n}
1236:                 castShadow
1237:                 receiveShadow
1238:                 geometry={nodes.K_N.geometry}
1239:                 material={keycapMat}
1240:                 position={[ -0.122, 0.008, -0.03]}
1241:             />
1242:             <mesh
1243:                 ref={keyRefs.m}
1244:                 castShadow
1245:                 receiveShadow
1246:                 geometry={nodes.K_M.geometry}
1247:                 material={keycapMat}
1248:                 position={[ -0.122, 0.008, -0.03]}
1249:             />
1250:             <mesh
1251:                 ref={keyRefs.comma}
1252:                 castShadow
1253:                 receiveShadow
1254:                 geometry={nodes.K_COMMA.geometry}
1255:                 material={keycapMat}
1256:                 position={[ -0.122, 0.008, -0.03]}
1257:             />
1258:             <mesh
1259:                 ref={keyRefs.period}
1260:                 castShadow
1261:                 receiveShadow
1262:                 geometry={nodes.K_PERIOD.geometry}
1263:                 material={keycapMat}
1264:                 position={[ -0.122, 0.008, -0.03]}
1265:             />
1266:             <mesh
1267:                 ref={keyRefs.slash}
```

```

1268:         castShadow
1269:         receiveShadow
1270:         geometry={nodes.K_SLASH.geometry}
1271:         material={keycapMat}
1272:         position=[[-0.122, 0.008, -0.03]]
1273:     />
1274:     <mesh
1275:         ref={keyRefs.rshift}
1276:         castShadow
1277:         receiveShadow
1278:         geometry={nodes.K_RSHIFT.geometry}
1279:         material={keycapMat}
1280:         position=[0.076, 0, -0.03]]
1281:     />
1282:     <mesh
1283:         ref={keyRefs.arrowup}
1284:         castShadow
1285:         receiveShadow
1286:         geometry={nodes.K_ARROWUP.geometry}
1287:         material={keycapMat}
1288:         position=[[-0.122, 0.008, -0.03]]
1289:     />
1290:     <mesh
1291:         ref={keyRefs.end}
1292:         castShadow
1293:         receiveShadow
1294:         geometry={nodes.K_END.geometry}
1295:         material={keycapMat}
1296:         position=[[-0.122, 0.008, -0.03]]
1297:     />
1298: </group>
1299:
1300: { /* Modifiers */
1301: <group ref={modifiersRef}>
1302:     <mesh
1303:         ref={keyRefs.lcontrol}
1304:         castShadow
1305:         receiveShadow
1306:         geometry={nodes.K_LCONTROL.geometry}
1307:         material={keycapMat}
1308:         position=[[-0.162, 0.008, -0.011]]
1309:     />
1310:     <mesh
1311:         ref={keyRefs.lwin}
1312:         castShadow
1313:         receiveShadow
1314:         geometry={nodes.K_LWIN.geometry}
1315:         material={keycapMat}
1316:         position=[[-0.162, 0.008, -0.011]]
1317:     />
1318:     <mesh
1319:         ref={keyRefs.lalt}
1320:         castShadow
1321:         receiveShadow
1322:         geometry={nodes.K_LALT.geometry}
1323:         material={keycapMat}
1324:         position=[[-0.162, 0.008, -0.011]]
1325:     />
1326:     <mesh
1327:         ref={keyRefs.space}
1328:         castShadow
1329:         receiveShadow
1330:         geometry={nodes.K_SPACE.geometry}
1331:         material={keycapMat}
1332:         position=[[-0.043, 0, -0.01]]
1333:     />
1334:     <mesh
1335:         ref={keyRefs.ralt}
1336:         castShadow
1337:         receiveShadow
1338:         geometry={nodes.K_RALT.geometry}
1339:         material={keycapMat}
1340:         position=[[-0.162, 0.008, -0.011]]
1341:     />
1342:     <mesh
1343:         ref={keyRefs.fn}
1344:         castShadow
1345:         receiveShadow
1346:         geometry={nodes.K_FN.geometry}
1347:         material={keycapMat}
1348:         position=[[-0.162, 0.008, -0.011]]
1349:     />
1350: </group>

```

```

1351:
1352:     /* Arrow Keys */
1353:     <group ref={arrowsRef}>
1354:         <mesh
1355:             ref={keyRefs.arrowleft}
1356:             castShadow
1357:             receiveShadow
1358:             geometry={nodes.K_ARROWLEFT.geometry}
1359:             material={keycapMat}
1360:             position={[0.083, 0.008, -0.011]}
1361:         />
1362:         <mesh
1363:             ref={keyRefs.arrowdown}
1364:             castShadow
1365:             receiveShadow
1366:             geometry={nodes.K_ARROWDOWN.geometry}
1367:             material={keycapMat}
1368:             position={[0.083, 0.008, -0.011]}
1369:         />
1370:         <mesh
1371:             ref={keyRefs.arrowright}
1372:             castShadow
1373:             receiveShadow
1374:             geometry={nodes.K_ARROWRIGHT.geometry}
1375:             material={keycapMat}
1376:             position={[0.083, 0.008, -0.011]}
1377:         />
1378:     </group>
1379:
1380:     <instancedMesh
1381:         args={[nodes["2U_Wires"].geometry, materials.Gold, 3]}
1382:         castShadow
1383:         receiveShadow
1384:         instanceMatrix={nodes["2U_Wires"].instanceMatrix}
1385:         position={[0.092, 0.009, -0.086]}
1386:     />
1387:     <instancedMesh
1388:         args={[nodes.Stab_Housing_Instances.geometry, materials.Stem, 8]}
1389:         castShadow
1390:         receiveShadow
1391:         instanceMatrix={nodes.Stab_Housing_Instances.instanceMatrix}
1392:         position={[0.08, -0.004, -0.085]}
1393:     />
1394: </group>
1395: </group>
1396: );
1397: },
1398: );
1399:
1400: NimbusKeyboard.displayName = "NimbusKeyboard";
1401:
1402: useGLTF.preload("/nimbus/keyboard.glTF", "/draco/");

```

```

### src/components/SiteNav.tsx

```

Purpose: Fixed top navigation with active route styling.

```

0001: "use client";
0002:
0003: import Link from "next/link";
0004: import { usePathname } from "next/navigation";
0005: import clsx from "clsx";
0006:
0007: const links = [
0008:   { href: "/", label: "Landing" },
0009:   { href: "/flavours", label: "Flavours" },
0010:   { href: "/reviews", label: "Reviews" },
0011: ];
0012:
0013: export function SiteNav() {
0014:   const pathname = usePathname();
0015:
0016:   return (
0017:     <nav className="fixed inset-x-0 top-0 z-50 px-4 py-4 md:px-6">
0018:       <div className="mx-auto flex max-w-7xl items-center justify-between rounded-lg border border-[#f5f7ff]/16
0019:         bg-[#0b0f14]/60 px-3 py-2 text-[#f5f7ff] shadow-2xl shadow-black/20 backdrop-blur-xl">
0020:         <Link
0021:           href="/"
0022:           className="text-sm font-black uppercase tracking-[0.32em] text-[#f5f7ff]"

```

```

0023:         Cryo
0024:     </Link>
0025:     <div className="flex items-center gap-1">
0026:         {links.map((link) => {
0027:             const active =
0028:                 link.href === "/"
0029:                 ? pathname === "/"
0030:                 : pathname.startsWith(link.href);
0031:
0032:             return (
0033:                 <Link
0034:                     key={link.href}
0035:                     href={link.href}
0036:                     className={clsx(
0037:                         "rounded-md px-3 py-2 text-xs font-black uppercase tracking-[0.16em] transition md:text-sm",
0038:                         active
0039:                         ? "bg-[#f5f7ff] text-[#0b0f14]"
0040:                         : "text-[#f5f7ff]/72 hover:bg-[#f5f7ff]/12 hover:text-[#f5f7ff]",
0041:                     )}
0042:                 >
0043:                     {link.label}
0044:                 </Link>
0045:             );
0046:         })}
0047:     </div>
0048: </div>
0049: </nav>
0050: );
0051: }

```

```

### src/components/SodaCan.tsx

```

Purpose: Reusable can model loader. It loads the GLTF, applies metal material to can body/tab, applies selected label texture to sleeve, and falls back by traversing mesh names.

```

0001: "use client";
0002:
0003: import { useGLTF, useTexture } from "@react-three/drei";
0004: import { useMemo } from "react";
0005: import type { ComponentProps } from "react";
0006: import * as THREE from "three";
0007:
0008: import { LABEL_TEXTURES, type CryoFlavorId } from "@data/flavors";
0009:
0010: const DRACO_PATH = "/draco/";
0011:
0012: useGLTF.preload("/Soda-can.gltf", DRACO_PATH);
0013:
0014: const metalMaterial = new THREE.MeshStandardMaterial({
0015:     roughness: 0.26,
0016:     metalness: 1,
0017:     color: "#cfd8de",
0018: });
0019:
0020: export type SodaCanProps = ComponentProps<"group"> & {
0021:     flavor: CryoFlavorId;
0022:     canScale?: number;
0023: };
0024:
0025: export function SodaCan({ flavor, canScale = 2, ...props }: SodaCanProps) {
0026:     const { nodes, scene } = useGLTF("/Soda-can.gltf", DRACO_PATH);
0027:     const label = usePreparedLabel(flavor);
0028:     const { body, sleeve, tab } = useMemo(
0029:         () => getCanMeshes(nodes, scene),
0030:         [nodes, scene],
0031:     );
0032:
0033:     if (!body || !sleeve || !tab) return null;
0034:
0035:     return (
0036:         <group
0037:             {...props}
0038:             dispose={null}
0039:             scale={canScale}
0040:             rotation={[0, -Math.PI, 0]}
0041:         >
0042:             <mesh
0043:                 castShadow
0044:                 receiveShadow
0045:                 geometry={body.geometry}

```



```

0046:         material={metalMaterial}
0047:     />
0048:     <mesh castShadow receiveShadow geometry={sleeve.geometry}>
0049:         <meshStandardMaterial
0050:             roughness={0.16}
0051:             metalness={0.68}
0052:             map={label}
0053:             toneMapped={false}
0054:         />
0055:     </mesh>
0056:     <mesh
0057:         castShadow
0058:         receiveShadow
0059:         geometry={tab.geometry}
0060:         material={metalMaterial}
0061:     />
0062: </group>
0063: );
0064: }
0065:
0066: function usePreparedLabel(flavor: CryoFlavorId) {
0067:     const label = useTexture(LABEL_TEXTURES[flavor]);
0068:
0069:     label.flipY = false;
0070:     label.colorSpace = THREE.SRGBColorSpace;
0071:     label.anisotropy = 8;
0072:     label.wrapS = THREE.RepeatWrapping;
0073:     label.wrapT = THREE.ClampToEdgeWrapping;
0074:
0075:     return label;
0076: }
0077:
0078: function getCanMeshes(
0079:     nodes: Record<string, THREE.Object3D>,
0080:     scene: THREE.Group,
0081: ) {
0082:     const directBody = nodes.cylinder as THREE.Mesh | undefined;
0083:     const directSleeve = nodes.cylinder_1 as THREE.Mesh | undefined;
0084:     const directTab = nodes.Tab as THREE.Mesh | undefined;
0085:
0086:     if (directBody?.isMesh && directSleeve?.isMesh && directTab?.isMesh) {
0087:         return {
0088:             body: directBody,
0089:             sleeve: directSleeve,
0090:             tab: directTab,
0091:         };
0092:     }
0093:
0094:     const meshes: THREE.Mesh[] = [];
0095:     scene.traverse((object) => {
0096:         if ((object as THREE.Mesh).isMesh) {
0097:             meshes.push(object as THREE.Mesh);
0098:         }
0099:     });
0100:
0101:     const tab =
0102:         directTab ??
0103:         meshes.find((mesh) => /tab/i.test(`${mesh.name} ${mesh.parent?.name}`));
0104:     const canMeshes = meshes.filter((mesh) => mesh !== tab);
0105:
0106:     return {
0107:         body:
0108:             directBody ??
0109:             canMeshes.find((mesh) => mesh.name === "cylinder") ??
0110:             canMeshes[0],
0111:         sleeve:
0112:             directSleeve ??
0113:             canMeshes.find((mesh) => mesh.name === "cylinder_1") ??
0114:             canMeshes[1],
0115:         tab,
0116:     };
0117: }

```

```

### src/data/flavorFrameTypes.ts

```

Purpose: Type definition for frame sequence metadata.

```

0001: import type { CryoFlavorId } from "@data/flavors";
0002:
0003: export type FlavorFrameSequence = {

```

```

0004:   flavorId: CryoFlavorId;
0005:   framePaths: string[];
0006:   frameCount: number;
0007:   frameRate: number;
0008:   videoPath?: string;
0009:   expectedFolder: string;
0010:   discoveredFolder?: string;
0011: };

```

```

### src/data/flavorFrames.ts

```

Purpose: Discovers exported image frame folders and corresponding MP4 paths from `public`.

```

0001: import { existsSync, readdirSync } from "node:fs";
0002: import path from "node:path";
0003:
0004: import type { CryoFlavor, CryoFlavorId } from "@data/flavors";
0005: import type { FlavorFrameSequence } from "@data/flavorFrameTypes";
0006:
0007: const FRAME_ROOT_NAMES = [
0008:   "flavour-frames",
0009:   "flavor-frames",
0010:   "fruit-frames",
0011:   "frames",
0012: ];
0013:
0014: const FRAME_EXTENSIONS = new Set([".jpg", ".jpeg", ".png", ".webp"]);
0015: const FRAME_RATE = 29;
0016:
0017: export function getFlavorFrameSequence(
0018:   flavor: CryoFlavor,
0019: ): FlavorFrameSequence {
0020:   const publicRoot = path.join(process.cwd(), "public");
0021:   const expectedFolder = `/flavour-frames/${flavor.name} frames/`;
0022:   const folder = findFrameFolder(publicRoot, flavor);
0023:   const framePaths = folder ? getFramePaths(publicRoot, folder) : [];
0024:   const videoPath = getFlavorVideoPath(publicRoot, flavor);
0025:
0026:   return {
0027:     flavorId: flavor.id,
0028:     framePaths,
0029:     frameCount: framePaths.length,
0030:     frameRate: FRAME_RATE,
0031:     videoPath,
0032:     expectedFolder,
0033:     discoveredFolder: folder
0034:       ? `${path.relative(publicRoot, folder).split(path.sep).join("/")}`
0035:       : undefined,
0036:   };
0037: }
0038:
0039: export function getFlavorFrameSequences(
0040:   flavors: readonly CryoFlavor[],
0041: ): Record<CryoFlavorId, FlavorFrameSequence> {
0042:   return Object.fromEntries(
0043:     flavors.map((flavor) => [flavor.id, getFlavorFrameSequence(flavor)]),
0044:   ) as Record<CryoFlavorId, FlavorFrameSequence>;
0045: }
0046:
0047: function findFrameFolder(publicRoot: string, flavor: CryoFlavor) {
0048:   const aliases = getFolderAliases(flavor).map(normalizeName);
0049:   const roots = [
0050:     ...FRAME_ROOT_NAMES.map((rootName) => path.join(publicRoot, rootName)),
0051:     publicRoot,
0052:   ];
0053:
0054:   for (const root of roots) {
0055:     if (!existsSync(root)) continue;
0056:
0057:     const direct = findDirectFolder(root, aliases);
0058:     if (direct) return direct;
0059:   }
0060:
0061:   return undefined;
0062: }
0063:
0064: function findDirectFolder(root: string, aliases: string[]) {
0065:   const entries = readdirSync(root, { withFileTypes: true });
0066:
0067:   for (const entry of entries) {

```

```

0068:     if (!entry.isDirectory()) continue;
0069:
0070:     const normalized = normalizeName(entry.name);
0071:
0072:     if (aliases.includes(normalized)) {
0073:         return path.join(root, entry.name);
0074:     }
0075: }
0076:
0077: return undefined;
0078: }
0079:
0080: function getFolderAliases(flavor: CryoFlavor) {
0081:     return [
0082:         `${flavor.name} frames`,
0083:         `${flavor.name} Frames`,
0084:         `${flavor.name}-frames`,
0085:         `${flavor.slug} frames`,
0086:         `${flavor.slug}-frames`,
0087:         `${flavor.id} frames`,
0088:         `${flavor.id}-frames`,
0089:         flavor.name,
0090:         flavor.slug,
0091:         flavor.id,
0092:     ];
0093: }
0094:
0095: function getFramePaths(publicRoot: string, folder: string) {
0096:     return readdirSync(folder, { withFileTypes: true })
0097:         .filter((entry) => entry.isFile())
0098:         .map((entry) => entry.name)
0099:         .filter((name) => FRAME_EXTENSIONS.has(path.extname(name).toLowerCase()))
0100:         .sort(naturalCompare)
0101:         .map((name) => toPublicPath(publicRoot, path.join(folder, name)));
0102: }
0103:
0104: function getFlavorVideoPath(publicRoot: string, flavor: CryoFlavor) {
0105:     const videoPath = path.join(
0106:         publicRoot,
0107:         "flavour-videos",
0108:         `${flavor.slug}.mp4`,
0109:     );
0110:
0111:     return existsSync(videoPath)
0112:         ? toPublicPath(publicRoot, videoPath)
0113:         : undefined;
0114: }
0115:
0116: function toPublicPath(publicRoot: string, filePath: string) {
0117:     const relativePath = path.relative(publicRoot, filePath);
0118:     const segments = relativePath.split(path.sep).map(encodeURIComponent);
0119:
0120:     return `/${segments.join("/")}`;
0121: }
0122:
0123: function normalizeName(value: string) {
0124:     return value.toLowerCase().replace(/^[a-z0-9]+/g, "");
0125: }
0126:
0127: function naturalCompare(a: string, b: string) {
0128:     return a.localeCompare(b, undefined, {
0129:         numeric: true,
0130:         sensitivity: "base",
0131:     });
0132: }

```

```

### src/data/flavorVideoTypes.ts

```

Purpose: Type definition for video sequence metadata.

```

0001: import type { CryoFlavorId } from "@data/flavors";
0002:
0003: export type FlavorVideoSequence = {
0004:     flavorId: CryoFlavorId;
0005:     videoPath?: string;
0006:     expectedVideoPath: string;
0007: };

```

```
### src/data/flavorVideos.ts
```

Purpose: Checks whether a flavour MP4 exists and returns public path metadata.

```
0001: import { existsSync } from "node:fs";
0002: import path from "node:path";
0003:
0004: import type { CryoFlavor } from "@data/flavors";
0005: import type { FlavorVideoSequence } from "@data/flavorVideoTypes";
0006:
0007: export function getFlavorVideoSequence(
0008:   flavor: CryoFlavor,
0009: ): FlavorVideoSequence {
0010:   const publicRoot = path.join(process.cwd(), "public");
0011:   const expectedVideoPath = `/${flavour-videos/${flavor.slug}.mp4`;
0012:   const diskPath = path.join(
0013:     publicRoot,
0014:     "flavour-videos",
0015:     `${flavor.slug}.mp4`,
0016:   );
0017:
0018:   return {
0019:     flavorId: flavor.id,
0020:     videoPath: existsSync(diskPath) ? expectedVideoPath : undefined,
0021:     expectedVideoPath,
0022:   };
0023: }
```

```
### src/data/flavors.ts
```

Purpose: Main data model for five flavours, including slugs, marketing copy, colors, juice colors, label paths, logos, AR paths, and resolver helpers.

```
0001: export const CRYO_FLAVORS = [
0002:   {
0003:     id: "frostbiteBerry",
0004:     slug: "frostbite-berry",
0005:     name: "Frostbite Berry",
0006:     flavor: "Berry and lychee",
0007:     headline: "Sharp berry frost with a lychee snap.",
0008:     description:
0009:       "Frostbite Berry opens with cold red fruit, then lands on a clean lychee finish. The profile is built for
a bright first hit and a crisp finish that does not turn syrupy.",
0010:     colors: {
0011:       primary: "#c21121",
0012:       secondary: "#b3e6fb",
0013:       text: "#21080d",
0014:       contrast: "#ffffff",
0015:     },
0016:     juiceColors: ["#d6132a", "#eef8ff", "#7dd3fc"],
0017:     label: "/assets/labels/frostbite-berry.png",
0018:     details: [
0019:       "Red fruit top note with a clean lychee finish.",
0020:       "Best served over ice with citrus peel.",
0021:       "Bright, sharp, and low on aftertaste.",
0022:     ],
0023:   },
0024:   {
0025:     id: "neonMeltdown",
0026:     slug: "neon-meltdown",
0027:     name: "Neon Meltdown",
0028:     flavor: "Watermelon and mint",
0029:     headline: "Watermelon voltage cut with cold mint.",
0030:     description:
0031:       "Neon Meltdown is juicy up front and cold at the edge. Watermelon brings the body, while mint keeps the
finish clean and electric.",
0032:     colors: {
0033:       primary: "#c1fe1a",
0034:       secondary: "#fe00ae",
0035:       text: "#170022",
0036:       contrast: "#101604",
0037:     },
0038:     juiceColors: ["#fe267f", "#b8ff23", "#79ffd9"],
0039:     label: "/assets/labels/neon-meltdown.png",
0040:     details: [
0041:       "Watermelon body with a cold mint finish.",
0042:       "The sweetest Cryo profile, balanced by herbal chill.",
0043:       "Pairs well with spicy food and late-night heat.",
0044:     ],
0045:   },
0046: ]
```

```

0045:   },
0046:   {
0047:     id: "cosmicCrush",
0048:     slug: "cosmic-crush",
0049:     name: "Cosmic Crush",
0050:     flavor: "Grape and kiwi",
0051:     headline: "Dark grape pressure with a kiwi flash.",
0052:     description:
0053:       "Cosmic Crush is darker and rounder than the rest of the lineup. Grape brings depth while kiwi cuts
through with a bright green edge.",
0054:     colors: {
0055:       primary: "#433455",
0056:       secondary: "#ddea78",
0057:       text: "#130c1d",
0058:       contrast: "#ffffff",
0059:     },
0060:     juiceColors: ["#56316d", "#c7ef59", "#f0ff96"],
0061:     label: "/assets/labels/cosmic-crush.png",
0062:     details: [
0063:       "Deep grape base with a tart kiwi lift.",
0064:       "The richest profile in the lineup.",
0065:       "Finishes bright instead of heavy.",
0066:     ],
0067:   },
0068:   {
0069:     id: "midnightCitrus",
0070:     slug: "midnight-citrus",
0071:     name: "Midnight Citrus",
0072:     flavor: "Blueberry and lemon",
0073:     headline: "Blueberry dusk split by lemon light.",
0074:     description:
0075:       "Midnight Citrus keeps blueberry smooth and lemon sharp. It starts dark, then opens into a clean citrus
flash.",
0076:     colors: {
0077:       primary: "#0d3b66",
0078:       secondary: "#faf0ca",
0079:       text: "#061526",
0080:       contrast: "#ffffff",
0081:     },
0082:     juiceColors: ["#1d4f8f", "#fff071", "#8bd3ff"],
0083:     label: "/assets/labels/midnight-citrus.png",
0084:     details: [
0085:       "Blueberry depth with lemon brightness.",
0086:       "The cleanest citrus finish in the range.",
0087:       "Built for a cold, sharp sip.",
0088:     ],
0089:   },
0090:   {
0091:     id: "velvetFrost",
0092:     slug: "velvet-frost",
0093:     name: "Velvet Frost",
0094:     flavor: "Peach and vanilla",
0095:     headline: "Peach cloud softened with vanilla frost.",
0096:     description:
0097:       "Velvet Frost is smooth, creamy, and still cold. Peach brings the lift while vanilla rounds the edges into
a soft finish.",
0098:     colors: {
0099:       primary: "#c8a2c9",
0100:       secondary: "#fefbce",
0101:       text: "#2f1736",
0102:       contrast: "#25112b",
0103:     },
0104:     juiceColors: ["#ffb179", "#fef5c7", "#d8a5df"],
0105:     label: "/assets/labels/velvet-frost.png",
0106:     details: [
0107:       "Peach top note with a creamy vanilla finish.",
0108:       "Softest mouthfeel in the Cryo set.",
0109:       "Best when poured very cold.",
0110:     ],
0111:   },
0112: ] as const;
0113:
0114: export type CryoFlavor = (typeof CRYO_FLAVORS)[number];
0115: export type CryoFlavorId = CryoFlavor["id"];
0116: export type CryoFlavorSlug = CryoFlavor["slug"];
0117:
0118: export const DEFAULT_FLAVOR_ID = CRYO_FLAVORS[0].id;
0119:
0120: export const LABEL_TEXTURES: Record<CryoFlavorId, string> = {
0121:   frostbiteBerry: "/assets/labels/frostbite-berry.png",
0122:   neonMeltdown: "/assets/labels/neon-meltdown.png",
0123:   cosmicCrush: "/assets/labels/cosmic-crush.png",
0124:   midnightCitrus: "/assets/labels/midnight-citrus.png",

```

```

0125:   velvetFrost: "/assets/labels/velvet-frost.png",
0126: };
0127:
0128: export const LOGO_TEXTURES: Record<CryoFlavorId, string> = {
0129:   frostbiteBerry: "/assets/logos/cryo-frostbite-berry.png",
0130:   neonMeltdown: "/assets/logos/cryo-neon-meltdown.png",
0131:   cosmicCrush: "/assets/logos/cryo-cosmic-crush.png",
0132:   midnightCitrus: "/assets/logos/cryo-midnight-citrus.png",
0133:   velvetFrost: "/assets/logos/cryo-velvet-frost.png",
0134: };
0135:
0136: export const AR_MODEL_PATHS: Record<CryoFlavorId, string> = {
0137:   frostbiteBerry: "/ar/frostbite-berry/can.glTF",
0138:   neonMeltdown: "/ar/neon-meltdown/can.glTF",
0139:   cosmicCrush: "/ar/cosmic-crush/can.glTF",
0140:   midnightCitrus: "/ar/midnight-citrus/can.glTF",
0141:   velvetFrost: "/ar/velvet-frost/can.glTF",
0142: };
0143:
0144: export function getFlavorById(id: CryoFlavorId) {
0145:   return CRYO_FLAVORS.find((flavor) => flavor.id === id) ?? CRYO_FLAVORS[0];
0146: }
0147:
0148: export function getFlavorBySlug(slug: string) {
0149:   return CRYO_FLAVORS.find((flavor) => flavor.slug === slug) ?? CRYO_FLAVORS[0];
0150: }
0151:
0152: export function getFlavorByRouteValue(value: string | undefined) {
0153:   if (!value) return CRYO_FLAVORS[0];
0154:
0155:   return (
0156:     CRYO_FLAVORS.find(
0157:       (flavor) => flavor.id === value || flavor.slug === value,
0158:     ) ?? CRYO_FLAVORS[0]
0159:   );
0160: }

```

src/hooks/useMediaQuery.ts

Purpose: SSR-safe media query hook using `useSyncExternalStore`.

```

0001: import { useCallback, useSyncExternalStore } from "react";
0002:
0003: export function useMediaQuery(query: string, serverFallback: boolean): boolean {
0004:   const subscribe = useCallback(
0005:     (onStoreChange: () => void) => {
0006:       const mediaQueryList = matchMedia(query);
0007:       mediaQueryList.addEventListener("change", onStoreChange);
0008:       return () => mediaQueryList.removeEventListener("change", onStoreChange);
0009:     },
0010:     [query],
0011:   );
0012:
0013:   return useSyncExternalStore(
0014:     subscribe,
0015:     () => matchMedia(query).matches,
0016:     () => serverFallback,
0017:   );
0018: }

```

src/slices/AR/ARExperience.tsx

Purpose: Full AR page. Chooses mobile model-viewer or desktop Three showcase, shows logo/color/atmosphere, and renders QR code handoff.

```

0001: "use client";
0002:
0003: import { Canvas } from "@react-three/fiber";
0004: import Image from "next/image";
0005: import Link from "next/link";
0006: import { QRCodeSVG } from "qrcode.react";
0007: import { Suspense, useEffect, useMemo, useRef, useState } from "react";
0008: import clsx from "clsx";
0009:
0010: import { AR_MODEL_PATHS, LOGO_TEXTURES, type CryoFlavor } from "@data/flavors";
0011: import { useMediaQuery } from "@hooks/useMediaQuery";
0012: import { DesktopShowcase } from "@slices/AR/DesktopShowcase";

```

```

0013: import {
0014:   ModelViewerAR,
0015:   type ModelViewerARHandle,
0016: } from "@slices/AR/ModelViewerAR";
0017:
0018: type ARExperienceProps = {
0019:   flavor: CryoFlavor;
0020:   autoLaunch?: boolean;
0021: };
0022:
0023: const ATMOSPHERE_COPY: Record<CryoFlavor["id"], string> = {
0024:   frostbiteBerry: "Icy frost particles with a cool red product glow.",
0025:   neonMeltdown: "Neon pink sparks cut through lime edge lighting.",
0026:   cosmicCrush: "Purple ambience with tart lime highlights around the can.",
0027:   midnightCitrus: "Deep navy staging with a clean citrus rim glow.",
0028:   velvetFrost: "Soft lilac light and a creamy haze around the surface.",
0029: };
0030:
0031: export function ARExperience({ flavor, autoLaunch = false }: ARExperienceProps) {
0032:   const modelViewerRef = useRef<ModelViewerARHandle>(null);
0033:   const isCoarsePointer = useMediaQuery("(pointer: coarse)", false);
0034:   const isSmallScreen = useMediaQuery("(max-width: 767px)", false);
0035:   const [origin, setOrigin] = useState("");
0036:   const [qrVisible, setQrVisible] = useState(false);
0037:   const isMobileAR = isCoarsePointer || isSmallScreen;
0038:   const modelSrc = AR_MODEL_PATHS[flavor.id];
0039:   const qrPath = `/ar/${flavor.slug}`;
0040:   const qrValue = `${origin}${qrPath}`;
0041:   const logoSrc = LOGO_TEXTURES[flavor.id];
0042:   const style = useMemo(
0043:     () =>
0044:     ({
0045:       "--flavor-primary": flavor.colors.primary,
0046:       "--flavor-secondary": flavor.colors.secondary,
0047:     }) as React.CSSProperties,
0048:     [flavor],
0049:   );
0050:
0051:   useEffect(() => {
0052:     setOrigin(window.location.origin);
0053:   }, []);
0054:
0055:   const handlePrimaryAction = () => {
0056:     if (isMobileAR) {
0057:       void modelViewerRef.current?.launchAR();
0058:       return;
0059:     }
0060:
0061:     setQrVisible(true);
0062:   };
0063:
0064:   return (
0065:     <main
0066:       className="min-h-screen overflow-hidden bg-[#080c11] text-[#f5f7ff]"
0067:       style={style}
0068:     >
0069:       <section className="relative min-h-screen px-4 py-24 md:px-8 md:py-28">
0070:         <div
0071:           aria-hidden="true"
0072:           className="pointer-events-none absolute inset-0"
0073:           style={{
0074:             background: `
0075:               radial-gradient(circle at 72% 26%, ${flavor.colors.primary}40, transparent 24rem),
0076:               radial-gradient(circle at 22% 74%, ${flavor.colors.secondary}30, transparent 22rem),
0077:               linear-gradient(180deg, #080c11 0%, #0b0f14 100%)
0078:             `,
0079:           }}
0080:         />
0081:         <div className="pointer-events-none absolute inset-0 bg-[linear-gradient(rgba(245,247,255,0.045)_1px,transparent_1px),linear-gradient(90deg,rgba(245,247,255,0.045)_1px,transparent_1px)] bg-[size:46px_46px]" />
0082:
0083:         <div className="relative z-10 mx-auto grid w-full max-w-7xl gap-6 lg:grid-cols-[minmax(0,1fr),24rem]">
0084:           <header className="flex items-center justify-between gap-4 lg:col-span-2">
0085:             <Link
0086:               href={`/${flavours}?flavor=${flavor.id}`}
0087:               className="rounded-md border border-[#f5f7ff]/18 bg-[#f5f7ff]/8 px-4 py-2 text-xs font-black uppercase tracking-[0.18em] text-[#f5f7ff]/78 transition hover:border-[#f5f7ff]/48 hover:bg-[#f5f7ff]/14"
0088:             >
0089:               Back to flavour
0090:             </Link>
0091:             <Image
0092:               alt={` ${flavor.name} logo`

```

```

0093:         className="h-10 w-auto object-contain md:h-12"
0094:         height={64}
0095:         src={logoSrc}
0096:         width={220}
0097:     />
0098: </header>
0099:
0100:     <div className="min-h-[34rem] overflow-hidden rounded-lg border border-[#f5f7ff]/14 bg-[#05080d]/72
shadow-2xl shadow-black/35">
0101:         {isMobileAR ? (
0102:             <ModelViewerAR
0103:                 ref={modelViewerRef}
0104:                 autoLaunch={autoLaunch}
0105:                 className="h-[calc(100vh-12rem)] min-h-[34rem]"
0106:                 flavor={flavor}
0107:                 modelSrc={modelSrc}
0108:             />
0109:         ) : (
0110:             <div className="relative h-[calc(100vh-12rem)] min-h-[34rem]">
0111:                 <Canvas
0112:                     camera={{ position: [0, 1, 4.6], fov: 36 }}
0113:                     dpr={[1, 1.5]}
0114:                     gl={{ antialias: true, alpha: false }}
0115:                     shadows
0116:                 >
0117:                     <Suspense fallback={null}>
0118:                         <DesktopShowcase flavor={flavor} />
0119:                     </Suspense>
0120:                 </Canvas>
0121:                 <div className="pointer-events-none absolute left-4 top-4 rounded-md border border-[#f5f7ff]/12
bg-[#0b0f14]/62 px-3 py-2 text-xs font-black uppercase tracking-[0.18em] text-[#f5f7ff]/78 backdrop-blur-xl">
0122:                     Desktop showcase
0123:                 </div>
0124:             </div>
0125:         )}
0126:     </div>
0127:
0128:     <aside className="flex flex-col gap-4 rounded-lg border border-[#f5f7ff]/14 bg-[#0b0f14]/72 p-5
shadow-2xl shadow-black/30 backdrop-blur-xl md:p-6">
0129:         <div>
0130:             <p className="text-xs font-black uppercase tracking-[0.28em] text-[#f5f7ff]/58">
0131:                 CRYO AR
0132:             </p>
0133:             <h1 className="mt-3 text-balance text-4xl font-black uppercase leading-[0.88] md:text-5xl">
0134:                 {flavor.name}
0135:             </h1>
0136:             <p className="mt-4 text-pretty text-base font-bold leading-snug text-[#f5f7ff]/72">
0137:                 {flavor.flavor}. Open the matching can in AR on a supported
0138:                 phone, or inspect the virtual product showcase here.
0139:             </p>
0140:         </div>
0141:
0142:         <div className="grid grid-cols-2 gap-3">
0143:             <div
0144:                 className="h-16 rounded-md border border-[#f5f7ff]/12"
0145:                 style={{ backgroundColor: flavor.colors.primary }}
0146:             />
0147:             <div
0148:                 className="h-16 rounded-md border border-[#f5f7ff]/12"
0149:                 style={{ backgroundColor: flavor.colors.secondary }}
0150:             />
0151:         </div>
0152:
0153:         <div className="rounded-lg border border-[#f5f7ff]/12 bg-[#f5f7ff]/6 p-4">
0154:             <p className="text-sm font-black uppercase text-[#f5f7ff]/86">
0155:                 Atmosphere
0156:             </p>
0157:             <p className="mt-2 text-sm font-bold leading-snug text-[#f5f7ff]/62">
0158:                 {ATMOSPHERE_COPY[flavor.id]}
0159:             </p>
0160:         </div>
0161:
0162:         <button
0163:             className={clsx(
0164:                 "mt-auto rounded-md border px-5 py-4 text-sm font-black uppercase tracking-normal shadow-xl
transition",
0165:                 isMobileAR
0166:                 ? "border-[#f5f7ff] bg-[#f5f7ff] text-[#0b0f14] shadow-black/28 hover:scale-[1.02]"
0167:                 : "border-[#f5f7ff]/18 bg-[#f5f7ff]/10 text-[#f5f7ff] hover:border-[#f5f7ff]/46
hover:bg-[#f5f7ff]/16",
0168:             )}
0169:             onClick={handlePrimaryAction}
0170:             style={isMobileAR ? { color: flavor.colors.text } : undefined}

```



```

0171:         type="button"
0172:     >
0173:         {isMobileAR ? "Launch AR" : "View on Phone"}
0174:     </button>
0175:
0176:     {!isMobileAR && (
0177:         <div
0178:             className={clsx(
0179:                 "rounded-lg border border-[#f5f7ff]/12 bg-[#f5f7ff] p-4 text-[#0b0f14] transition",
0180:                 qrVisible ? "opacity-100" : "opacity-80",
0181:             )}
0182:         >
0183:             <div className="grid place-items-center rounded-md bg-white p-3">
0184:                 {origin ? (
0185:                     <QRCodeSVG
0186:                         bgColor="#ffffff"
0187:                         fgColor="#0b0f14"
0188:                         includeMargin={false}
0189:                         level="M"
0190:                         size={190}
0191:                         value={qrValue}
0192:                     />
0193:                 ) : null}
0194:             </div>
0195:             <p className="mt-3 text-center text-xs font-black uppercase leading-tight">
0196:                 Scan to open {qrPath}
0197:             </p>
0198:         </div>
0199:     )}
0200: </aside>
0201: </div>
0202: </section>
0203: </main>
0204: );
0205: }

```

```

### src/slices/AR/ARLaunchButton.tsx

```

Purpose: Inline flavour-page AR launcher that attempts direct mobile AR and falls back to `/ar/<slug>?launch=1`.

```

0001: "use client";
0002:
0003: import { useRouter } from "next/navigation";
0004: import { useEffect, useRef, useState } from "react";
0005:
0006: import { AR_MODEL_PATHS, type CryoFlavor } from "@/data/flavors";
0007: import { useMediaQuery } from "@/hooks/useMediaQuery";
0008:
0009: type ModelViewerElement = HTMLElement & {
0010:   activateAR?: () => Promise<void>;
0011: };
0012:
0013: type ARLaunchButtonProps = {
0014:   flavor: CryoFlavor;
0015: };
0016:
0017: export function ARLaunchButton({ flavor }: ARLaunchButtonProps) {
0018:   const router = useRouter();
0019:   const modelViewerRef = useRef<ModelViewerElement | null>(null);
0020:   const isCoarsePointer = useMediaQuery("(pointer: coarse)", false);
0021:   const isSmallScreen = useMediaQuery("(max-width: 767px)", false);
0022:   const [status, setStatus] = useState("");
0023:   const isMobileARCandidate = isCoarsePointer || isSmallScreen;
0024:   const arHref = `/ar/${flavor.slug}?launch=1`;
0025:   const modelSrc = AR_MODEL_PATHS[flavor.id];
0026:
0027:   useEffect(() => {
0028:     if (!isMobileARCandidate) return;
0029:
0030:     void import("@google/model-viewer");
0031:   }, [isMobileARCandidate]);
0032:
0033:   const handleLaunch = async () => {
0034:     if (!isMobileARCandidate) {
0035:       router.push(arHref);
0036:       return;
0037:     }
0038:
0039:     const modelViewer = modelViewerRef.current;
0040:

```

```

0041:     if (!modelViewer?.activateAR) {
0042:         router.push(arHref);
0043:         return;
0044:     }
0045:
0046:     try {
0047:         await modelViewer.activateAR();
0048:         setStatus("");
0049:     } catch {
0050:         setStatus("Opening the AR viewer page instead.");
0051:         router.push(arHref);
0052:     }
0053: };
0054:
0055: return (
0056:     <>
0057:         <button
0058:             className="pointer-events-auto mt-7 inline-flex rounded-md border px-5 py-3 text-sm font-black uppercase
0059:             tracking-normal shadow-2xl shadow-black/30 transition hover:scale-[1.02]"
0060:             onClick={handleLaunch}
0061:             style={{
0062:                 backgroundColor: `${flavor.colors.secondary}20`,
0063:                 borderColor: flavor.colors.secondary,
0064:                 color: flavor.colors.secondary,
0065:             }}
0066:             type="button"
0067:         >
0068:             Experience in AR
0069:         </button>
0070:         {status ? (
0071:             <span className="sr-only" role="status">
0072:                 {status}
0073:             </span>
0074:         ) : null}
0075:         {isMobileARCandidate ? (
0076:             <div
0077:                 aria-hidden="true"
0078:                 className="pointer-events-none fixed bottom-0 left-0 size-px overflow-hidden opacity-0"
0079:             >
0080:                 <model-viewer
0081:                     ref={modelViewerRef}
0082:                     alt={`${flavor.name} CRYO can`}
0083:                     ar
0084:                     ar-modes="webxr scene-viewer quick-look"
0085:                     ar-placement="floor"
0086:                     ar-scale="fixed"
0087:                     camera-orbit="-22deg 72deg 0.42m"
0088:                     environment-image="/hdr/lobby.hdr"
0089:                     exposure="1.05"
0090:                     field-of-view="28deg"
0091:                     quick-look-browsers="safari chrome"
0092:                     shadow-intensity="0.9"
0093:                     shadow-softness="0.72"
0094:                     src={modelSrc}
0095:                     xr-environment
0096:                 />
0097:             </div>
0098:         ) : null}
0099:     </>
0100: );

```

```

### src/slices/AR/DesktopShowcase.tsx

```

Purpose: Desktop 3D fallback scene with orbit controls, desk surface, flavour particles, rings, condensation, lighting, and shadows.

```

0001: "use client";
0002:
0003: import {
0004:     ContactShadows,
0005:     Environment,
0006:     OrbitControls,
0007:     PerspectiveCamera,
0008: } from "@react-three/drei";
0009: import { useFrame } from "@react-three/fiber";
0010: import { useMemo, useRef } from "react";
0011: import * as THREE from "three";
0012:
0013: import { SodaCan } from "@components/SodaCan";

```

```
0014: import type { CryoFlavor } from "@data/flavors";
0015:
0016: type DesktopShowcaseProps = {
0017:   flavor: CryoFlavor;
0018: };
0019:
0020: export function DesktopShowcase({ flavor }: DesktopShowcaseProps) {
0021:   const canRef = useRef<THREE.Group>(null);
0022:
0023:   useFrame(({ clock }) => {
0024:     if (!canRef.current) return;
0025:     canRef.current.rotation.y = Math.sin(clock.elapsedTime * 0.4) * 0.18;
0026:   });
0027:
0028:   return (
0029:     <>
0030:       <color attach="background" args={["#0b0f14"]} />
0031:       <fog attach="fog" args={["#0b0f14", 6, 13]} />
0032:       <PerspectiveCamera makeDefault position={[0, 1.05, 4.6]} fov={36} />
0033:       <OrbitControls
0034:         enableDamping
0035:         enablePan={false}
0036:         maxDistance={6}
0037:         maxPolarAngle={Math.PI / 2.05}
0038:         minDistance={2.5}
0039:         target={[0, -0.1, 0]}
0040:       />
0041:
0042:       <ambientLight intensity={1.3} color={flavor.colors.secondary} />
0043:       <spotLight
0044:         castShadow
0045:         color={flavor.colors.secondary}
0046:         intensity={58}
0047:         penumbra={0.68}
0048:         position={[-3.5, 4, 4.5]}
0049:       />
0050:       <pointLight
0051:         color={flavor.colors.primary}
0052:         intensity={24}
0053:         position={[2.4, 0.65, 2.2]}
0054:       />
0055:       <pointLight
0056:         color={flavor.colors.secondary}
0057:         intensity={12}
0058:         position={[-2.2, -0.35, 1.5]}
0059:       />
0060:
0061:       <Atmosphere flavor={flavor} />
0062:
0063:       <group>
0064:         <mesh receiveShadow position={[0, -0.84, 0]}>
0065:           <boxGeometry args={[5.6, 0.16, 4.25]} />
0066:           <meshStandardMaterial
0067:             color="#11161b"
0068:             metalness={0.06}
0069:             roughness={0.64}
0070:           />
0071:         </mesh>
0072:         <mesh position={[0, -0.752, 0]} rotation={[-Math.PI / 2, 0, 0]}>
0073:           <ringGeometry args={[0.72, 2.96, 96]} />
0074:           <meshBasicMaterial
0075:             color={flavor.colors.primary}
0076:             opacity={0.1}
0077:             transparent
0078:           />
0079:         </mesh>
0080:         <mesh position={[0, -0.75, -1.16]} rotation={[-Math.PI / 2, 0, 0]}>
0081:           <planeGeometry args={[4.4, 0.012]} />
0082:           <meshBasicMaterial
0083:             color={flavor.colors.secondary}
0084:             opacity={0.18}
0085:             transparent
0086:           />
0087:         </mesh>
0088:         <mesh position={[0, -0.749, 1.12]} rotation={[-Math.PI / 2, 0, 0]}>
0089:           <planeGeometry args={[4.1, 0.012]} />
0090:           <meshBasicMaterial
0091:             color={flavor.colors.primary}
0092:             opacity={0.12}
0093:             transparent
0094:           />
0095:         </mesh>
0096:       <FlavorRings flavor={flavor} />
0097:     </>
0098:   );
0099: }
```

```

0097:         <CondensationField flavor={flavor} />
0098:
0099:         <group ref={canRef} position={[0, -0.18, 0]} rotation={[0.04, 0, -0.04]}>
0100:             <SodaCan flavor={flavor.id} canScale={1.58} />
0101:         </group>
0102:
0103:         <ContactShadows
0104:             blur={2.6}
0105:             far={4}
0106:             opacity={0.45}
0107:             position={[0, -0.735, 0]}
0108:             resolution={1024}
0109:             scale={5}
0110:         />
0111:     </group>
0112:
0113:     <Environment files="/hdr/lobby.hdr" environmentIntensity={1.2} />
0114: </>
0115: });
0116: }
0117:
0118: type FlavorRingsProps = {
0119:   flavor: CryoFlavor;
0120: };
0121:
0122: function FlavorRings({ flavor }: FlavorRingsProps) {
0123:   const groupRef = useRef<THREE.Group>(null);
0124:
0125:   useFrame(({ clock }) => {
0126:     if (!groupRef.current) return;
0127:     groupRef.current.rotation.y = -clock.elapsedTime * 0.12;
0128:   });
0129:
0130:   return (
0131:     <group ref={groupRef} position={[0, -0.21, 0]}>
0132:       <mesh rotation={[Math.PI / 2.35, 0, 0.22]}>
0133:         <torusGeometry args={[0.86, 0.006, 8, 128]} />
0134:         <meshBasicMaterial
0135:           color={flavor.colors.primary}
0136:           opacity={0.34}
0137:           transparent
0138:         />
0139:       </mesh>
0140:       <mesh rotation={[Math.PI / 2.18, 0, -0.32]}>
0141:         <torusGeometry args={[1.04, 0.004, 8, 128]} />
0142:         <meshBasicMaterial
0143:           color={flavor.colors.secondary}
0144:           opacity={0.22}
0145:           transparent
0146:         />
0147:       </mesh>
0148:     </group>
0149:   );
0150: }
0151:
0152: type CondensationFieldProps = {
0153:   flavor: CryoFlavor;
0154: };
0155:
0156: function CondensationField({ flavor }: CondensationFieldProps) {
0157:   const droplets = useMemo(() => {
0158:     const seed = hashString(`${flavor.id}-droplets`);
0159:
0160:     return Array.from({ length: 18 }, (_, index) => {
0161:       const angle = index * 2.399963 + seed * 0.000013;
0162:       const radius = 0.42 + ((index * 23) % 100) / 260;
0163:
0164:       return {
0165:         position: [
0166:           Math.cos(angle) * radius,
0167:           -0.746,
0168:           Math.sin(angle) * radius * 0.72,
0169:         ] as [number, number, number],
0170:         scale: 0.012 + ((index * 11) % 10) * 0.002,
0171:       };
0172:     });
0173:   }, [flavor.id]);
0174:
0175:   return (
0176:     <group>
0177:       {droplets.map((droplet, index) => (
0178:         <mesh key={index} position={droplet.position} scale={droplet.scale}>
0179:           <sphereGeometry args={[1, 12, 8]} />

```

```

0180:         <meshPhysicalMaterial
0181:             color="#dff7ff"
0182:             metalness={0}
0183:             opacity={0.48}
0184:             roughness={0.08}
0185:             transmission={0.35}
0186:             transparent
0187:         />
0188:     </mesh>
0189:   )}}
0190: </group>
0191: );
0192: }
0193:
0194: type AtmosphereProps = {
0195:   flavor: CryoFlavor;
0196: };
0197:
0198: function Atmosphere({ flavor }: AtmosphereProps) {
0199:   const pointsRef = useRef<THREE.Points>(null);
0200:   const particleColor = new THREE.Color(flavor.colors.secondary);
0201:   const accentColor = new THREE.Color(flavor.colors.primary);
0202:   const positions = useMemo(() => {
0203:     const values = new Float32Array(90 * 3);
0204:     const seed = hashString(flavor.id);
0205:
0206:     for (let index = 0; index < 90; index += 1) {
0207:       const angle = index * 2.399963 + seed * 0.00001;
0208:       const radius = 1.05 + ((index * 37) % 100) / 80;
0209:       values[index * 3] = Math.cos(angle) * radius;
0210:       values[index * 3 + 1] = -0.48 + ((index * 17) % 100) / 74;
0211:       values[index * 3 + 2] = Math.sin(angle) * radius * 0.72;
0212:     }
0213:
0214:     return values;
0215:   }, [flavor.id]);
0216:
0217:   useFrame(({ clock }) => {
0218:     if (!pointsRef.current) return;
0219:     pointsRef.current.rotation.y = clock.elapsedTime * 0.045;
0220:     pointsRef.current.position.y = Math.sin(clock.elapsedTime * 0.55) * 0.025;
0221:   });
0222:
0223:   return (
0224:     <>
0225:       <points ref={pointsRef}>
0226:         <bufferGeometry>
0227:           <bufferAttribute
0228:             attach="attributes-position"
0229:             args={[positions, 3]}
0230:             count={positions.length / 3}
0231:             itemSize={3}
0232:           />
0233:         </bufferGeometry>
0234:         <pointsMaterial
0235:           color={particleColor}
0236:           depthWrite={false}
0237:           opacity={getParticleOpacity(flavor.id)}
0238:           size={getParticleSize(flavor.id)}
0239:           transparent
0240:         />
0241:       </points>
0242:       <mesh position={[0, -0.72, 0]} rotation={[-Math.PI / 2, 0, 0]}>
0243:         <circleGeometry args={[2.2, 96]} />
0244:         <meshBasicMaterial
0245:           color={accentColor}
0246:           opacity={getGlowOpacity(flavor.id)}
0247:           transparent
0248:         />
0249:       </mesh>
0250:     </>
0251:   );
0252: }
0253:
0254: function getParticleOpacity(flavorId: CryoFlavor["id"]) {
0255:   if (flavorId === "velvetFrost") return 0.24;
0256:   if (flavorId === "frostbiteBerry") return 0.34;
0257:   return 0.28;
0258: }
0259:
0260: function getParticleSize(flavorId: CryoFlavor["id"]) {
0261:   if (flavorId === "neonMeltdown") return 0.028;
0262:   if (flavorId === "cosmicCrush") return 0.024;

```

```

0263:   return 0.02;
0264: }
0265:
0266: function getGlowOpacity(flavorId: CryoFlavor["id"]) {
0267:   if (flavorId === "midnightCitrus") return 0.13;
0268:   if (flavorId === "velvetFrost") return 0.11;
0269:   return 0.16;
0270: }
0271:
0272: function hashString(value: string) {
0273:   let hash = 2166136261;
0274:   for (let index = 0; index < value.length; index += 1) {
0275:     hash ^= value.charCodeAt(index);
0276:     hash = Math.imul(hash, 16777619);
0277:   }
0278:   return hash >>> 0;
0279: }

```

```

### src/slices/AR/ModelViewerAR.tsx

```

Purpose: Model-viewer wrapper with imperative `launchAR`, status messages, auto launch, AR events, and a visible Place Can button.

```

0001: "use client";
0002:
0003: import {
0004:   forwardRef,
0005:   useCallback,
0006:   useEffect,
0007:   useImperativeHandle,
0008:   useRef,
0009:   useState,
0010: } from "react";
0011: import clsx from "clsx";
0012:
0013: import type { CryoFlavor } from "@data/flavors";
0014:
0015: type ModelViewerElement = HTMLElement & {
0016:   activateAR?: () => Promise<void>;
0017:   canActivateAR?: boolean;
0018: };
0019:
0020: export type ModelViewerARHandle = {
0021:   launchAR: () => Promise<void>;
0022: };
0023:
0024: type ModelViewerARProps = {
0025:   flavor: CryoFlavor;
0026:   modelSrc: string;
0027:   className?: string;
0028:   autoLaunch?: boolean;
0029: };
0030:
0031: export const ModelViewerAR = forwardRef<
0032:   ModelViewerARHandle,
0033:   ModelViewerARProps
0034: >((function ModelViewerAR({ flavor, modelSrc, className, autoLaunch }, ref) {
0035:   const modelViewerRef = useRef<ModelViewerElement | null>(null);
0036:   const [isReady, setIsReady] = useState(false);
0037:   const [arStatus, setArStatus] = useState("Ready for AR placement");
0038:
0039:   const launchAR = useCallback(async () => {
0040:     const modelViewer = modelViewerRef.current;
0041:     if (!modelViewer?.activateAR) {
0042:       setArStatus("AR is not available on this browser. Use the 3D viewer.");
0043:       return;
0044:     }
0045:
0046:     try {
0047:       await modelViewer.activateAR();
0048:     } catch {
0049:       setArStatus("AR could not start. Use the 3D viewer or scan from a phone.");
0050:     }
0051:   }, []);
0052:
0053:   useImperativeHandle(ref, () => ({
0054:     launchAR,
0055:   })), [launchAR]);
0056:
0057:   useEffect(() => {

```

```

0058:     void import("@google/model-viewer");
0059: }, []);
0060:
0061: useEffect(() => {
0062:     const modelViewer = modelViewerRef.current;
0063:     if (!modelViewer) return;
0064:
0065:     const handleLoad = () => {
0066:         setIsReady(true);
0067:         setArStatus(
0068:             modelViewer.canActivateAR === false
0069:             ? "3D viewer ready. AR is unavailable on this device."
0070:             : "Ready for AR placement",
0071:         );
0072:     };
0073:     const handleError = () => {
0074:         setArStatus("The AR model failed to load. The desktop showcase is still available.");
0075:     };
0076:     const handleARStatus = (event: Event) => {
0077:         const detail = (event as CustomEvent<{ status?: string }>).detail;
0078:         if (detail?.status === "session-started") {
0079:             setArStatus("Move your phone to find a desk, table, or floor.");
0080:         } else if (detail?.status === "object-placed") {
0081:             setArStatus("Placed. Walk around the can to inspect it.");
0082:         } else if (detail?.status === "failed") {
0083:             setArStatus("AR failed to start. Try opening this page on a supported phone.");
0084:         } else {
0085:             setArStatus("Ready for AR placement");
0086:         }
0087:     };
0088:
0089:     modelViewer.addEventListener("load", handleLoad);
0090:     modelViewer.addEventListener("error", handleError);
0091:     modelViewer.addEventListener("ar-status", handleARStatus);
0092:
0093:     return () => {
0094:         modelViewer.removeEventListener("load", handleLoad);
0095:         modelViewer.removeEventListener("error", handleError);
0096:         modelViewer.removeEventListener("ar-status", handleARStatus);
0097:     };
0098: }, [modelSrc]);
0099:
0100: useEffect(() => {
0101:     if (!autoLaunch || !isReady) return;
0102:
0103:     const timer = window.setTimeout(() => {
0104:         void launchAR();
0105:     }, 350);
0106:
0107:     return () => window.clearTimeout(timer);
0108: }, [autoLaunch, isReady, launchAR]);
0109:
0110: return (
0111:     <div
0112:         className={clsx(
0113:             "relative h-full min-h-[30rem] overflow-hidden rounded-lg border border-[#f5f7ff]/14 bg-[#05080d]",
0114:             className,
0115:         )}
0116:     >
0117:         <model-viewer
0118:             ref={modelViewerRef}
0119:             alt={`${flavor.name} CRYO can in AR`}
0120:             ar
0121:             ar-modes="webxr scene-viewer quick-look"
0122:             ar-placement="floor"
0123:             ar-scale="fixed"
0124:             auto-rotate
0125:             camera-controls
0126:             camera-orbit="-22deg 72deg 0.42m"
0127:             className="absolute inset-0 h-full w-full"
0128:             environment-image="/hdr/lobby.hdr"
0129:             exposure="1.05"
0130:             field-of-view="28deg"
0131:             interaction-prompt="auto"
0132:             max-camera-orbit="Infinity 88deg 0.9m"
0133:             min-camera-orbit="-Infinity 34deg 0.24m"
0134:             quick-look-browsers="safari chrome"
0135:             shadow-intensity="0.9"
0136:             shadow-softness="0.72"
0137:             src={modelSrc}
0138:             touch-action="pan-y"
0139:             xr-environment
0140:         >

```

```

0141:         <button
0142:             slot="ar-button"
0143:             aria-hidden="true"
0144:             className="hidden"
0145:             tabIndex=-1
0146:             type="button"
0147:         />
0148:     </model-viewer>
0149:
0150:     <div
0151:         aria-hidden="true"
0152:         className="pointer-events-none absolute inset-x-0 top-0 h-32"
0153:         style={{
0154:             background: `linear-gradient(180deg, ${flavor.colors.primary}33, transparent)`,
0155:         }}
0156:     />
0157:     <div className="pointer-events-none absolute left-4 top-4 rounded-md border border-[#f5f7ff]/12
bg-[#0b0f14]/62 px-3 py-2 text-xs font-black uppercase tracking-[0.18em] text-[#f5f7ff]/78 backdrop-blur-xl">
0158:         {arStatus}
0159:     </div>
0160:     <button
0161:         className="absolute bottom-5 left-1/2 z-10 -translate-x-1/2 rounded-md border border-[#f5f7ff]/20
bg-[#f5f7ff] px-5 py-3 text-sm font-black uppercase tracking-normal text-[#0b0f14] shadow-2xl shadow-black/30
transition hover:scale-[1.02]"
0162:         onClick={() => {
0163:             void launchAR();
0164:         }}
0165:         style={{ color: flavor.colors.text }}
0166:         type="button"
0167:     >
0168:         Place Can in AR
0169:     </button>
0170: </div>
0171: );
0172: });

```

src/slices/Flavours/FlavorExperience.tsx

Purpose: Main flavour storytelling page. Manages video playback, can drop handoff, scroll scan progress, scan box, flavour details, quick links, and helpers.

```

0001: "use client";
0002:
0003: import { Environment, PerspectiveCamera } from "@react-three/drei";
0004: import { Canvas, useFrame } from "@react-three/fiber";
0005: import clsx from "clsx";
0006: import gsap from "gsap";
0007: import { ScrollTrigger } from "gsap/ScrollTrigger";
0008: import Link from "next/link";
0009: import { Suspense, useEffect, useMemo, useRef, useState } from "react";
0010: import * as THREE from "three";
0011:
0012: import { Bounded } from "@components/Bounded";
0013: import { SodaCan } from "@components/SodaCan";
0014: import type { FlavorVideoSequence } from "@data/flavorVideoTypes";
0015: import {
0016:     CRYO_FLAVORS,
0017:     getFlavorById,
0018:     type CryoFlavor,
0019:     type CryoFlavorId,
0020: } from "@data/flavors";
0021: import { ARLaunchButton } from "@slices/AR/ARLaunchButton";
0022:
0023: gsap.registerPlugin(ScrollTrigger);
0024:
0025: const CAN_DROP_START_PROGRESS = 0.78;
0026: const CAN_DROP_END_PROGRESS = 0.94;
0027: const VIDEO_FADE_START_PROGRESS = 0.9;
0028: const VIDEO_FADE_END_PROGRESS = 0.99;
0029: const SCAN_BOX_ENTER_START = 0.04;
0030: const SCAN_BOX_ENTER_END = 0.18;
0031: const SCAN_TURN_START = 0.2;
0032: const SCAN_TURN_END = 0.68;
0033: const SCAN_SETTLE_START = 0.72;
0034: const SCAN_SETTLE_END = 0.9;
0035:
0036: type PlaybackState = {
0037:     progress: number;
0038:     duration: number;
0039:     ended: boolean;

```



```

0040:   ready: boolean;
0041: };
0042:
0043: type FlavorExperienceProps = {
0044:   initialFlavorId: CryoFlavorId;
0045:   videoSequence: FlavorVideoSequence;
0046: };
0047:
0048: export function FlavorExperience({
0049:   initialFlavorId,
0050:   videoSequence,
0051: }: FlavorExperienceProps) {
0052:   const flavor = getFlavorById(initialFlavorId);
0053:   const scanSectionRef = useRef<HTMLDivElement>(null);
0054:   const [handoffCanProgress, setHandoffCanProgress] = useState(0);
0055:   const [scanProgress, setScanProgress] = useState(0);
0056:   const style = useMemo(
0057:     () =>
0058:     ({
0059:       "--flavor-primary": flavor.colors.primary,
0060:       "--flavor-secondary": flavor.colors.secondary,
0061:     }) as React.CSSProperties,
0062:     [flavor],
0063:   );
0064:
0065:   useEffect(() => {
0066:     const section = scanSectionRef.current;
0067:     if (!section) return;
0068:
0069:     if (window.matchMedia("(prefers-reduced-motion: reduce)").matches) {
0070:       setScanProgress(1);
0071:       return;
0072:     }
0073:
0074:     let frame = 0;
0075:     const trigger = ScrollTrigger.create({
0076:       trigger: section,
0077:       start: "top top",
0078:       end: "bottom bottom",
0079:       scrub: true,
0080:       onUpdate: (self) => {
0081:         window.cancelAnimationFrame(frame);
0082:         frame = window.requestAnimationFrame(() =>
0083:           setScanProgress(Number(self.progress.toFixed(4))),
0084:         );
0085:       },
0086:     });
0087:
0088:     ScrollTrigger.refresh();
0089:
0090:     return () => {
0091:       window.cancelAnimationFrame(frame);
0092:       trigger.kill();
0093:     };
0094:   }, [flavor.id]);
0095:
0096:   return (
0097:     <main
0098:       className="min-h-screen bg-[color:var(--flavor-primary)] text-[color:var(--flavor-secondary)]"
0099:       style={style}
0100:     >
0101:       <PersistentHandoffCan
0102:         flavor={flavor}
0103:         reveal={handoffCanProgress}
0104:         scanProgress={scanProgress}
0105:       />
0106:
0107:       <section className="flavour-story relative min-h-dvh overflow-hidden">
0108:         <div className="relative h-dvh overflow-hidden">
0109:           <FrameSequenceShowcase
0110:             flavor={flavor}
0111:             onCanProgress={setHandoffCanProgress}
0112:             sequence={videoSequence}
0113:           />
0114:
0115:           <Bounded className="pointer-events-none relative z-30 flex min-h-dvh items-start pt-28 md:pt-32">
0116:             <div className="max-w-3xl">
0117:               <p className="text-sm font-black uppercase tracking-[0.34em] opacity-75 md:text-base">
0118:                 {flavor.flavor}
0119:               </p>
0120:               <h1 className="mt-4 text-balance text-5xl font-black uppercase leading-[0.86] tracking-normal
0121:                 md:text-7xl lg:text-[5.8rem]">
0122:                 {flavor.name}

```

```

0122:         </h1>
0123:         <p className="mt-6 max-w-2xl text-pretty text-lg font-bold leading-snug opacity-85 md:text-2xl">
0124:             {flavor.headline}
0125:         </p>
0126:         <ARLaunchButton flavor={flavor} />
0127:     </div>
0128: </Bounded>
0129:
0130:     <FlavorQuickLinks activeFlavorId={flavor.id} />
0131: </div>
0132: </section>
0133:
0134:     <FlavorScrollScan
0135:         flavor={flavor}
0136:         progress={scanProgress}
0137:         sectionRef={scanSectionRef}
0138:     />
0139: </main>
0140: );
0141: }
0142:
0143: type FlavorScrollScanProps = {
0144:   flavor: CryoFlavor;
0145:   progress: number;
0146:   sectionRef: React.RefObject<HTMLElement>;
0147: };
0148:
0149: function FlavorScrollScan({
0150:   flavor,
0151:   progress,
0152:   sectionRef,
0153: }: FlavorScrollScanProps) {
0154:   const scanBoxProgress = smoothstep(
0155:     clamp(
0156:       (progress - SCAN_BOX_ENTER_START) /
0157:       (SCAN_BOX_ENTER_END - SCAN_BOX_ENTER_START),
0158:       0,
0159:       1,
0160:     ),
0161:   );
0162:   const detailProgress = smoothstep(clamp((progress - 0.42) / 0.28, 0, 1));
0163:
0164:   return (
0165:     <section
0166:       ref={sectionRef}
0167:       className="flavour-scan relative min-h-[190dvh] bg-[color:var(--flavor-primary)] text-[color:var(--flavor-secondary)]"
0168:       data-scan-progress={progress.toFixed(4)}
0169:       data-scroll-scan="soda-can-to-box"
0170:     >
0171:       <div className="sticky top-0 h-dvh overflow-hidden">
0172:         <Bounded className="relative min-h-dvh pt-24 md:pt-20">
0173:           <div className="relative z-20 min-h-[58dvh] md:min-h-[74dvh]">
0174:             <ScanBox flavor={flavor} progress={scanBoxProgress} />
0175:           </div>
0176:
0177:           <FlavorScanDetails flavor={flavor} progress={detailProgress} />
0178:         </Bounded>
0179:       </div>
0180:     </section>
0181:   );
0182: }
0183:
0184: type ScanBoxProps = {
0185:   flavor: CryoFlavor;
0186:   progress: number;
0187: };
0188:
0189: function ScanBox({ flavor, progress }: ScanBoxProps) {
0190:   const lineProgress = smoothstep(clamp((progress - 0.18) / 0.72, 0, 1));
0191:
0192:   return (
0193:     <div
0194:       aria-hidden="true"
0195:       className="fixed right-[-12vw] top-1/2 h-[min(54dvh,32rem)] w-[min(72vw,28rem)] -translate-y-1/2 rounded-lg border-2 md:right-[calc(8vw+3rem)] md:h-[min(56dvh,34rem)] md:w-[30rem]"
0196:       data-scan-box="flavour-profile"
0197:       style={{
0198:         borderColor: flavor.colors.secondary,
0199:         boxShadow: `0 0 ${2.5 + progress * 3.5}rem ${flavor.colors.secondary}30`,
0200:         opacity: progress,
0201:         transform: `translate3d(0, -50%, 0) scale(${progress})`,
0202:       }}

```

```

0203:   >
0204:   <div
0205:     className="absolute inset-x-5 top-5 h-px"
0206:     style={{ backgroundColor: `${flavor.colors.secondary}70` }}
0207:   />
0208:   <div
0209:     className="absolute inset-x-5 bottom-5 h-px"
0210:     style={{ backgroundColor: `${flavor.colors.secondary}70` }}
0211:   />
0212:   <div
0213:     className="absolute left-5 top-5 h-12 w-px"
0214:     style={{ backgroundColor: `${flavor.colors.secondary}70` }}
0215:   />
0216:   <div
0217:     className="absolute bottom-5 right-5 h-12 w-px"
0218:     style={{ backgroundColor: `${flavor.colors.secondary}70` }}
0219:   />
0220:   <div
0221:     className="absolute inset-x-6 top-1/2 h-1 -translate-y-1/2 rounded-full"
0222:     data-scan-line="active"
0223:     style={{
0224:       backgroundColor: flavor.colors.secondary,
0225:       boxShadow: `0 0 2.25rem ${flavor.colors.secondary}`,
0226:       opacity: progress,
0227:       transform: `translate3d(0, calc(-50% + ${lineProgress - 0.5} * 18}rem), 0)`,
0228:     }}
0229:   />
0230:   <p className="absolute bottom-8 left-6 text-xs font-black uppercase tracking-[0.32em] opacity-75">
0231:     CRYO profile
0232:   </p>
0233: </div>
0234:   );
0235: }
0236:
0237: type PersistentHandoffCanProps = {
0238:   flavor: CryoFlavor;
0239:   reveal: number;
0240:   scanProgress: number;
0241: };
0242:
0243: function PersistentHandoffCan({
0244:   flavor,
0245:   reveal,
0246:   scanProgress,
0247: }: PersistentHandoffCanProps) {
0248:   const turnProgress = smoothstep(
0249:     clamp(
0250:       (scanProgress - SCAN_TURN_START) / (SCAN_TURN_END - SCAN_TURN_START),
0251:       0,
0252:       1,
0253:     ),
0254:   );
0255:   const settleProgress = smoothstep(
0256:     clamp(
0257:       (scanProgress - SCAN_SETTLE_START) /
0258:         (SCAN_SETTLE_END - SCAN_SETTLE_START),
0259:       0,
0260:       1,
0261:     ),
0262:   );
0263:   const opacity = scanProgress > 0.001 ? 1 : reveal;
0264:   const dropLift = scanProgress > 0.001 ? 0 : (1 - reveal) * 46;
0265:   const dropRotation = scanProgress > 0.001 ? 0 : (1 - reveal) * -8;
0266:   const scale =
0267:     scanProgress > 0.001 ? 1 - settleProgress * 0.34 : 0.72 + reveal * 0.28;
0268:
0269:   return (
0270:     <div
0271:       aria-hidden="true"
0272:       className="pointer-events-none fixed right-[-12vw] top-1/2 z-40 h-[66dvh] w-[min(28rem,66vw)] min-w-48
0273:       origin-center will-change-transform md:right-[-8vw] md:h-[78dvh] md:w-[min(36rem,44vw)]"
0274:       data-can-drop="same-soda-can"
0275:       data-persistent-can="video-to-scan-box"
0276:       data-scan-settle-progress={settleProgress.toFixed(4)}
0277:       data-scan-turn-progress={turnProgress.toFixed(4)}
0278:       data-scroll-can="same-soda-can"
0279:       style={{
0280:         opacity,
0281:         filter: `drop-shadow(0 ${1.5 + opacity * 1.4}rem ${1.4 + opacity * 1.8}rem rgba(0, 0, 0, ${0.22 +
0282:         scanProgress * 0.18}))`,
0283:         transform: `translate3d(0, calc(-50% - ${dropLift}dvh), 0) rotate(${dropRotation}deg) scale(${scale})`,
0284:       }}
0285:     >

```

```

0284:     <Canvas
0285:         camera={{ position: [0, 0.15, 5.3], fov: 27 }}
0286:         dpr={{[1, 1.5]}}
0287:         gl={{ antialias: true, alpha: true }}
0288:         shadows
0289:     >
0290:     <Suspense fallback={null}>
0291:         <PersistentHandoffCanScene
0292:             flavor={flavor}
0293:             reveal={reveal}
0294:             scanProgress={scanProgress}
0295:         />
0296:     </Suspense>
0297: </Canvas>
0298: </div>
0299: );
0300: }
0301:
0302: type PersistentHandoffCanSceneProps = {
0303:     flavor: CryoFlavor;
0304:     reveal: number;
0305:     scanProgress: number;
0306: };
0307:
0308: function PersistentHandoffCanScene({
0309:     flavor,
0310:     reveal,
0311:     scanProgress,
0312: }: PersistentHandoffCanSceneProps) {
0313:     const groupRef = useRef<THREE.Group>(null);
0314:
0315:     useFrame(({ clock }) => {
0316:         if (!groupRef.current) return;
0317:
0318:         if (scanProgress <= 0.001) {
0319:             groupRef.current.rotation.x = 0;
0320:             groupRef.current.rotation.y =
0321:                 -0.5 + reveal * 0.42 + Math.sin(clock.elapsedTime * 0.55) * 0.045;
0322:             groupRef.current.rotation.z = -0.08 + (1 - reveal) * 0.16;
0323:             groupRef.current.position.y =
0324:                 -0.12 + reveal * 0.1 + Math.sin(clock.elapsedTime * 0.7) * 0.025;
0325:             return;
0326:         }
0327:
0328:         const turnProgress = smoothstep(
0329:             clamp(
0330:                 (scanProgress - SCAN_TURN_START) / (SCAN_TURN_END - SCAN_TURN_START),
0331:                 0,
0332:                 1,
0333:             ),
0334:         );
0335:         const settleProgress = smoothstep(
0336:             clamp(
0337:                 (scanProgress - SCAN_SETTLE_START) /
0338:                 (SCAN_SETTLE_END - SCAN_SETTLE_START),
0339:                 0,
0340:                 1,
0341:             ),
0342:         );
0343:         const floating =
0344:             Math.sin(clock.elapsedTime * 1.45) *
0345:             0.045 *
0346:             (1 - turnProgress) *
0347:             (1 - settleProgress);
0348:
0349:         groupRef.current.rotation.x = turnProgress * Math.PI * 2;
0350:         groupRef.current.rotation.y =
0351:             -0.58 + clock.elapsedTime * 0.1 * (1 - settleProgress);
0352:         groupRef.current.rotation.z = -0.08 + settleProgress * 0.08;
0353:         groupRef.current.position.y = floating - settleProgress * 0.08;
0354:     });
0355:
0356:     return (
0357:         <>
0358:             <PerspectiveCamera makeDefault position={[0, 0.15, 5.3]} fov={27} />
0359:             <ambientLight intensity={2.35} color={flavor.colors.secondary} />
0360:             <spotLight
0361:                 castShadow
0362:                 color="#ffffff"
0363:                 intensity={34}
0364:                 penumbra={0.72}
0365:                 position={[-2.8, 4.1, 4.2]}
0366:             />

```

```

0367:     <pointLight
0368:       color={flavor.colors.primary}
0369:       intensity={42}
0370:       position={[2.6, 0.4, 2.4]}
0371:     />
0372:     <pointLight
0373:       color={flavor.colors.secondary}
0374:       intensity={26}
0375:       position={[-1.8, -0.7, 2.4]}
0376:     />
0377:
0378:     <group
0379:       ref={groupRef}
0380:       position={[0, -0.08, 0]}
0381:       rotation={[0, -0.5, -0.08]}
0382:     >
0383:       <SodaCan flavor={flavor.id} canScale={2.72} />
0384:     </group>
0385:
0386:     <Environment files="/hdr/lobby.hdr" environmentIntensity={1.18} />
0387:   </>
0388: );
0389: }
0390:
0391: type FlavorScanDetailsProps = {
0392:   flavor: CryoFlavor;
0393:   progress: number;
0394: };
0395:
0396: function FlavorScanDetails({ flavor, progress }: FlavorScanDetailsProps) {
0397:   const ingredients = getFlavorIngredients(flavor);
0398:
0399:   return (
0400:     <div
0401:       className="absolute inset-x-5 bottom-6 z-30 grid gap-5 rounded-lg border p-5 backdrop-blur-md md:bottom-10
0402:       md:left-8 md:right-auto md:w-[min(42rem,48vw)] md:p-6"
0403:       data-flavour-details="scan-result"
0404:       style={{
0405:         backgroundColor: `${flavor.colors.primary}e8`,
0406:         borderColor: `${flavor.colors.secondary}52`,
0407:         boxShadow: `0 2rem 5rem rgba(0, 0, 0, ${0.18 + progress * 0.14})`,
0408:         opacity: progress,
0409:         transform: `translate3d(0, ${(1 - progress) * 2.6}rem, 0)`,
0410:       }}
0411:     >
0412:       <div>
0413:         <p className="text-xs font-black uppercase tracking-[0.32em] opacity-65">
0414:           Scan complete
0415:         </p>
0416:         <h3 className="mt-3 text-balance text-3xl font-black uppercase leading-[0.9] md:text-5xl">
0417:           {flavor.name}
0418:         </h3>
0419:       </div>
0420:
0421:       <p className="text-pretty text-base font-bold leading-snug opacity-[0.82] md:text-xl">
0422:         {flavor.description}
0423:       </p>
0424:
0425:       <div className="grid gap-4 md:grid-cols-[0.78fr,1.22fr]">
0426:         <div>
0427:           <p className="text-xs font-black uppercase tracking-[0.28em] opacity-60">
0428:             Ingredients
0429:           </p>
0430:           <ul className="mt-3 flex flex-wrap gap-2">
0431:             {ingredients.map((ingredient) => (
0432:               <li
0433:                 key={ingredient}
0434:                 className="rounded-md border px-3 py-2 text-sm font-black uppercase"
0435:                 style={{
0436:                   backgroundColor: `${flavor.colors.secondary}16`,
0437:                   borderColor: `${flavor.colors.secondary}42`,
0438:                 }}
0439:               >
0440:                 {ingredient}
0441:               </li>
0442:             ))}
0443:           </ul>
0444:         </div>
0445:
0446:         <div>
0447:           <p className="text-xs font-black uppercase tracking-[0.28em] opacity-60">
0448:             Notes

```

```

0449:         <ul className="mt-3 grid gap-2">
0450:             {flavor.details.map((detail) => (
0451:                 <li
0452:                     key={detail}
0453:                     className="text-sm font-bold leading-tight opacity-[0.86]"
0454:                 >
0455:                     {detail}
0456:                 </li>
0457:             )}}
0458:         </ul>
0459:     </div>
0460: </div>
0461: </div>
0462: );
0463: }
0464:
0465: type FrameSequenceShowcaseProps = {
0466:   flavor: CryoFlavor;
0467:   onCanProgress: React.Dispatch<React.SetStateAction<number>>;
0468:   sequence: FlavorVideoSequence;
0469: };
0470:
0471: function FrameSequenceShowcase({
0472:   flavor,
0473:   onCanProgress,
0474:   sequence,
0475: }: FrameSequenceShowcaseProps) {
0476:   const [playback, setPlayback] = useState<PlaybackState>({
0477:     progress: 0,
0478:     duration: 0,
0479:     ended: false,
0480:     ready: false,
0481:   });
0482:   const canDropProgress = smoothstep(
0483:     clamp(
0484:       (playback.progress - CAN_DROP_START_PROGRESS) /
0485:       (CAN_DROP_END_PROGRESS - CAN_DROP_START_PROGRESS),
0486:       0,
0487:       1,
0488:     ),
0489:   );
0490:   const playbackComplete = playback.ended || playback.progress >= 0.995;
0491:   const videoOpacity = playbackComplete
0492:     ? 0
0493:     : 1 -
0494:       smoothstep(
0495:         clamp(
0496:           (playback.progress - VIDEO_FADE_START_PROGRESS) /
0497:           (VIDEO_FADE_END_PROGRESS - VIDEO_FADE_START_PROGRESS),
0498:           0,
0499:           1,
0500:         ),
0501:       );
0502:   const visibleCanProgress = playbackComplete ? 1 : canDropProgress;
0503:
0504:   useEffect(() => {
0505:     setPlayback({
0506:       progress: 0,
0507:       duration: 0,
0508:       ended: false,
0509:       ready: false,
0510:     });
0511:   }, [sequence.videoPath]);
0512:
0513:   useEffect(() => {
0514:     const refreshFrame = window.requestAnimationFrame(() =>
0515:       ScrollTrigger.refresh(),
0516:     );
0517:
0518:     return () => window.cancelAnimationFrame(refreshFrame);
0519:   }, [sequence.videoPath]);
0520:
0521:   useEffect(() => {
0522:     onCanProgress(visibleCanProgress);
0523:   }, [onCanProgress, visibleCanProgress]);
0524:
0525:   useEffect(() => {
0526:     return () => onCanProgress(0);
0527:   }, [onCanProgress]);
0528:
0529:   return (
0530:     <div
0531:       className="pointer-events-none absolute inset-0 z-10 overflow-hidden"

```

```
0532:     data-frame-sequence={flavor.slug}
0533:     data-playback-source="video-autoplay"
0534:     data-video-ended={playbackComplete ? "true" : "false"}
0535:     data-video-progress={playback.progress.toFixed(4)}
0536:     data-can-drop-progress={visibleCanProgress.toFixed(4)}
0537:   >
0538:     {sequence.videoPath ? (
0539:       <FrameSequenceVideo
0540:         flavor={flavor}
0541:         onPlayback={setPlayback}
0542:         opacity={videoOpacity}
0543:         sequence={sequence}
0544:       />
0545:     ) : (
0546:       <MissingFrameSequence sequence={sequence} />
0547:     )}
0548:   </div>
0549: );
0550: }
0551:
0552: type FrameSequenceVideoProps = {
0553:   flavor: CryoFlavor;
0554:   sequence: FlavorVideoSequence;
0555:   opacity: number;
0556:   onPlayback: React.Dispatch<React.SetStateAction<PlaybackState>>;
0557: };
0558:
0559: function FrameSequenceVideo({
0560:   flavor,
0561:   sequence,
0562:   opacity,
0563:   onPlayback,
0564: }: FrameSequenceVideoProps) {
0565:   const videoRef = useRef<HTMLVideoElement>(null);
0566:   const [ready, setReady] = useState(false);
0567:
0568:   useEffect(() => {
0569:     const video = videoRef.current;
0570:     if (!video) return;
0571:
0572:     let animationFrame = 0;
0573:     let cancelled = false;
0574:
0575:     const publishPlayback = () => {
0576:       if (cancelled) return;
0577:
0578:       const duration = Number.isFinite(video.duration) ? video.duration : 0;
0579:       const progress = duration ? clamp(video.currentTime / duration, 0, 1) : 0;
0580:
0581:       onPlayback({
0582:         progress: video.ended ? 1 : progress,
0583:         duration,
0584:         ended: video.ended,
0585:         ready: video.readyState >= 2,
0586:       });
0587:
0588:       if (!video.ended) {
0589:         animationFrame = window.requestAnimationFrame(publishPlayback);
0590:       }
0591:     };
0592:
0593:     const startPlayback = () => {
0594:       setReady(true);
0595:       onPlayback((current) => ({
0596:         ...current,
0597:         ready: true,
0598:         duration: Number.isFinite(video.duration) ? video.duration : 0,
0599:       }));
0600:       window.cancelAnimationFrame(animationFrame);
0601:       animationFrame = window.requestAnimationFrame(publishPlayback);
0602:       void video.play().catch(() => undefined);
0603:     };
0604:
0605:     const markEnded = () => {
0606:       const duration = Number.isFinite(video.duration) ? video.duration : 0;
0607:       window.cancelAnimationFrame(animationFrame);
0608:       onPlayback({
0609:         progress: 1,
0610:         duration,
0611:         ended: true,
0612:         ready: true,
0613:       });
0614:     };
0615:   };
```

```

0615:
0616:     setReady(false);
0617:     onPlayback({
0618:         progress: 0,
0619:         duration: 0,
0620:         ended: false,
0621:         ready: false,
0622:     });
0623:
0624:     video.addEventListener("loadedmetadata", startPlayback);
0625:     video.addEventListener("canplay", startPlayback);
0626:     video.addEventListener("play", startPlayback);
0627:     video.addEventListener("ended", markEnded);
0628:
0629:     if (video.readyState >= 1) {
0630:         startPlayback();
0631:     }
0632:
0633:     return () => {
0634:         cancelled = true;
0635:         window.cancelAnimationFrame(animationFrame);
0636:         video.removeEventListener("loadedmetadata", startPlayback);
0637:         video.removeEventListener("canplay", startPlayback);
0638:         video.removeEventListener("play", startPlayback);
0639:         video.removeEventListener("ended", markEnded);
0640:     };
0641: }, [onPlayback, sequence.videoPath]);
0642:
0643:     return (
0644:         <video
0645:             ref={videoRef}
0646:             aria-label={`${flavor.name} explosion video`}
0647:             autoPlay
0648:             className="absolute inset-0 h-full w-full scale-[1.08] object-cover opacity-0 transition-opacity
duration-150"
0649:             data-video-autoplay="true"
0650:             data-video-visible-opacity={opacity.toFixed(3)}
0651:             muted
0652:             playsInline
0653:             preload="auto"
0654:             src={sequence.videoPath}
0655:             style={{ opacity: ready ? opacity * 0.96 : 0 }}
0656:         />
0657:     );
0658: }
0659:
0660: type MissingFrameSequenceProps = {
0661:     sequence: FlavorVideoSequence;
0662: };
0663:
0664: function MissingFrameSequence({ sequence }: MissingFrameSequenceProps) {
0665:     return (
0666:         <div className="absolute inset-0 grid place-items-center px-6 text-center">
0667:             <div className="max-w-xl rounded-lg border border-[color:var(--flavor-secondary)] bg-[color:var(--flavor-
primary)] p-6 text-[color:var(--flavor-secondary)] shadow-2xl shadow-black/35 backdrop-blur-xl">
0668:                 <p className="text-xs font-black uppercase tracking-[0.28em] opacity-60">
0669:                     Video missing
0670:                 </p>
0671:                 <p className="mt-3 text-pretty text-lg font-black uppercase leading-tight">
0672:                     Add {sequence.expectedVideoPath}
0673:                 </p>
0674:             </div>
0675:         </div>
0676:     );
0677: }
0678:
0679: type FlavorQuickLinksProps = {
0680:     activeFlavorId: CryoFlavorId;
0681: };
0682:
0683: function FlavorQuickLinks({ activeFlavorId }: FlavorQuickLinksProps) {
0684:     const activeFlavor = getFlavorById(activeFlavorId);
0685:
0686:     return (
0687:         <div className="absolute inset-x-0 bottom-4 z-30 px-4 md:bottom-6">
0688:             <div
0689:                 className="mx-auto max-w-6xl overflow-x-auto rounded-lg border p-2 shadow-2xl shadow-black/35 backdrop-
blur-xl"
0690:                 style={{
0691:                     backgroundColor: `${activeFlavor.colors.primary}d9`,
0692:                     borderColor: `${activeFlavor.colors.secondary}3d`,
0693:                 }}
0694:             >

```



```

0695:         <ul className="grid min-w-[48rem] grid-cols-5 gap-2 md:min-w-0">
0696:             {CRYO_FLAVORS.map((item) => {
0697:                 const active = item.id === activeFlavorId;
0698:
0699:                 return (
0700:                     <li key={item.id}>
0701:                         <Link
0702:                             href={`\flavours?flavor=${item.id}`}
0703:                             className={clsx(
0704:                                 "grid h-full rounded-lg border p-3 text-left transition",
0705:                                 active ? "shadow-lg shadow-black/15" : "hover:opacity-85",
0706:                             )}
0707:                             style={{
0708:                                 backgroundColor: active
0709:                                     ? `${activeFlavor.colors.secondary}24`
0710:                                     : `${activeFlavor.colors.secondary}10`,
0711:                                 borderColor: active
0712:                                     ? activeFlavor.colors.secondary
0713:                                     : `${activeFlavor.colors.secondary}3d`,
0714:                                 color: activeFlavor.colors.secondary,
0715:                             }}
0716:                         >
0717:                             <span className="text-sm font-black uppercase">
0718:                                 {item.name}
0719:                             </span>
0720:                             <span className="mt-1 truncate text-xs font-bold opacity-70">
0721:                                 {item.flavor}
0722:                             </span>
0723:                         </Link>
0724:                     </li>
0725:                 );
0726:             }}
0727:         </ul>
0728:     </div>
0729: </div>
0730: );
0731: }
0732:
0733: function clamp(value: number, min: number, max: number) {
0734:     return Math.min(max, Math.max(min, value));
0735: }
0736:
0737: function smoothstep(value: number) {
0738:     return value * value * (3 - 2 * value);
0739: }
0740:
0741: function getFlavorIngredients(flavor: CryoFlavor) {
0742:     return flavor.flavor
0743:         .split(/\s+and\s+/i)
0744:         .map((ingredient) =>
0745:             ingredient.trim().replace(/\b\w/g, (letter) => letter.toUpperCase()),
0746:         )
0747:         .filter(Boolean);
0748: }

```

```

### src/slices/Hero/Bubbles.tsx

```

Purpose: Instanced floating bubble field around the landing cans.

```

0001: "use client";
0002:
0003: import { useEffect, useMemo, useRef } from "react";
0004: import { useFrame } from "@react-three/fiber";
0005: import * as THREE from "three";
0006: import gsap from "gsap";
0007:
0008: const object = new THREE.Object3D();
0009:
0010: type BubblesProps = {
0011:     color: string;
0012:     count?: number;
0013:     speed?: number;
0014: };
0015:
0016: export function Bubbles({ color, count = 180, speed = 1.4 }: BubblesProps) {
0017:     const meshRef = useRef<THREE.InstancedMesh>(null);
0018:     const velocities = useRef(new Float32Array(count));
0019:     const material = useMemo(
0020:         () =>
0021:             new THREE.MeshStandardMaterial({

```

```

0022:         color,
0023:         transparent: true,
0024:         opacity: 0.36,
0025:         roughness: 0.2,
0026:         metalness: 0.1,
0027:     }},
0028:     [color],
0029: );
0030:
0031: useEffect(() => {
0032:     const mesh = meshRef.current;
0033:     if (!mesh) return;
0034:
0035:     for (let i = 0; i < count; i++) {
0036:         object.position.set(
0037:             gsap.utils.random(-5, 5),
0038:             gsap.utils.random(-3.4, 3.8),
0039:             gsap.utils.random(-2.5, 3.5),
0040:         );
0041:         object.scale.setScalar(gsap.utils.random(0.4, 1.25));
0042:         object.updateMatrix();
0043:         mesh.setMatrixAt(i, object.matrix);
0044:         velocities.current[i] = gsap.utils.random(speed * 0.004, speed * 0.012);
0045:     }
0046:
0047:     mesh.instanceMatrix.needsUpdate = true;
0048: }, [count, speed]);
0049:
0050: useEffect(() => {
0051:     material.color = new THREE.Color(color);
0052: }, [color, material]);
0053:
0054: useFrame(() => {
0055:     const mesh = meshRef.current;
0056:     if (!mesh) return;
0057:
0058:     for (let i = 0; i < count; i++) {
0059:         mesh.getMatrixAt(i, object.matrix);
0060:         object.position.setFromMatrixPosition(object.matrix);
0061:         object.position.y += velocities.current[i];
0062:
0063:         if (object.position.y > 4) {
0064:             object.position.y = -3.2;
0065:             object.position.x = gsap.utils.random(-5, 5);
0066:         }
0067:
0068:         object.updateMatrix();
0069:         mesh.setMatrixAt(i, object.matrix);
0070:     }
0071:
0072:     mesh.instanceMatrix.needsUpdate = true;
0073: });
0074:
0075: return (
0076:     <instancedMesh ref={meshRef} args={[undefined, undefined, count]} material={material}>
0077:         <sphereGeometry args={[0.032, 12, 12]} />
0078:     </instancedMesh>
0079: );
0080: }

```

```

### src/slices/Hero/Scene.tsx

```

Purpose: React Three Fiber hero scene with multi-can intro, scroll-trigger convergence, flavour highlight scaling, camera, lights, bubbles, and environment.

```

0001: "use client";
0002:
0003: import { Environment, PerspectiveCamera } from "@react-three/drei";
0004: import { useFrame, useThree } from "@react-three/fiber";
0005: import { useGSAP } from "@gsap/react";
0006: import gsap from "gsap";
0007: import { ScrollTrigger } from "gsap/ScrollTrigger";
0008: import { useEffect, useMemo, useRef } from "react";
0009: import * as THREE from "three";
0010:
0011: import { SodaCan } from "@components/SodaCan";
0012: import { Bubbles } from "@slices/Hero/Bubbles";
0013: import { CRYO_FLAVORS, getFlavorById, type CryoFlavorId } from "@data/flavors";
0014:
0015: gsap.registerPlugin(useGSAP, ScrollTrigger);

```

```
0016:
0017: const STARTING_ROTATIONS = [-0.5, 0.5, -0.1, 0.35, -0.25] as const;
0018: const ENDING_ROTATIONS = [0.3, 0, -0.1, 0.3, -0.25] as const;
0019:
0020: function toVectorTuple(
0021:   value: readonly [number, number, number],
0022: ): [number, number, number] {
0023:   return [value[0], value[1], value[2]];
0024: }
0025:
0026: type SceneProps = {
0027:   activeFlavorId: CryoFlavorId;
0028:   compact: boolean;
0029:   onIntroComplete?: () => void;
0030: };
0031:
0032: export function Scene({
0033:   activeFlavorId,
0034:   compact,
0035:   onIntroComplete,
0036: }: SceneProps) {
0037:   const rootRef = useRef<THREE.Group>(null);
0038:   const cansGroupRef = useRef<THREE.Group>(null);
0039:   const canRefs = useRef<Array<THREE.Group | null>>([]);
0040:   const entranceRefs = useRef<Array<THREE.Group | null>>([]);
0041:   const { viewport } = useThree();
0042:   const activeFlavor = getFlavorById(activeFlavorId);
0043:
0044:   const entranceIntroStates = useMemo(() => {
0045:     if (compact) {
0046:       return [
0047:         {
0048:           position: [0.85, -4.85, 0] as const,
0049:           rotation: [0, 0, 3] as const,
0050:         },
0051:         {
0052:           position: [0.85, 4.85, 0] as const,
0053:           rotation: [0, 0, 3] as const,
0054:         },
0055:       ] as const;
0056:     }
0057:
0058:     return [
0059:       {
0060:         position: [1, -5, 0] as const,
0061:         rotation: [0, 0, 3] as const,
0062:       },
0063:       {
0064:         position: [1, 5, 0] as const,
0065:         rotation: [0, 0, 3] as const,
0066:       },
0067:     ] as const;
0068:   }, [compact]);
0069:
0070:   const basePosition = useMemo(
0071:     () => ({
0072:       x: 0,
0073:       y: compact ? 0.12 : -0.02,
0074:     }),
0075:     [compact],
0076:   );
0077:
0078:   const startingPositions = useMemo(() => {
0079:     if (compact) {
0080:       return [
0081:         [-1.05, -1.18, 0.16],
0082:         [1.05, -1.04, 0.08],
0083:         [0, 8.7, 2],
0084:         [1.85, 8.2, 2],
0085:         [0, -8.8, 1],
0086:       ] as const;
0087:     }
0088:
0089:     return [
0090:       [-3.45, -0.32, 0.16],
0091:       [3.45, -0.16, 0.08],
0092:       [0, 8.2, 2],
0093:       [2.35, 7.7, 2],
0094:       [0, -8.4, 1],
0095:     ] as const;
0096:   }, [compact]);
0097:
0098:   const endingPositions = useMemo(() => {
```

```

0099:     if (compact) {
0100:         return [
0101:             [-0.18, -0.62, -2],
0102:             [0.72, -0.16, -1],
0103:             [-0.26, 0.42, -1],
0104:             [0.04, -0.25, 0.45],
0105:             [0.28, 0.44, -0.5],
0106:         ] as const;
0107:     }
0108:
0109:     return [
0110:         [-0.2, -0.7, -2],
0111:         [1, -0.2, -1],
0112:         [-0.3, 0.5, -1],
0113:         [0, -0.3, 0.5],
0114:         [0.3, 0.5, -0.5],
0115:     ] as const;
0116: }, [compact]);
0117:
0118: useGSAP(
0119:     () => {
0120:         const cans = canRefs.current.filter(Boolean) as THREE.Group[];
0121:         const entrances = entranceRefs.current.filter(Boolean) as THREE.Group[];
0122:         if (
0123:             !rootRef.current ||
0124:             !cansGroupRef.current ||
0125:             cans.length !== CRYO_FLAVORS.length ||
0126:             entrances.length < 2
0127:         ) {
0128:             return;
0129:         }
0130:
0131:         gsap.set(rootRef.current.position, {
0132:             x: basePosition.x,
0133:             y: basePosition.y,
0134:         });
0135:         gsap.set(rootRef.current.rotation, { y: compact ? -0.03 : -0.08 });
0136:         gsap.set(cansGroupRef.current.position, { x: 0, y: 0, z: 0 });
0137:         gsap.set(cansGroupRef.current.rotation, { x: 0, y: 0, z: 0 });
0138:         gsap.set(
0139:             entrances.slice(0, 2).map((entry) => entry.position),
0140:             {
0141:                 x: 0,
0142:                 y: 0,
0143:                 z: 0,
0144:             },
0145:         );
0146:         gsap.set(
0147:             entrances.slice(0, 2).map((entry) => entry.rotation),
0148:             {
0149:                 x: 0,
0150:                 y: 0,
0151:                 z: 0,
0152:             },
0153:         );
0154:         gsap.set(
0155:             entrances.map((entry) => entry.scale),
0156:             { x: 1, y: 1, z: 1 },
0157:         );
0158:
0159:         cans.forEach((can, index) => {
0160:             gsap.set(can.position, {
0161:                 x: startingPositions[index][0],
0162:                 y: startingPositions[index][1],
0163:                 z: startingPositions[index][2],
0164:             });
0165:             gsap.set(can.rotation, {
0166:                 x: 0,
0167:                 y: 0,
0168:                 z: STARTING_ROTATIONS[index],
0169:             });
0170:         });
0171:
0172:         if (window.scrollY < 20) {
0173:             const intro = gsap.timeline({
0174:                 defaults: { duration: compact ? 2.25 : 2.75, ease: "back.out(1.4)" },
0175:                 onComplete: onIntroComplete,
0176:             });
0177:
0178:             entrances.slice(0, 2).forEach((entry, index) => {
0179:                 const introState = entranceIntroStates[index];
0180:                 const startAt = index * 0.08;
0181:

```

```

0182:         intro
0183:         .fromTo(
0184:             entry.position,
0185:             {
0186:                 x: introState.position[0],
0187:                 y: introState.position[1],
0188:                 z: introState.position[2],
0189:             },
0190:             { x: 0, y: 0, z: 0 },
0191:             startAt,
0192:         )
0193:         .fromTo(
0194:             entry.rotation,
0195:             {
0196:                 x: introState.rotation[0],
0197:                 y: introState.rotation[1],
0198:                 z: introState.rotation[2],
0199:             },
0200:             { x: 0, y: 0, z: 0 },
0201:             startAt,
0202:         )
0203:         .fromTo(
0204:             entry.scale,
0205:             { x: 0.62, y: 0.62, z: 0.62 },
0206:             { x: 1, y: 1, z: 1 },
0207:             startAt,
0208:         );
0209:     });
0210: } else {
0211:     onIntroComplete?.();
0212: }
0213:
0214: const tl = gsap.timeline({
0215:     defaults: { duration: 1.08, ease: "none" },
0216:     scrollTrigger: {
0217:         trigger: ".cryo-hero-section",
0218:         start: "top top",
0219:         end: "bottom bottom",
0220:         scrub: 1.15,
0221:     },
0222: });
0223:
0224: tl.to(cansGroupRef.current.rotation, { y: Math.PI * 2 }, 0);
0225:
0226: cans.forEach((can, index) => {
0227:     tl.to(
0228:         can.position,
0229:         {
0230:             x: endingPositions[index][0],
0231:             y: endingPositions[index][1],
0232:             z: endingPositions[index][2],
0233:         },
0234:         0,
0235:     ).to(
0236:         can.rotation,
0237:         {
0238:             x: 0,
0239:             y: 0,
0240:             z: ENDING_ROTATIONS[index],
0241:         },
0242:         0,
0243:     );
0244: });
0245:
0246: tl.to(
0247:     cansGroupRef.current.position,
0248:     {
0249:         x: compact ? 0.12 : 1.28,
0250:         duration: 0.75,
0251:         ease: "sine.inOut",
0252:     },
0253:     1.18,
0254: );
0255: },
0256: {
0257:     dependencies: [
0258:         basePosition,
0259:         compact,
0260:         endingPositions,
0261:         entranceIntroStates,
0262:         onIntroComplete,
0263:         startingPositions,
0264:     ],

```

```

0265:     },
0266:   );
0267:
0268:   useEffect(() => {
0269:     entranceRefs.current.forEach((entry, index) => {
0270:       if (!entry) return;
0271:       const flavor = CRYO_FLAVORS[index];
0272:       const isActive = flavor.id === activeFlavorId;
0273:
0274:       gsap.to(entry.scale, {
0275:         x: isActive ? 1.14 : 1,
0276:         y: isActive ? 1.14 : 1,
0277:         z: isActive ? 1.14 : 1,
0278:         duration: 0.55,
0279:         ease: "power2.out",
0280:       });
0281:     });
0282:   }, [activeFlavorId]);
0283:
0284:   useFrame(({ clock }) => {
0285:     if (!rootRef.current) return;
0286:
0287:     rootRef.current.position.x = basePosition.x;
0288:     rootRef.current.position.y =
0289:       basePosition.y + Math.sin(clock.elapsedTime * 0.72) * 0.04;
0290:   });
0291:
0292:   const scale = Math.min(
0293:     viewport.width / (compact ? 5.2 : 8.9),
0294:     compact ? 0.78 : 0.88,
0295:   );
0296:
0297:   return (
0298:     <
0299:       <PerspectiveCamera
0300:         makeDefault
0301:         position={[0, compact ? 0.4 : 0.12, compact ? 7.4 : 6.8]}
0302:         fov={compact ? 36 : 32}
0303:       />
0304:
0305:       <ambientLight intensity={2.2} color={activeFlavor.colors.secondary} />
0306:       <spotLight
0307:         position={[-4, 4, 5]}
0308:         intensity={30}
0309:         color={activeFlavor.colors.secondary}
0310:         penumbra={0.72}
0311:         castShadow
0312:       />
0313:       <pointLight
0314:         position={[3, -1, 3]}
0315:         intensity={16}
0316:         color={activeFlavor.colors.primary}
0317:       />
0318:
0319:       <Bubbles color={activeFlavor.colors.primary} count={compact ? 90 : 150} />
0320:
0321:       <group ref={rootRef} scale={scale}>
0322:         <group ref={cansGroupRef}>
0323:           {CRYO_FLAVORS.map((flavor, index) => (
0324:             <group
0325:               key={flavor.id}
0326:               ref={(node) => {
0327:                 entranceRefs.current[index] = node;
0328:               }}
0329:               position={
0330:                 index < 2
0331:                   ? toVectorTuple(entranceIntroStates[index].position)
0332:                   : [0, 0, 0]
0333:               }
0334:               rotation={
0335:                 index < 2
0336:                   ? toVectorTuple(entranceIntroStates[index].rotation)
0337:                   : [0, 0, 0]
0338:               }
0339:               scale={index < 2 ? 0.62 : 1}
0340:             >
0341:               <group
0342:                 ref={(node) => {
0343:                   canRefs.current[index] = node;
0344:                 }}
0345:                 position={toVectorTuple(startingPositions[index])}
0346:                 rotation={[0, 0, STARTING_ROTATIONS[index]]}
0347:               >

```

```

0348:         <SodaCan flavor={flavor.id} canScale={compact ? 2.08 : 2.55} />
0349:     </group>
0350: </group>
0351: )))
0352: </group>
0353: </group>
0354:
0355:     <Environment files="/hdr/lobby.hdr" environmentIntensity={1.25} />
0356: </>
0357: );
0358: }

```

```

### src/slices/Hero/index.tsx

```

Purpose: Client landing UI that manages active flavour state, GSAP text reveal, intro readiness, 3D canvas, and flavour picker.

```

0001: "use client";
0002:
0003: import { Canvas } from "@react-three/fiber";
0004: import { useGSAP } from "@gsap/react";
0005: import gsap from "gsap";
0006: import { ScrollTrigger } from "gsap/ScrollTrigger";
0007: import Link from "next/link";
0008: import { Suspense, useCallback, useMemo, useState } from "react";
0009: import clsx from "clsx";
0010:
0011: import { Bounded } from "@components/Bounded";
0012: import {
0013:   CRYO_FLAVORS,
0014:   DEFAULT_FLAVOR_ID,
0015:   getFlavorById,
0016:   type CryoFlavorId,
0017: } from "@data/flavors";
0018: import { CryoLogo } from "@components/CryoLogo";
0019: import { useMediaQuery } from "@hooks/useMediaQuery";
0020: import { Scene } from "@slices/Hero/Scene";
0021:
0022: gsap.registerPlugin(useGSAP, ScrollTrigger);
0023:
0024: export default function Hero() {
0025:   const [activeFlavorId, setActiveFlavorId] =
0026:     useState<CryoFlavorId>(DEFAULT_FLAVOR_ID);
0027:   const [introReady, setIntroReady] = useState(false);
0028:   const compact = useMediaQuery("(max-width: 767px)", false);
0029:   const activeFlavor = getFlavorById(activeFlavorId);
0030:   const handleSceneIntroComplete = useCallback(() => {
0031:     setIntroReady(true);
0032:   }, []);
0033:
0034:   const style = useMemo(
0035:     () =>
0036:     ({
0037:       "--flavor-primary": activeFlavor.colors.primary,
0038:       "--flavor-secondary": activeFlavor.colors.secondary,
0039:       "--flavor-text": activeFlavor.colors.text,
0040:     }) as React.CSSProperties,
0041:     [activeFlavor],
0042:   );
0043:
0044:   useGSAP(() => {
0045:     const mm = gsap.matchMedia();
0046:
0047:     mm.add("(prefers-reduced-motion: no-preference)", () => {
0048:       gsap.set(".cryo-second-panel", { opacity: 0, y: 48 });
0049:       gsap.set(".cryo-second-word", {
0050:         opacity: 0,
0051:         y: 34,
0052:         rotate: -12,
0053:         scale: 1.18,
0054:       });
0055:       gsap.set(".cryo-second-copy", { opacity: 0, y: 22 });
0056:       gsap.set(".cryo-flavor-picker", { autoAlpha: 0, y: 40 });
0057:
0058:       const tl = gsap.timeline({
0059:         scrollTrigger: {
0060:           trigger: ".cryo-hero-section",
0061:           start: "top top",
0062:           end: "bottom bottom",
0063:           scrub: 1.15,

```

```

0064:     },
0065:   });
0066:
0067:   tl.to(".cryo-intro-panel", { opacity: 0, y: -56, duration: 0.32 }, 0.22)
0068:   .to(
0069:     ".cryo-second-panel",
0070:     { opacity: 1, y: 0, duration: 0.25, ease: "power2.out" },
0071:     1.16,
0072:   )
0073:   .to(
0074:     ".cryo-second-word",
0075:     {
0076:       opacity: 1,
0077:       y: 0,
0078:       rotate: 0,
0079:       scale: 1,
0080:       duration: 0.45,
0081:       ease: "back.out(2.4)",
0082:       stagger: 0.08,
0083:     },
0084:     1.25,
0085:   )
0086:   .to(
0087:     ".cryo-second-copy",
0088:     { opacity: 1, y: 0, duration: 0.35, ease: "power2.out" },
0089:     1.56,
0090:   )
0091:   .to(
0092:     ".cryo-flavor-picker",
0093:     {
0094:       autoAlpha: 1,
0095:       y: 0,
0096:       duration: 0.32,
0097:       ease: "power2.out",
0098:     },
0099:     1.82,
0100:   );
0101: });
0102:
0103: mm.add("(prefers-reduced-motion: reduce)", () => {
0104:   gsap.set(
0105:     ".cryo-intro-panel, .cryo-second-panel, .cryo-second-word, .cryo-second-copy, .cryo-flavor-picker",
0106:     {
0107:       opacity: 1,
0108:       y: 0,
0109:       rotate: 0,
0110:       scale: 1,
0111:     },
0112:   );
0113: });
0114:
0115: return () => mm.revert();
0116: }, []);
0117:
0118: return (
0119:   <section
0120:     className="cryo-hero-section cryo-hero-bg relative min-h-[275dvh] overflow-clip text-[#f5f7ff] transition-
0121: colors duration-700"
0122:     style={style}
0123:   >
0124:     <div className="sticky top-0 h-dvh overflow-hidden">
0125:       <div className="cryo-frost pointer-events-none absolute inset-0 opacity-35" />
0126:
0127:       <div className="absolute inset-0 z-0">
0128:         <Canvas
0129:           shadows
0130:           dpr={[1, 1.5]}
0131:           gl={{ antialias: true, alpha: true }}
0132:           className="h-full w-full"
0133:         >
0134:           <Suspense fallback={null}>
0135:             <Scene
0136:               activeFlavorId={activeFlavorId}
0137:               compact={compact}
0138:               onIntroComplete={handleSceneIntroComplete}
0139:             />
0140:           </Suspense>
0141:         </Canvas>
0142:       </div>
0143:
0144:       <Bounded className="relative z-10 flex min-h-dvh items-center justify-center pb-36 pt-10 md:pb-32
md:pt-14">
0145:         <div className="relative grid w-full place-items-center text-center">

```



```

0145:         <div className="cryo-intro-panel mx-auto grid w-full max-w-4xl place-items-center">
0146:             <div
0147:                 className={clsx(
0148:                     "cryo-intro-content grid w-full place-items-center transition duration-700 ease-out",
0149:                     introReady
0150:                     ? "translate-y-0 opacity-100"
0151:                     : "pointer-events-none translate-y-6 opacity-0",
0152:                 )}
0153:             >
0154:                 <h1 className="grid w-[min(74vw,30rem)] place-items-center">
0155:                     <CryoLogo className="h-auto w-full text-[#f5f7ff] drop-
shadow-[0_18px_44px_rgba(245,247,255,0.16)]" />
0156:                 </h1>
0157:
0158:                 <p className="mt-9 text-balance text-5xl font-black uppercase leading-[0.86] tracking-normal
text-[#f5f7ff] md:text-7xl lg:text-8xl">
0159:                     Sip the Chillwave
0160:                 </p>
0161:                 <p className="mt-6 max-w-2xl text-pretty text-lg font-bold leading-snug text-[#f5f7ff]
md:text-2xl">
0162:                     Cryo is crafted to deliver an explosive fusion of freshness,
0163:                     bold flavors and a mixture of fruits.
0164:                 </p>
0165:             </div>
0166:         </div>
0167:
0168:         <div className="cryo-second-panel pointer-events-none absolute inset-x-0 top-[7dvh] mx-auto
max-w-[23rem] px-4 text-center opacity-0 md:inset-x-auto md:left-0 md:top-[10dvh] md:max-w-[36rem] md:px-0 md:text-
left lg:max-w-[41rem]">
0169:             <p className="mb-3 text-sm font-black uppercase tracking-[0.28em] text-[#f5f7ff] md:text-base">
0170:                 Try out flavours
0171:             </p>
0172:             <h2 className="text-balance text-4xl font-black uppercase leading-[0.86] tracking-normal
text-[#f5f7ff] md:text-6xl lg:text-7xl">
0173:                 {["Fruit", "flavour", "with", "ice-cold", "fizz."].map(
0174:                     (word) => (
0175:                         <span
0176:                             key={word}
0177:                             className="cryo-second-word mr-[0.22em] inline-block"
0178:                         >
0179:                             {word}
0180:                         </span>
0181:                     ),
0182:                 )}
0183:             </h2>
0184:             <p className="cryo-second-copy mt-6 max-w-xl text-pretty text-base font-bold leading-snug
text-[#f5f7ff] md:text-xl">
0185:                 Berry, melon, grape, citrus, and velvet peach snap into one cold
0186:                 lineup. Pick a flavor below and open its freeze-burst story.
0187:             </p>
0188:         </div>
0189:     </div>
0190: </Bounded>
0191: </div>
0192:
0193: <FlavorPicker
0194:     activeFlavorId={activeFlavorId}
0195:     onSelect={setActiveFlavorId}
0196:     ready={introReady}
0197: />
0198: </section>
0199: );
0200: }
0201:
0202: type FlavorPickerProps = {
0203:     activeFlavorId: CryoFlavorId;
0204:     onSelect: (flavorId: CryoFlavorId) => void;
0205:     ready: boolean;
0206: };
0207:
0208: function FlavorPicker({ activeFlavorId, onSelect, ready }: FlavorPickerProps) {
0209:     return (
0210:         <div
0211:             className={clsx(
0212:                 "cryo-flavor-picker invisible fixed inset-x-0 bottom-4 z-40 px-4 opacity-0 md:bottom-6",
0213:                 ready ? "" : "pointer-events-none translate-y-4 opacity-0",
0214:             )}
0215:         >
0216:             <div className="bg-[#0b0f14]/72 mx-auto max-w-6xl overflow-x-auto rounded-lg border border-[#f5f7ff]/25
p-2 shadow-2xl shadow-black/35 backdrop-blur-xl">
0217:                 <ul className="grid min-w-[48rem] grid-cols-5 gap-2 md:min-w-0">
0218:                     {CRYO_FLAVORS.map((flavor) => {
0219:                         const isActive = flavor.id === activeFlavorId;

```

```

0220:
0221:         return (
0222:             <li key={flavor.id}>
0223:                 <Link
0224:                     href={`/${flavours?flavor=${flavor.id}`}
0225:                     onMouseEnter={() => onSelect(flavor.id)}
0226:                     onFocus={() => onSelect(flavor.id)}
0227:                     aria-label={`Open ${flavor.name} flavor page`}
0228:                     className={clsx(
0229:                         "grid h-full w-full grid-cols-[auto,1fr] items-center gap-3 rounded-lg border p-3 text-left
transition duration-200",
0230:                         isActive
0231:                         ? "border-[#f5f7ff] bg-[#f5f7ff] text-[#0b0f14] shadow-lg"
0232:                         : "bg-[#f5f7ff]/8 hover:bg-[#f5f7ff]/14 border-[#f5f7ff]/25 text-[#f5f7ff]
hover:border-[#f5f7ff]/55",
0233:                     )}
0234:                 >
0235:                     <span className="grid size-11 grid-cols-2 overflow-hidden rounded-md border border-black/10">
0236:                         <span style={{ backgroundColor: flavor.colors.primary }} />
0237:                         <span
0238:                             style={{ backgroundColor: flavor.colors.secondary }}
0239:                         />
0240:                     </span>
0241:                     <span className="min-w-0">
0242:                         <span className="block truncate text-sm font-black uppercase md:text-base">
0243:                             {flavor.name}
0244:                         </span>
0245:                         <span
0246:                             className={clsx(
0247:                                 "block truncate text-xs font-semibold",
0248:                                 isActive ? "text-[#0b0f14]/70" : "text-[#f5f7ff]/72",
0249:                             )}
0250:                         >
0251:                             {flavor.flavor}
0252:                         </span>
0253:                     </span>
0254:                 </Link>
0255:             </li>
0256:         );
0257:     }
0258: }
0259: </div>
0260: </div>
0261: );
0262: }

```

```

### src/slices/Reviews/Scene.tsx

```

Purpose: Reviews 3D scene. Adds mouse-reactive camera and scroll-driven Nimbus key wave animation.

```

0001: "use client";
0002:
0003: import { Environment, PerspectiveCamera } from "@react-three/drei";
0004: import { useFrame, useThree } from "@react-three/fiber";
0005: import { useGSAP } from "@gsap/react";
0006: import gsap from "gsap";
0007: import { ScrollTrigger } from "gsap/ScrollTrigger";
0008: import { useEffect, useMemo, useRef } from "react";
0009: import * as THREE from "three";
0010:
0011: import {
0012:     NimbusKeyboard,
0013:     type NimbusKeyboardRefs,
0014: } from "@components/NimbusKeyboard";
0015:
0016: gsap.registerPlugin(useGSAP, ScrollTrigger);
0017:
0018: const KEYBOARD_COLUMNS = [
0019:     ["esc", "grave", "tab", "caps", "lshift", "lcontrol"],
0020:     ["f1", "one", "q", "a", "z", "lalt"],
0021:     ["f2", "two", "w", "s", "x", "lwin"],
0022:     ["f3", "three", "e", "d", "c"],
0023:     ["f4", "four", "r", "f", "v"],
0024:     ["f5", "five", "t", "g", "b", "space"],
0025:     ["f6", "six", "y", "h", "n"],
0026:     ["f7", "seven", "u", "j", "m"],
0027:     ["f8", "eight", "i", "k", "comma"],
0028:     ["f9", "nine", "o", "l", "period"],
0029:     ["f10", "zero", "dash", "p", "semicolon", "slash", "ralt"],
0030: ]

```

```

0031:     "f11",
0032:     "lsquarebracket",
0033:     "quote",
0034:     "rshift",
0035:     "fn",
0036:     "arrowleft",
0037:     "rsquarebracket",
0038:     "enter",
0039:     "f12",
0040:     "equal",
0041:     "arrowup",
0042: ],
0043: [],
0044: [
0045:     "del",
0046:     "backspace",
0047:     "backslash",
0048:     "pagedown",
0049:     "end",
0050:     "arrowdown",
0051:     "pageup",
0052:     "arrowright",
0053: ],
0054: [],
0055: ];
0056:
0057: function CameraController() {
0058:     const { camera, size } = useThree();
0059:     const mouseRef = useRef({ x: 0.5, y: 0.5 });
0060:     const reducedMotionRef = useRef(false);
0061:     const targetRef = useRef(new THREE.Vector3(0, 0, 0));
0062:     const currentPositionRef = useRef(new THREE.Vector3(0, 0, 4));
0063:     const baseCameraPosition = useMemo(
0064:         () => ({
0065:             x: 0,
0066:             y: 0,
0067:             z: 4,
0068:         }),
0069:         [],
0070:     );
0071:
0072:     useFrame(() => {
0073:         if (reducedMotionRef.current) {
0074:             camera.position.set(
0075:                 baseCameraPosition.x,
0076:                 baseCameraPosition.y,
0077:                 baseCameraPosition.z,
0078:             );
0079:             camera.lookAt(targetRef.current);
0080:             return;
0081:         }
0082:
0083:         const mouse = mouseRef.current;
0084:         const tiltX = (mouse.y - 0.5) * 0.3;
0085:         const tiltY = (mouse.x - 0.5) * 0.3;
0086:         const targetPosition = new THREE.Vector3(
0087:             baseCameraPosition.x + tiltY,
0088:             baseCameraPosition.y - tiltX,
0089:             baseCameraPosition.z,
0090:         );
0091:
0092:         currentPositionRef.current.lerp(targetPosition, 0.1);
0093:         camera.position.copy(currentPositionRef.current);
0094:         camera.lookAt(targetRef.current);
0095:     });
0096:
0097:     useEffect(() => {
0098:         const prefersReducedMotion = window.matchMedia(
0099:             "(prefers-reduced-motion: reduce)",
0100:         );
0101:         reducedMotionRef.current = prefersReducedMotion.matches;
0102:
0103:         const handleMotionPreference = (event: MediaQueryListEvent) => {
0104:             reducedMotionRef.current = event.matches;
0105:         };
0106:
0107:         const handleMouseMove = (event: MouseEvent) => {
0108:             mouseRef.current.x = event.clientX / size.width;
0109:             mouseRef.current.y = event.clientY / size.height;
0110:         };
0111:
0112:         prefersReducedMotion.addEventListener("change", handleMotionPreference);
0113:         window.addEventListener("mousemove", handleMouseMove);

```

```

0114:
0115:     return () => {
0116:         prefersReducedMotion.removeEventListener(
0117:             "change",
0118:             handleMotionPreference,
0119:         );
0120:         window.removeEventListener("mousemove", handleMouseMove);
0121:     };
0122: }, [size.height, size.width]);
0123:
0124: return null;
0125: }
0126:
0127: export function ReviewsKeyboardScene() {
0128:     const keyboardGroupRef = useRef<THREE.Group>(null);
0129:     const keyboardAnimationRef = useRef<NimbusKeyboardRefs>(null);
0130:     const { size } = useThree();
0131:     const scalingFactor = size.width <= 500 ? 0.68 : 0.96;
0132:
0133:     useGSAP(
0134:         () => {
0135:             const mm = gsap.matchMedia();
0136:
0137:             mm.add("(prefers-reduced-motion: no-preference)", () => {
0138:                 const keyboard = keyboardGroupRef.current;
0139:                 const keyboardRefs = keyboardAnimationRef.current;
0140:                 if (!keyboard || !keyboardRefs) return;
0141:
0142:                 gsap.set(keyboard.position, {
0143:                     x: 0.2,
0144:                     y: -0.5,
0145:                     z: 1.9,
0146:                 });
0147:                 gsap.set(keyboard.rotation, {
0148:                     x: 1.6,
0149:                     y: 0.4,
0150:                     z: 0,
0151:                 });
0152:
0153:                 const scrollTimeline = gsap.timeline({
0154:                     scrollTrigger: {
0155:                         trigger: ".reviews-section",
0156:                         start: "top top",
0157:                         end: "bottom bottom",
0158:                         scrub: 1,
0159:                     },
0160:                 });
0161:
0162:                 scrollTimeline
0163:                     .to(keyboard.position, {
0164:                         x: 0,
0165:                         y: -0.5,
0166:                         z: 2.2,
0167:                     })
0168:                     .to(
0169:                         keyboard.rotation,
0170:                         {
0171:                             x: Math.PI * -2 + 0.8,
0172:                             y: 0,
0173:                             z: 0,
0174:                         },
0175:                         "<",
0176:                     );
0177:
0178:                 addNimbusKeyWave(scrollTimeline, keyboardRefs);
0179:
0180:                 scrollTimeline.to({}, { duration: 0.72 }, 1.76);
0181:             });
0182:
0183:             mm.add("(prefers-reduced-motion: reduce)", () => {
0184:                 const keyboard = keyboardGroupRef.current;
0185:                 if (!keyboard) return;
0186:
0187:                 keyboard.position.set(0, -0.5, 1.9);
0188:                 keyboard.rotation.set(1.6, 0.4, 0);
0189:             });
0190:
0191:             return () => mm.revert();
0192:         },
0193:         { dependencies: [] },
0194:     );
0195:
0196:     return (

```

```

0197:     <>
0198:     <CameraController />
0199:     <PerspectiveCamera makeDefault position=[[0, 0, 4]] fov={50} />
0200:
0201:     <group scale={scalingFactor}>
0202:         <group ref={keyboardGroupRef}>
0203:             <NimbusKeyboard scale={9} ref={keyboardAnimationRef} />
0204:         </group>
0205:     </group>
0206:
0207:     <Environment
0208:         files={["/nimbus/hdr/blue-studio.hdr"]}
0209:         environmentIntensity={0.3}
0210:     />
0211:
0212:     <spotLight
0213:         position=[[-2, 1.5, 3]]
0214:         intensity={44}
0215:         castShadow
0216:         penumbra={0}
0217:         shadow-bias={-0.0002}
0218:         shadow-normalBias={0.002}
0219:         shadow-mapSize={1024}
0220:     />
0221: </>
0222: );
0223: }
0224:
0225: function addNimbusKeyWave(
0226:     scrollTimeline: gsap.core.Timeline,
0227:     keyboardRefs: NimbusKeyboardRefs,
0228: ) {
0229:     const switchRefs = keyboardRefs.switches;
0230:     const individualKeys = keyboardRefs.keys;
0231:     const allSwitches: THREE.Object3D[] = [];
0232:
0233:     [
0234:         switchRefs.functionRow.current,
0235:         switchRefs.numberRow.current,
0236:         switchRefs.topRow.current,
0237:         switchRefs.homeRow.current,
0238:         switchRefs.bottomRow.current,
0239:         switchRefs.modifiers.current,
0240:         switchRefs.arrows.current,
0241:     ].forEach((row) => {
0242:         if (row) {
0243:             allSwitches.push(...Array.from(row.children));
0244:         }
0245:     });
0246:
0247:     const keyCapsByColumn: THREE.Mesh[][] = [];
0248:     const switchesByColumn: THREE.Object3D[][] = [];
0249:     const sortedSwitches = allSwitches.sort(
0250:         (a, b) => a.position.x - b.position.x,
0251:     );
0252:
0253:     KEYBOARD_COLUMNS.forEach((column, columnIndex) => {
0254:         const columnKeycaps: THREE.Mesh[] = [];
0255:         const columnSwitches: THREE.Object3D[] = [];
0256:
0257:         column.forEach((keyName) => {
0258:             const keycap = keyName ? individualKeys[keyName]?.current : null;
0259:             if (keycap) {
0260:                 columnKeycaps.push(keycap);
0261:             }
0262:         });
0263:
0264:         const switchesPerColumn = Math.ceil(
0265:             sortedSwitches.length / KEYBOARD_COLUMNS.length,
0266:         );
0267:         const startIndex = columnIndex * switchesPerColumn;
0268:         const endIndex = Math.min(
0269:             startIndex + switchesPerColumn,
0270:             sortedSwitches.length,
0271:         );
0272:
0273:         for (let i = startIndex; i < endIndex; i += 1) {
0274:             if (sortedSwitches[i]) {
0275:                 columnSwitches.push(sortedSwitches[i]);
0276:             }
0277:         }
0278:
0279:         keyCapsByColumn.push(columnKeycaps);

```

```
0280:     switchesByColumn.push(columnSwitches);
0281:   });
0282:
0283:   keyCapsByColumn.forEach((columnKeycaps, columnIndex) => {
0284:     const columnSwitches = switchesByColumn[columnIndex];
0285:
0286:     if (columnKeycaps.length === 0 && columnSwitches.length === 0) return;
0287:
0288:     const waveProgress = columnIndex / (KEYBOARD_COLUMNS.length - 1);
0289:     const waveStartTime = waveProgress * 1.08 + 0.18;
0290:
0291:     if (columnKeycaps.length > 0) {
0292:       const keycapPositions = columnKeycaps.map((keycap) => keycap.position);
0293:
0294:       scrollTimeline.to(
0295:         keycapPositions,
0296:         {
0297:           y: "+=0.08",
0298:           duration: 0.29,
0299:           ease: "power2.inOut",
0300:         },
0301:         waveStartTime,
0302:       );
0303:
0304:       scrollTimeline.to(
0305:         keycapPositions,
0306:         {
0307:           y: "-=0.08",
0308:           duration: 0.5,
0309:           ease: "power2.inOut",
0310:         },
0311:         waveStartTime + 0.29,
0312:       );
0313:     }
0314:
0315:     if (columnSwitches.length > 0) {
0316:       const switchPositions = columnSwitches.map(
0317:         (switchObj) => switchObj.position,
0318:       );
0319:
0320:       scrollTimeline.to(
0321:         switchPositions,
0322:         {
0323:           y: "+=0.04",
0324:           duration: 0.2,
0325:           ease: "power2.inOut",
0326:         },
0327:         waveStartTime + 0.12,
0328:       );
0329:
0330:       scrollTimeline.to(
0331:         switchPositions,
0332:         {
0333:           y: "-=0.04",
0334:           duration: 0.2,
0335:           ease: "power2.inOut",
0336:         },
0337:         waveStartTime + 0.29,
0338:       );
0339:     }
0340:   });
0341: }
```

src/slices/Reviews/index.tsx

Purpose: Reviews page UI. Types the heading, shows blinking cursor, animates keyboard in/out, and reveals review cards.

```
0001: "use client";
0002:
0003: import { Canvas } from "@react-three/fiber";
0004: import { useGSAP } from "@gsap/react";
0005: import gsap from "gsap";
0006: import { ScrollTrigger } from "gsap/ScrollTrigger";
0007: import { Suspense, useRef } from "react";
0008:
0009: import { Bounded } from "@components/Bounded";
0010: import { ReviewsKeyboardScene } from "@slices/Reviews/Scene";
0011:
0012: gsap.registerPlugin(useGSAP, ScrollTrigger);
```

```
0013:
0014: const reviews = [
0015:   {
0016:     quote:
0017:       "Cryo became the drink our team reaches for before every late-night build. It tastes bright, stays crisp,
and the cans look unreal on a desk.",
0018:     name: "Maya Chen",
0019:     role: "Creative director, Northline Studio",
0020:   },
0021:   {
0022:     quote:
0023:       "Frostbite Berry is sharp without turning syrupy. It feels like a proper cold soda instead of another
heavy energy drink.",
0024:     name: "Andre Patel",
0025:     role: "Product designer",
0026:   },
0027:   {
0028:     quote:
0029:       "The flavours hit fast, the finish stays clean, and the can design makes the whole thing feel premium.",
0030:     name: "Sofia Reyes",
0031:     role: "Cafe owner",
0032:   },
0033:   {
0034:     quote:
0035:       "Cosmic Crush is the one I keep buying. Grape, kiwi, cold fizz, no weird aftertaste.",
0036:     name: "Noah Brooks",
0037:     role: "Motion artist",
0038:   },
0039: ];
0040:
0041: const REVIEW_HEADING = "Learn what our customers have to say about us.";
0042:
0043: export default function Reviews() {
0044:   const typedHeadingRef = useRef<HTMLSpanElement>(null);
0045:
0046:   useGSAP(() => {
0047:     const mm = gsap.matchMedia();
0048:
0049:     mm.add("(prefers-reduced-motion: no-preference)", () => {
0050:       const headingText = typedHeadingRef.current;
0051:       const typeState = { count: 0 };
0052:
0053:       gsap.set(".reviews-keyboard-wrap", { opacity: 0, y: 54, scale: 0.96 });
0054:       gsap.set(".reviews-card", { opacity: 0, y: 72, scale: 0.94 });
0055:       gsap.set(".reviews-heading-wrap", { opacity: 1, y: 0, scale: 1 });
0056:
0057:       const updateHeadingText = () => {
0058:         if (!headingText) return;
0059:         headingText.textContent = REVIEW_HEADING.slice(
0060:           0,
0061:           Math.round(typeState.count),
0062:         );
0063:       };
0064:
0065:       updateHeadingText();
0066:
0067:       const tl = gsap.timeline({
0068:         scrollTrigger: {
0069:           trigger: ".reviews-section",
0070:           start: "top top",
0071:           end: "bottom bottom",
0072:           scrub: 1,
0073:         },
0074:       });
0075:
0076:       tl.to(
0077:         typeState,
0078:         {
0079:           count: REVIEW_HEADING.length,
0080:           duration: 0.48,
0081:           ease: "none",
0082:           snap: { count: 1 },
0083:           onUpdate: updateHeadingText,
0084:         },
0085:         0.04,
0086:       )
0087:       .to(
0088:         ".reviews-keyboard-wrap",
0089:         {
0090:           opacity: 1,
0091:           y: 0,
0092:           scale: 1,
0093:           duration: 0.2,
```

```

0094:         ease: "power2.out",
0095:     },
0096:     0.02,
0097: )
0098: .to(
0099:     ".reviews-keyboard-wrap",
0100:     {
0101:         opacity: 1,
0102:         duration: 0.48,
0103:     },
0104:     0.18,
0105: )
0106: .to(
0107:     ".reviews-keyboard-wrap",
0108:     {
0109:         opacity: 0,
0110:         y: -92,
0111:         scale: 1.08,
0112:         duration: 0.12,
0113:         ease: "power2.in",
0114:     },
0115:     0.72,
0116: )
0117: .to(
0118:     ".reviews-heading-wrap",
0119:     {
0120:         opacity: 0,
0121:         y: -128,
0122:         scale: 0.96,
0123:         duration: 0.14,
0124:         ease: "power2.in",
0125:     },
0126:     0.86,
0127: )
0128: .to(
0129:     ".reviews-card",
0130:     {
0131:         opacity: 1,
0132:         y: 0,
0133:         scale: 1,
0134:         duration: 0.2,
0135:         ease: "power2.out",
0136:         stagger: 0.05,
0137:     },
0138:     1,
0139: );
0140: });
0141:
0142: mm.add("(prefers-reduced-motion: reduce)", () => {
0143:     if (typedHeadingRef.current) {
0144:         typedHeadingRef.current.textContent = REVIEW_HEADING;
0145:     }
0146:
0147:     gsap.set(".reviews-heading-wrap, .reviews-keyboard-wrap, .reviews-card", {
0148:         opacity: 1,
0149:         y: 0,
0150:         scale: 1,
0151:     });
0152: });
0153:
0154: return () => mm.revert();
0155: }, []);
0156:
0157: return (
0158:     <section className="reviews-section relative h-[285dvh] overflow-clip bg-[#0b0f14] text-[#f5f7ff]">
0159:         <div className="sticky top-0 min-h-dvh overflow-hidden">
0160:             <div className="pointer-events-none absolute inset-0 bg-[radial-gradient(circle_at_50%_20%, rgba(245,247,255,0.1), transparent_28rem)]" />
0161:             <div className="pointer-events-none absolute inset-0 bg-[linear-gradient(rgba(245,247,255,0.045)_1px, transparent_1px), linear-gradient(90deg, rgba(245,247,255,0.045)_1px, transparent_1px)] bg-[size:44px_44px]" />
0162:
0163:             <Bounded className="reviews-heading-wrap relative z-20 pt-14 text-center will-change-transform md:pt-20">
0164:                 <p className="text-sm font-black uppercase tracking-[0.36em] text-[#f5f7ff]/70 md:text-base">
0165:                     Reviews
0166:                 </p>
0167:                 <h2
0168:                     aria-label={REVIEW_HEADING}
0169:                     className="mx-auto mt-4 max-w-5xl text-balance text-4xl font-black uppercase leading-[0.88] tracking-normal text-[#f5f7ff] md:text-6xl lg:text-7xl"
0170:                 >
0171:                     <span ref={typedHeadingRef} aria-hidden="true" />

```



```

0172:         <span
0173:             aria-hidden="true"
0174:             className="reviews-typing-cursor ml-[0.08em] inline-block h-[0.82em] w-[0.08em]
translate-y-[0.08em] bg-[#f5f7ff]"
0175:         />
0176:     </h2>
0177: </Bounded>
0178:
0179:     <div className="reviews-keyboard-wrap pointer-events-none absolute inset-x-0 top-[24dvh] z-10 h-[58dvh]
opacity-0 md:top-[32dvh] md:h-[60dvh]">
0180:         <Canvas
0181:             shadows
0182:             dpr={[1, 1.5]}
0183:             gl={{ antialias: true, alpha: true }}
0184:             className="h-full w-full"
0185:         >
0186:             <Suspense fallback={null}>
0187:                 <ReviewsKeyboardScene />
0188:             </Suspense>
0189:         </Canvas>
0190:     </div>
0191:
0192:     <Bounded className="pointer-events-none absolute inset-x-0 bottom-[5dvh] z-30 md:bottom-[7dvh]">
0193:         <div className="mx-auto grid w-full max-w-6xl gap-3 md:grid-cols-2 lg:grid-cols-4">
0194:             {reviews.map((review) => (
0195:                 <article
0196:                     key={review.name}
0197:                     className="reviews-card border-[#f5f7ff]/18 rounded-lg border bg-[#f5f7ff]/10 p-4 text-center
opacity-0 shadow-2xl shadow-black/40 backdrop-blur-xl md:p-5"
0198:                 >
0199:                     <div className="mb-3 flex justify-center gap-1 text-base text-[#f5f7ff] md:text-lg">
0200:                         {Array.from({ length: 5 }).map((_, index) => (
0201:                             <span key={index} aria-hidden="true">
0202:                                 *
0203:                             </span>
0204:                         ))}
0205:                     </div>
0206:                     <blockquote className="text-balance text-sm font-black uppercase leading-tight text-[#f5f7ff]
md:text-base">
0207:                         <span aria-hidden="true">&quot;</span>
0208:                         {review.quote}
0209:                         <span aria-hidden="true">&quot;</span>
0210:                     </blockquote>
0211:                     <p className="mt-4 text-xs font-black uppercase tracking-[0.2em] text-[#f5f7ff]">
0212:                         {review.name}
0213:                     </p>
0214:                     <p className="text-[#f5f7ff]/68 mt-1 text-xs font-semibold">
0215:                         {review.role}
0216:                     </p>
0217:                 </article>
0218:             ))}
0219:         </div>
0220:     </Bounded>
0221: </div>
0222: </section>
0223: );
0224: }

```

```

### src/types/model-viewer.d.ts

```

Purpose: Extends JSX so TypeScript accepts the custom `<model-viewer>` element and its AR attributes.

```

0001: import type React from "react";
0002:
0003: declare global {
0004:     namespace JSX {
0005:         interface IntrinsicElements {
0006:             "model-viewer": React.DetailedHTMLProps<
0007:                 React.HTMLAttributes<HTMLDivElement>,
0008:                 HTMLDivElement
0009:             > & {
0010:                 alt?: string;
0011:                 ar?: boolean | string;
0012:                 "ar-modes"?: string;
0013:                 "ar-placement"?: string;
0014:                 "ar-scale"?: string;
0015:                 "auto-rotate"?: boolean | string;
0016:                 "camera-controls"?: boolean | string;
0017:                 "camera-orbit"?: string;
0018:                 exposure?: string;

```

```
0019:         "environment-image"?: string;
0020:         "field-of-view"?: string;
0021:         "interaction-prompt"?: string;
0022:         "max-camera-orbit"?: string;
0023:         "min-camera-orbit"?: string;
0024:         "quick-look-browsers"?: string;
0025:         "shadow-intensity"?: string;
0026:         "shadow-softness"?: string;
0027:         scale?: string;
0028:         src?: string;
0029:         "touch-action"?: string;
0030:         "xr-environment"?: boolean | string;
0031:     };
0032: }
0033: }
0034: }
```